

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Курсова робота з освітнього компоненту
«Комп’ютерне моделювання дискретно-подійних систем.
Курсова робота»

Тема: Імітаційна модель роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування

Керівник:
д.т.н., професор
Стеценко Інна Вячеславівна

Виконавець:
Понзель Тетяна Юріївна
студентка групи ІП-31
залікова книжка №

«Допущено до захисту»
_____

«___» _____ 2026 р.

_____

«11» листопада 2026 р.

Захищено з оцінкою
_____

Члени комісії:

_____ Інна СТЕЦЕНКО

_____ Дмитро КОРИНЕЦЬКИЙ

Київ – 2026

ЗАВДАННЯ

Виконати дослідження для такої системи:

Експериментальна роботизована гнучка виробнича система має два верстати із числовим пультом керування, три роботи, пункт прибуття і склад оброблених деталей. Деталі прибувають на пункт прибуття кожні 40 секунд згідно з експоненціальним законом розподілу, захоплюються одним з вільних роботів і переміщуються ним до першого верстата, після чого робот звільняється. Після завершення обробки на першому верстаті деталь захоплюється одним з роботів і переміщується на другий верстат, а після обробки на другому верстаті – одним з роботів переміщується на склад оброблених деталей. Кожний з верстатів може одночасно обробляти до трьох деталей.

Час переміщення робота між пунктом прибуття і першим верстатом, першим і другим верстатом, другим верстатом і пунктом зберігання оброблених деталей складає відповідно 6, 7, і 5 секунд незалежно від того, холостий це хід, чи ні. Роботу потрібний час 8±1 секунд на захоплення або вивільнення деталей. Час обробки на першому верстаті розподілений за нормальним законом із середнім значення 60 секунд і має стандартне відхилення 10 секунд. Середній час обробки на другому верстаті дорівнює 100 секунд і має експоненціальний закон розподілу.

Визначте найкращий (з точки зору підвищення пропускної здатності гнучкої виробничої системи) спосіб закріплення роботів до операцій. Можливі варіанти закріплення:

1) по одному роботу на кожний з трьох шляхів переміщення деталей (пункт прибуття – перший верстат, перший верстат – другий верстат, другий верстат – склад);

2) кожний робот може використовуватися на кожному шляху переміщення деталей (при цьому повинен займатися найближчий з роботів).

Знайдіть розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів, максимальну місткість місця зберігання деталей на ділянці прибуття.

Для вищезазначеної системи:

1) розробити опис концептуальної моделі системи;

2) виконати формалізацію опису об’єкта моделювання в термінах визначеного у завданні формалізму (мережа масового обслуговування);

3) розробити алгоритм імітації моделі дискретно-подійної системи у відповідності до побудованого формального опису;

4) для доведення коректності побудованого алгоритму виконати верифікацію алгоритму імітації;

5) визначити статистичні оцінки заданих характеристик моделі, що є метою моделювання;

6) провести експериментальне дослідження моделі;

7) інтерпретувати результати моделювання та сформулювати пропозиції щодо поліпшення функціонування системи;

8) зробити висновки щодо складності розробки моделі та алгоритму імітації на основі використаного формалізму, отриманих результатів моделювання та їх корисності.

АНОТАЦІЯ

Курсова робота на тему «Імітаційна модель роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування».

Текстова частина курсової роботи складається із вступу, 5 розділів, висновків, списку використаних джерел з 10 найменувань та 3 додатків, які викладено на 102 сторінках. В тексті роботи оформлено 15 рисунків, 25 таблиць.

У роботі поставлено завдання визначити найкращий з погляду пропускної здатності спосіб закріплення роботів до операцій переміщення деталей в експериментальній роботизованій гнучкій виробничій системі з двох верстатів і трьох роботів. Порівнюються закріплення по одному роботу за кожним шляхом переміщення і призначення найближчого вільного робота на будь-який шлях. Для кожного способу потрібно знайти розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів і максимальну місткість місця зберігання на ділянці прибуття.

Імітаційна модель розроблена на основі формального опису мережею масового обслуговування з блокуванням маршрутів. Алгоритм імітації власний, з просуванням часу до найближчої події, реалізований мовою Python. Для статистичної обробки результатів використано бібліотеки NumPy і SciPy, для побудови рисунків – Matplotlib. Коректність моделі підтверджено верифікацією, автоматичними тестами і протоколом подій.

Проведено експериментальне дослідження моделі: визначено тривалість перехідного періоду і кількість прогонів, способи закріплення роботів порівняно методом дисперсійного аналізу, досліджено межу стійкості системи, закон розподілу часу проходження ідентифіковано за критерієм χ^2.

Результати моделювання свідчать, що найкращим є закріплення по одному роботу за кожним шляхом: його пропускна здатність становить 80,7 дет./год проти 76,4 дет./год при призначенні найближчого робота. За умови завдання, 90 дет./год, обидва варіанти перевантажені, а вузьким місцем є другий верстат. Час проходження деталей розподілений за гамма-законом зі зсувом.

Зроблено висновок, що для обробки всього потоку деталей доцільно додати четверте місце обробки на другому верстаті і зберегти закріплення роботів за шляхами. Пробні запуски показали, що після цієї зміни система обробляє 90 дет./год із середнім часом проходження 489 с.

Ключові слова: імітаційна модель, алгоритм імітації, мережа масового обслуговування, гнучка виробнича система, закріплення роботів, дисперсійний аналіз, Python.

ЗМІСТ

ВСТУП 9

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ 11

1.1 Мета моделювання 11
1.2 Складові елементи системи 11
1.3 Опис процесу функціонування системи 13
1.4 Параметри моделі 16
1.5 Вхідні змінні 16
1.6 Вихідні змінні 17
1.7 Висновки до розділу 17

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ 19

2.1 Відповідність елементів системи елементам мережі МО 19
2.2 Мережа МО для варіанта 1 20
2.3 Мережа МО для варіанта 2 21
2.4 Час обслуговування роботів 25
2.5 Числові параметри та генератори випадкових величин 26
2.6 Стан мережі, правила функціонування та початковий стан 27
2.7 Обчислення вихідних характеристик та вимоги до точності 28
2.8 Висновки до розділу 29

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ 31

3.1 Обґрунтування вибору мови програмування 31
3.2 Структура програми 31
3.3 Алгоритм просування часу 32
3.4 Події та зміна стану моделі 34
3.5 Призначення роботів 35
3.6 Генератори випадкових величин 36
3.7 Збір статистики 37

3.8 Опис класів та UML-діаграма класів 38
3.9 Верифікація моделі 40
3.10 Висновки до розділу 43
4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ 44
4.1 Мета і план експериментального дослідження 44
4.2 Тривалість перехідного періоду та кількість прогонів 45
4.3 Порівняння способів закріплення роботів 50
4.4 Пропускна здатність залежно від інтенсивності надходження деталей 53
4.5 Коефіцієнти використання роботів і верстатів 54
4.6 Розподіл часу проходження деталей 56
4.7 Максимальна місткість місця зберігання на ділянці прибуття 60
4.8 Програмна реалізація експериментів 61
4.9 Висновки до розділу 62
5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ СИСТЕМИ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ 64
5.1 Інтерпретація результатів моделювання 64
5.2 Пропозиції щодо поліпшення функціонування системи 67
5.3 Пропозиції щодо подальшого дослідження 70
5.4 Висновки до розділу 70
ВИСНОВКИ 71
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 74
ДОДАТОК А. ПРОТОКОЛИ ПОДІЙ 75
А.1 Протокол подій для варіанта 1 75
А.2 Протокол подій для варіанта 2 78
ДОДАТОК Б. ЛІСТИНГ КОДУ АЛГОРИТМУ ІМІТАЦІЇ 81
Б.1 Модуль rng.py – генератори випадкових величин 81
Б.2 Модуль params.py – вхідні змінні та параметри моделі 82

Б.3 Модуль elements.py – елементи мережі масового обслуговування.. 83
Б.4 Модуль model.py – алгоритм імітації 89
ДОДАТОК В. ЛІСТИНГ КОДУ ЕКСПЕРИМЕНТАЛЬНОГО ДОСЛІДЖЕННЯ 94
В.1 Модуль common.py – спільні засоби для серій прогонів 94
В.2 Модуль verification.py – верифікація моделі 95
В.3 Модуль warmup.py – тривалість перехідного періоду 96
В.4 Модуль replications.py – кількість прогонів 97
В.5 Модуль compare.py – порівняння способів закріплення роботів 98
В.6 Модуль capacity.py – межа стійкості 99
В.7 Модуль distribution.py – розподіл часу проходження деталей 100
В.8 Модуль improvement.py – пробні запуски пропозицій 101

ВСТУП

Гнучкі виробничі системи (ГВС) з верстатами з числовим програмним керуванням і промисловими роботами дають змогу автоматизувати обробку деталей і швидко переналаштовувати виробництво. Продуктивність такої системи залежить не лише від швидкодії верстатів, а й від організації транспортування деталей між ними. Одних і тих самих роботів можна закріпити за окремими шляхами переміщення або дозволити кожному роботу виконувати будь-яке перевезення. Який спосіб кращий, заздалегідь не очевидно. Перевіряти це на реальному обладнанні дорого, а аналітичні методи теорії масового обслуговування погано враховують блокування верстатів, очікування роботів із деталлю і залежність вибору робота від його положення. Тому для розв’язання такого завдання застосовують імітаційне моделювання. Воно відтворює роботу системи з урахуванням випадкових тривалостей операцій і дає змогу порівняти варіанти організації роботи без змін у реальній системі [1; 2].

Тема курсової роботи – «Імітаційна модель роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування». Об’єктом моделювання є експериментальна роботизована ГВС із двох верстатів, трьох роботів, пункту прибуття деталей і складу оброблених деталей.

Метою роботи є визначення найкращого з погляду пропускної здатності способу закріплення роботів до операцій переміщення деталей. Порівнюються два способи: по одному роботу на кожному з трьох шляхів переміщення і призначення на будь-який шлях найближчого вільного робота. Для кожного способу потрібно знайти розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів та максимальну місткість місця зберігання деталей на ділянці прибуття.

Для досягнення мети розв’язано такі задачі:

– розроблено концептуальну модель системи з BPMN-діаграмами процесу;

– побудовано формалізовані моделі обох варіантів у вигляді мереж масового обслуговування з блокуванням маршрутів;

– розроблено власний алгоритм імітації з просуванням часу до найближчої події і реалізовано його мовою Python [4];

– виконано верифікацію моделі варіюванням вхідних змінних і порівнянням з ручним розрахунком;

– визначено тривалість перехідного періоду і кількість прогонів, порівняно варіанти методом дисперсійного аналізу, ідентифіковано закон розподілу часу проходження за критерієм χ^2;

– інтерпретовано результати моделювання і перевірено пробними запусками пропозиції щодо поліпшення системи.

Статистичну обробку результатів експериментів виконано з використанням бібліотек NumPy і SciPy, рисунки побудовано бібліотекою Matplotlib.

Пояснювальна записка складається з п’яти розділів. У розділі 1 описано концептуальну модель системи, у розділі 2 – формалізовані моделі обох варіантів, у розділі 3 – алгоритм імітації, його програмну реалізацію і верифікацію. Розділ 4 містить експериментальне дослідження моделі, розділ 5 – інтерпретацію результатів і пропозиції щодо поліпшення системи. Протоколи подій і код програми наведено в додатках.

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ

Концептуальна модель системи – це деталізований опис складових елементів системи, правил їх взаємодії та процесу, який моделюється, разом із зазначенням мети моделювання [3, с. 49]. За умовою завдання потрібно порівняти два способи закріплення роботів. Якщо завдання передбачає альтернативні моделі, концептуальний опис розробляють для кожної з них [3, с. 50]. Тому спільні елементи і процес описано один раз, а відмінності – окремо для кожного варіанта.

1.1 Мета моделювання

Об’єктом моделювання є експериментальна роботизована гнучка виробнича система (ГВС), що складається з двох верстатів із числовим програмним керуванням, трьох роботів, пункту прибуття деталей і складу оброблених деталей.

Метою моделювання є визначення найкращого з погляду пропускної здатності ГВС способу закріплення роботів до операцій переміщення деталей. Розглядаються два способи:

1) варіант 1 – по одному роботу на кожному з трьох шляхів переміщення деталей: пункт прибуття – перший верстат, перший верстат – другий верстат, другий верстат – склад;

2) варіант 2 – кожен робот може виконувати переміщення на будь-якому шляху, при цьому на перевезення призначається найближчий з вільних роботів.

Для кожного варіанта потрібно визначити:

– пропускну здатність ГВС, тобто кількість деталей, що надходять на склад за одиницю часу;

– розподіл часу проходження деталей через систему;

– коефіцієнти використання роботів і верстатів;

– максимальну місткість місця зберігання деталей на ділянці прибуття.

1.2 Складові елементи системи

За текстом завдання виділено такі елементи ГВС:

1) деталі – об’єкти обробки; вони надходять на пункт прибуття в середньому кожні 40 с, інтервали між надходженнями розподілені за експоненціальним законом;

2) пункт прибуття A з місцем зберігання (площадкою), на якому деталі очікують на робота; кількість місць на площадці не обмежена, а її найбільше заповнення є однією з шуканих характеристик;

3) верстати M1 і M2 з числовим програмним керуванням; кожен із них одночасно обробляє до трьох деталей, тобто має три місця обробки; час обробки на M1 розподілений за нормальним законом із середнім 60 с і стандартним відхиленням 10 с, на M2 – за експоненціальним законом із середнім 100 с;

4) три роботи R1, R2, R3, які захоплюють деталь, переміщують її між пунктами і вивільняють; захоплення і вивільнення триває 8 ± 1 с;

5) склад оброблених деталей W, на якому деталь залишає систему.

Час переміщення робота між пунктом прибуття і першим верстатом, першим і другим верстатом, другим верстатом і складом становить відповідно 6, 7 і 5 с і однаковий для руху з деталлю і порожнім. Схему розташування обладнання та закріплення роботів для обох варіантів наведено на рисунку 1.1.

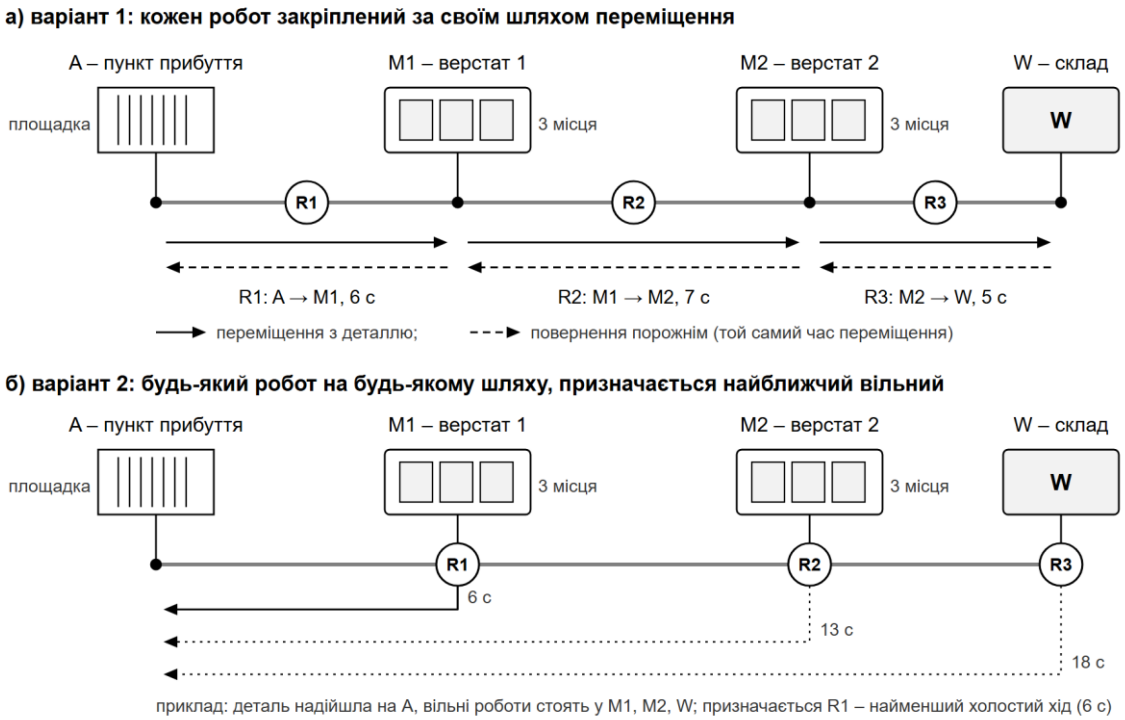


Рисунок 1.1 – Схема розташування обладнання ГВС і закріплення роботів

1.3 Опис процесу функціонування системи

Деталь, що надійшла на пункт прибуття, залишається на площадці, доки її не захопить робот. Перевезення деталі між двома пунктами складається з таких дій робота:

1) холостий хід до точки завантаження, якщо робот стоїть в іншому пункті;

2) захоплення деталі; місце, яке займала деталь, звільняється після завершення захоплення;

3) переміщення з деталлю до пункту призначення;

4) очікування з деталлю, якщо на верстаті-приймачі немає вільного місця; робот тримає деталь і не може виконувати інших перевезень, доки місце не звільниться;

5) вивільнення деталі; з цього моменту на верстаті починається обробка, а робот стає вільним і залишається в пункті вивантаження.

у першому варіанті він має повернутись до стартової точки свого маршруту

Отже, повернення робота не є окремою дією: робот залишається там, де вивільнив деталь, і рухається порожнім, коли його призначають на наступне перевезення.

Оброблена на верстаті деталь залишається на своєму місці, доки її не захопить робот. Поки деталь чекає на робота, місце верстата зайняте і нова деталь на нього потрапити не може. Після обробки на M1 деталь перевозиться на M2, після обробки на M2 – на склад, де вона залишає систему.

Варіанти відрізняються лише правилом призначення робота на перевезення:

– у варіанті 1 кожен робот обслуговує лише свій шлях: R1 – A $\rightarrow$ M1, R2 – M1 $\rightarrow$ M2, R3 – M2 $\rightarrow$ W; після вивільнення робот повертається порожнім своїм шляхом до точки завантаження;

– у варіанті 2 будь-який вільний робот може виконати будь-яке перевезення; призначається робот, холостий хід якого до точки завантаження найменший; при однаковій відстані перевагу має перевезення, ближче до складу, далі – деталь, що чекає довше.

цього в умові немає, навіщо додали?

кількість роботів однай та сама - три за умовою

У варіанті 2 до кожного верстата одночасно призначається не більше роботів, ніж на ньому вільних місць, плюс один. Без цього обмеження всі роботи можуть узяти деталі для одного заповненого верстата й чекати біля нього, а оброблені деталі на цьому верстаті не матимуть вільного робота, і система зупиниться (підрозділ 2.3).

Під час розробки моделі прийнято такі припущення:

1) пункти розташовані на одній лінії A – M1 – M2 – W, тому час переміщення між несусідніми пунктами дорівнює сумі часів між сусідніми: A – M2 становить 13 с, M1 – W 12 с, A – W 18 с;

2) після вивільнення деталі робот залишається в пункті вивантаження;

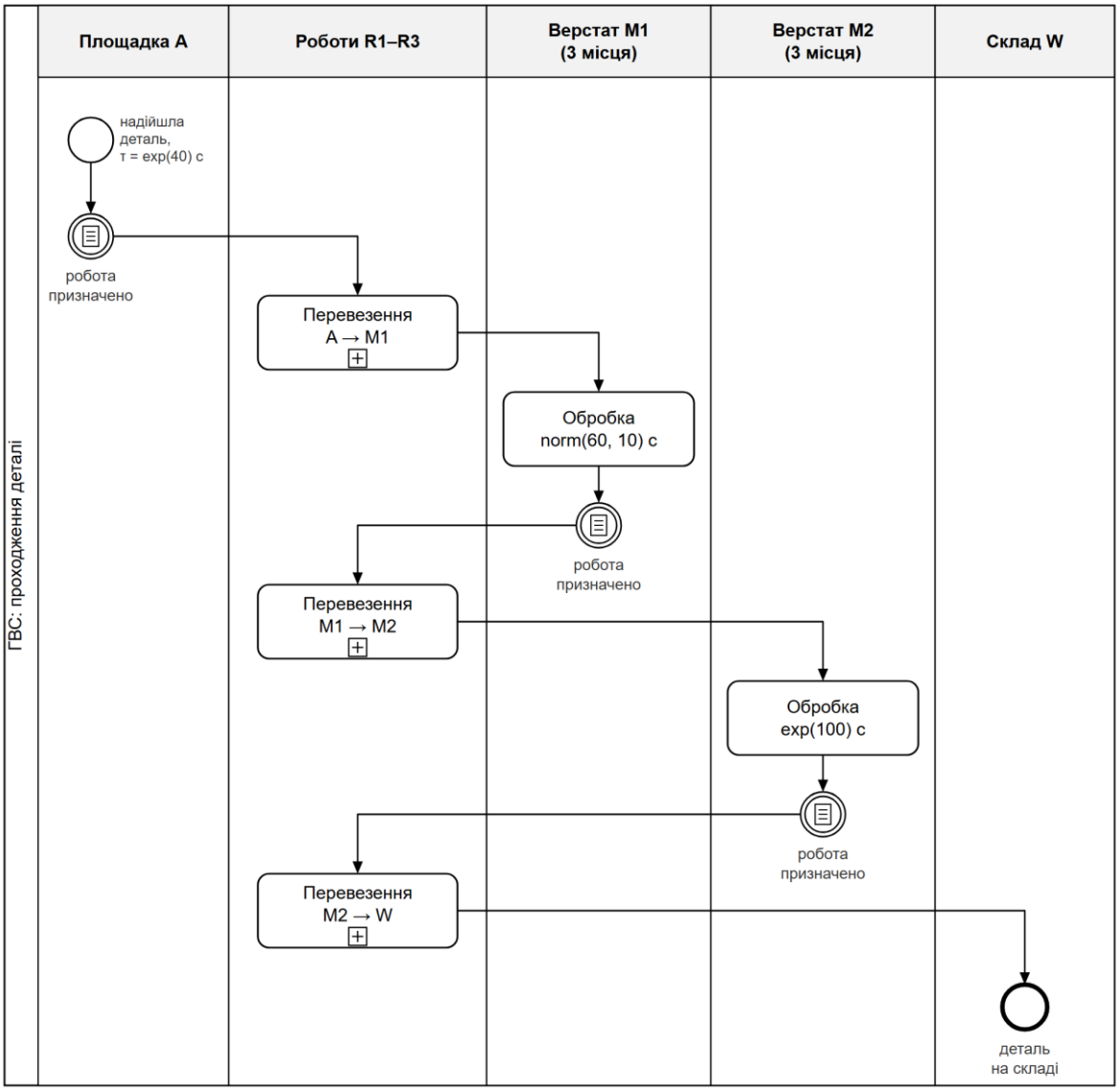
3) час обробки на M1 не може бути від'ємним, тому від'ємні значення нормального закону відкидаються; ймовірність такого значення менша за 10^{-9};

4) деталі на площадці A і на кожному верстаті перевозяться в порядку готовності до перевезення (FIFO);

5) події, що відбуваються одночасно, виконуються у випадковому порядку;

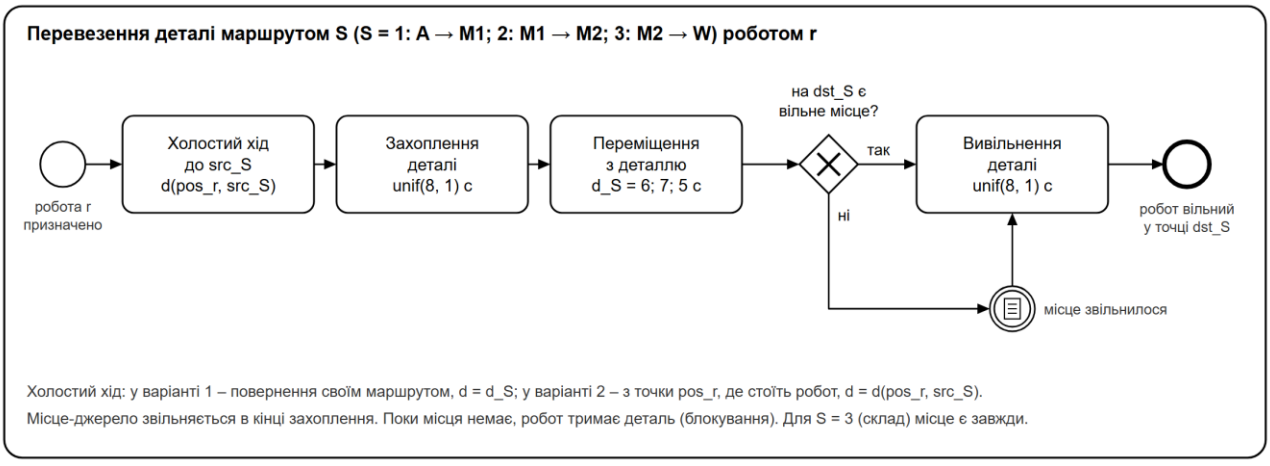
6) у початковий момент система порожня, а роботи стоять у точках завантаження своїх шляхів варіанта 1: R1 – в A, R2 – в M1, R3 – в M2; для варіанта 2 прийнято той самий початковий стан, щоб варіанти порівнювались за рівних умов.

Процес проходження деталі через ГВС у нотації BPMN [5] наведено на рисунку 1.2. Перевезення деталі на діаграмі є підпроцесом, однаковим для всіх трьох шляхів. Його розгорнуто на рисунку 1.3.



◎ – проміжна подія очікування: деталь чекає на призначення робота (на площадці А – у черзі FIFO, на верстаті – займаючи місце);
[+] – підпроцес перевезення (рис. 1.3); робота призначає правило варіанта 1 (закріплений робот) або варіанта 2 (найближчий вільний).

Рисунок 1.2 – BPMN-діаграма процесу проходження деталі через ГВС



Холостий хід: у варіанті 1 – повернення своїм маршрутом, d = d_S; у варіанті 2 – з точки pos_r, де стоїть робот, d = d(pos_r, src_S).
Місце-джерело звільняється в кінці захоплення. Поки місця немає, робот тримає деталь (блокування). Для S = 3 (склад) місце є завжди.

Рисунок 1.3 – BPMN-діаграма підпроцесу перевезення деталі роботом

1.4 Параметри моделі

Параметри моделі – це числові характеристики системи, які не змінюються під час експериментів з базовою моделлю. Перелік параметрів наведено в таблиці 1.1.

Таблиця 1.1 – Параметри моделі

Позначення	Параметр	Значення
m	кількість місць обробки на кожному верстаті	3
N_R	кількість роботів	3
$d_1,\ d_2,\ d_3$	час переміщення робота шляхами A – M1, M1 – M2, M2 – W, с	6; 7; 5
—	час переміщення між несусідніми пунктами A – M2, M1 – W, A – W, с	13; 12; 18
max	кількість місць на площадці A	не обмежена
—	закріплення роботів у варіанті 1	R1: A → M1; R2: M1 → M2; R3: M2 → W
$\rho_1,\ \rho_2,\ \rho_3$	пріоритет шляху 1, 2, 3 при однаковій відстані до робота у варіанті 2 (менше значення – вищий пріоритет)	3; 2; 1

1.5 Вхідні змінні

Вхідні змінні моделі описують зовнішній потік деталей і тривалості операцій. Для кожної змінної в таблиці 1.2 наведено закон розподілу з числовими значеннями, початкове значення і діапазон варіювання під час верифікації. Закони розподілу позначено за методичними рекомендаціями [3, с. 51]: $\exp(d)$ – експоненціальний із середнім d; $\mathrm{norm}(d, s)$ – нормальний із середнім d і середнім квадратичним відхиленням s; $\mathrm{unif}(d, s)$ – рівномірний на інтервалі $(d - s;\ d + s)$, тобто $\mathrm{unif}(8,1)$ означає 8 ± 1 с.

Таблиця 1.2 – Вхідні змінні моделі

Позначення	Вхідна змінна	Початкове значення	Діапазон варіювання
τ_A	інтервал між надходженнями деталей, с	$\exp(40)$	від $\exp(30)$ до $\exp(80)$
$t_{\text{обр1}}$	час обробки на верстаті M1, с	$\mathrm{norm}(60,10)$	від $\mathrm{norm}(60,10)$ до $\mathrm{norm}(120,10)$
$t_{\text{обр2}}$	час обробки на верстаті M2, с	$\exp(100)$	від $\exp(50)$ до $\exp(100)$
$t_{\text{зх}}$	час захоплення деталі роботом, с	$\mathrm{unif}(8,1)$	від $\mathrm{unif}(2;\ 0{,}5)$ до $\mathrm{unif}(8,1)$
$t_{\text{вив}}$	час вивільнення деталі роботом, с	$\mathrm{unif}(8,1)$	від $\mathrm{unif}(2;\ 0{,}5)$ до $\mathrm{unif}(8,1)$

Під час верифікації змінювались також час переміщення роботів (до трикратного) і кількість місць на верстаті М2 (до шести), а для пропозицій щодо поліпшення системи – кількість місць на М2 і кількість роботів (підрозділи 3.9 і 5.2).

1.6 Вихідні змінні

Вихідні змінні визначаються метою моделювання. Їх перелік і величини, які спостерігаються під час імітації для їх обчислення, наведено в таблиці 1.3. Формули обчислення наведено в підрозділі 2.7.

Таблиця 1.3 – Вихідні змінні моделі

Позначення	Вихідна змінна	Що спостерігається під час імітації
Λ	пропускна здатність ГВС, дет./год	кількість деталей, вивільнених на складі за час спостереження
τ	час проходження деталі, с	момент надходження деталі на площадку А і момент завершення її вивільнення на складі; за масивом значень обчислюються середнє, середнє квадратичне відхилення, гістограма і закон розподілу
$\rho_{\mathrm{R}k}$	коефіцієнт використання робота Rk	тривалість кожної фази роботи робота: холостого ходу, захоплення, переміщення з деталлю, очікування з деталлю, вивільнення
$\rho_{\mathrm{M}i}$	коефіцієнт використання верстата Mi	тривалість станів кожного місця верстата: вивільнення, обробки, блокування (оброблена деталь чекає на робота), захоплення
L_{max}	максимальна місткість місця зберігання на ділянці прибуття, шт.	максимальна кількість деталей на площадці А ~~в кожний момент часу~~
$\bar{L}_A$	середня кількість деталей на площадці А, ~~шт.~~ од.	те саме з урахуванням тривалості інтервалів між подіями

Для оцінювання стійкості режиму додатково фіксується кількість деталей, що перебувають у системі в кінці прогону. Якщо вона зростає разом із часом моделювання, черга на площадці накопичується необмежено, і стаціонарного режиму немає.

1.7 Висновки до розділу

У розділі описано концептуальну модель роботизованої ГВС: визначено мету моделювання, складові елементи системи, процес проходження деталі і дії робота під час перевезення. Процес для обох способів закріплення роботів

однаковий, а відрізняється правило призначення робота на перевезення. Визначено параметри, вхідні та вихідні змінні моделі. Процес представлено BPMN-діаграмами, які є основою для формалізованого опису мережею масового обслуговування в розділі 2.

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ

Формалізацію виконано засобами мережі масового обслуговування (МО). Мережа МО складається із систем масового обслуговування (СМО), з'єднаних дугами, що представляють маршрут слідування запитів, і зовнішнього потоку надходження запитів [3, с. 50]. Графічні позначення елементів і їх числові параметри прийнято за таблицею 3.1 методичних рекомендацій [3, с. 52-54], правила функціонування мережі і блокування маршрутів – за навчальним посібником [2, с. 62-67].

2.1 Відповідність елементів системи елементам мережі МО

Запитом (вимогою) мережі є деталь. Мережа розімкнена: деталі надходять ззовні на площадку А і залишають мережу після вивільнення на складі. Під час обслуговування деталь не знищується, не з'єднується з іншими деталями і не розпадається, тому процес можна описати мережею МО [2, с. 65]. Роботи є пристроями обслуговування. Дуга мережі позначає маршрут слідування запиту, тобто деталі, тому рух робота порожнім не має окремої дуги, а входить у часову затримку пристрою (підрозділ 2.4). Відповідність елементів системи елементам мережі МО для обох варіантів наведено в таблиці 2.1.

Таблиця 2.1 – Відповідність елементів системи елементам мережі МО

Елемент системи	Варіант 1	Варіант 2
деталь	запит	запит типу S – номер маршруту, яким деталь треба перевезти
надходження деталей	зовнішнє надходження, $t = \exp(40)$	те саме
площадка А	необмежена черга СМО R1	необмежена черга СМО R для запитів типу 1
роботи	три одноканальні СМО R1, R2, R3, кожна обслуговує свій маршрут	одна триканальна СМО R з вибором найближчого вільного пристрою
верстат М1, М2	СМО з трьох пристроїв (місць) без черги	те саме
склад	вихід з мережі	те саме
«на верстаті немає місця», «немає вільного робота»	блокування маршрутів B_1-B_4	блокування маршрутів B_0-B_4 і керування роботами

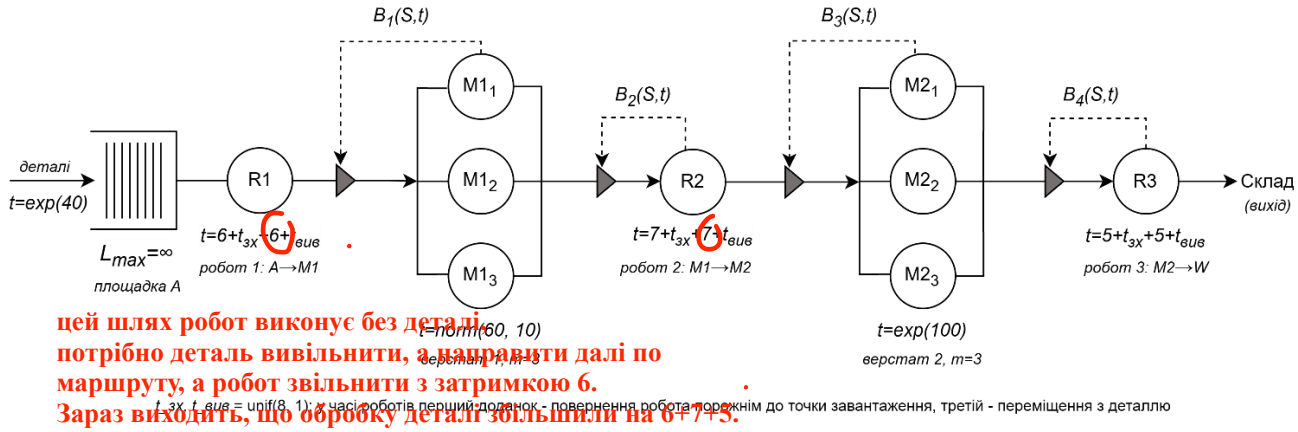
Маршрути переміщення деталей наведено в таблиці 2.2. Тут і далі src_S і dst_S – точки завантаження і вивантаження маршруту S, d_S – час переміщення з деталлю.

Таблиця 2.2 – Маршрути переміщення деталей

Маршрут S	Звідки, src_S	Куди, dst_S	d_S, с	Робот у варіанті 1	Де деталь чекає на робота
1	A	M1	6	R1	у черзі площадки A
2	M1	M2	7	R2	на місці верстата M1
3	M2	W	5	R3	на місці верстата M2

2.2 Мережа МО для варіанта 1

Мережу МО, що представляє формалізовану модель ГВС для варіанта 1, наведено на рисунку 2.1. Кожен робот закріплений за одним маршрутом, тому йому відповідає окрема одноканальна СМО. Верстат, що одночасно обробляє до трьох деталей, є СМО з трьох ідентичних пристроїв (місць) без черги.



цей шлях робот виконує без деталі
потрібно деталь вивільнити, а направити далі по маршруту, а робот звільнити з затримкою 6.
Зараз виходить, що обробку деталі збільшили на 6+7+5.

Рисунок 2.1 – Мережа масового обслуговування, що представляє формалізовану модель ГВС (варіант 1)

Деталь проходить мережу послідовно: площадка A → R1 → M1 → R2 → M2 → R3 → склад. Площадка A є чергою СМО R1, тому з пристроєм R1 її з'єднує відрізок, а не дуга [3, с. 53]. Під кожним роботом указано його часову затримку; перший доданок у ній – повернення робота порожнім до точки завантаження. Трикутники позначають блокування маршрутів B_1-B_4, пунктирні лінії – керуючі зв'язки від елемента, стан якого визначає умову блокування. Блокування задано предикатами, які залежать від стану мережі S у момент t [3, с. 54]:

K_M11(t)+K_M12(t)+K_M13(t) =3

$$B_1(S,t) = \mathbf{1}\{n_1(t) = 3\}, \qquad B_2(S,t) = \mathbf{1}\{K_{\mathrm{R}2}(t) \neq 0\},$$
$$B_3(S,t) = \mathbf{1}\{n_2(t) = 3\}, \qquad B_4(S,t) = \mathbf{1}\{K_{\mathrm{R}3}(t) \neq 0\}, \qquad (2.1)$$

використовуйте стан елементів мережі

де $\mathbf{1}\{P\}$ дорівнює 1, якщо умова P виконується, і 0, якщо ні; $n_i(t)$ – кількість зайнятих місць верстата Mi; $K_{\mathrm{R}k}(t)$ – стан робота Rk (0 – вільний). Маршрут закритий, якщо $B(S,t) = 1$. Маршрут R3 → склад не блокується. Зміст блокувань наведено в таблиці 2.3.

Не потрібна таблиця, усе задано вже схемою і предикатом. Або зробіть її перед визначенням предикату B.

Таблиця 2.3 – Блокування маршрутів у варіанті 1

Умова	Маршрут	Маршрут закритий, якщо	Де чекає деталь	Блокування знімає подія
B_1	R1 → M1	усі 3 місця M1 зайняті	у пристрої R1: робот тримає деталь біля M1	R2 завершив захоплення деталі з M1
B_2	M1 → R2	робот R2 зайнятий	на місці верстата M1	R2 завершив вивільнення на M2
B_3	R2 → M2	усі 3 місця M2 зайняті	у пристрої R2: робот тримає деталь біля M2	R3 завершив захоплення деталі з M2
B_4	M2 → R3	робот R3 зайнятий	на місці верстата M2	R3 завершив вивільнення на складі

Блокування B_1 і B_3 перевіряються в момент, коли робот із деталлю прибуває до верстата. Якщо вільне місце є, робот одразу починає вивільнення, інакше тримає деталь, доки місце не звільниться. Місце верстата зайняте, поки на ньому фізично лежить деталь: від початку вивільнення до кінця захоплення роботом наступного маршруту. Перед R2 і R3 черги немає, оскільки оброблена деталь лежить на місці верстата і займає його, а черга звільнила б місце. За відсутності черги запит, який не застав пристрій вільним, отримує відмову в обслуговуванні [3, с. 50]. Тому використано блокування B_2 і B_4: деталь залишається в пристрої, з якого намагається вийти, а місце переходить у стан «блоковане» [3, с. 54].

2.3 Мережа МО для варіанта 2

Мережу МО для варіанта 2 наведено на рисунку 2.2. Роботи утворюють одну триканальну СМО R з пристроями R1, R2, R3, оскільки будь-який вільний робот може обслужити будь-яку деталь. Деталь проходить СМО R тричі, по одному разу на кожному маршруті. Номер маршруту, яким деталь треба перевезти, є типом запиту S. Від типу залежать місце, звідки деталь надходить у

СМО R, тривалість обслуговування і маршрут після СМО R, а саме для таких випадків у мережу МО вводять кілька типів запитів [2, с. 63]. Деталь надходить на площадку А з типом $S = 1$, після обробки на М1 отримує тип $S = 2$, після обробки на М2 – тип $S = 3$.

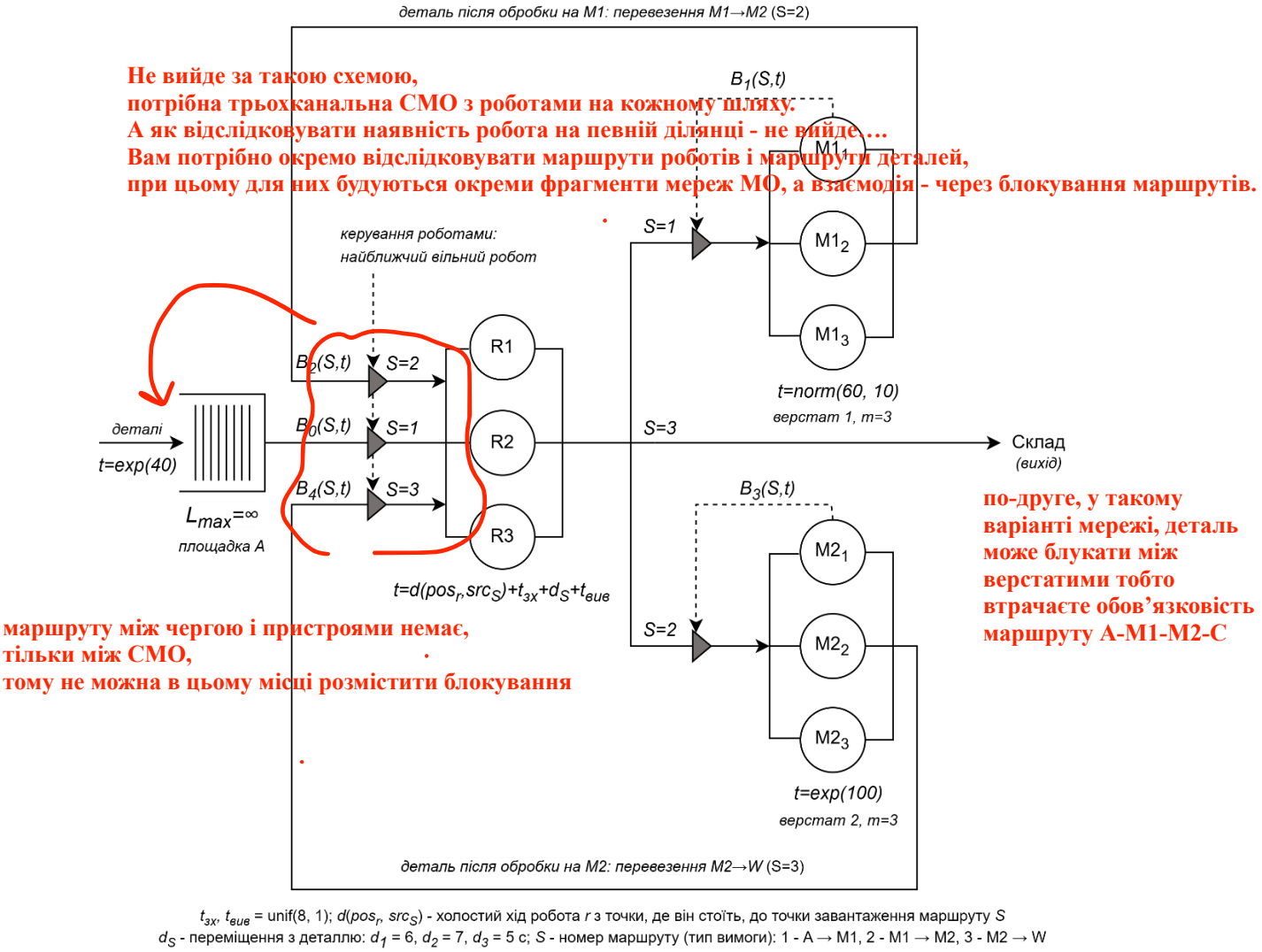


Рисунок 2.2 – Мережа масового обслуговування, що представляє формалізовану модель ГВС (варіант 2)

Площадка А є чергою СМО R, і в ній чекають лише деталі типу 1. Деталі типів 2 і 3 після обробки лишаються на місцях верстатів і надходять у СМО R дугами М1 → R і М2 → R, минаючи чергу. Ці дуги позначають маршрут деталі до наступного перевезення, а не рух робота: після обробки деталь повертається в ту саму СМО роботів, як запит на рисунку 3.3 посібника [2, с. 65] повертається

на доробку. На виході СМО R маршрут розгалужується за типом запиту: $S = 1$ – на М1, $S = 2$ – на М2, $S = 3$ – на склад.

Блокування B_1 і B_3 мають той самий зміст, що й у варіанті 1. Умови B_0, B_2, B_4 на входах СМО R задає керування роботами – правило призначення найближчого вільного робота. Його керуючий зв'язок проходить через усі три трикутники, як керування рухом на рисунку 3.30 посібника [2, с. 97], де одне джерело керує двома блокуваннями, а блокування стоїть між чергою і пристроєм. Предикати блокувань:

$$B_0(S,t) = \mathbf{1}\{u_1(t) > f_1(t)\}, \quad B_1(S,t) = \mathbf{1}\{n_1(t) = 3\}, \quad B_3(S,t) = \mathbf{1}\{n_2(t) = 3\},$$
$$B_2(S,t) = \mathbf{1}\{N_{\text{вільн}}(t) = 0 \ \vee \ u_2(t) > f_2(t)\}, \quad B_4(S,t) = \mathbf{1}\{N_{\text{вільн}}(t) = 0\}, \tag{2.2}$$

де $f_i(t)$ – кількість вільних місць верстата Мi; $N_{\text{вільн}}(t)$ – кількість вільних роботів; $u_i(t)$ – кількість роботів, призначених доставити деталь на верстат Мi, які ще не зайняли на ньому місця. Зміст блокувань наведено в таблиці 2.4.

Таблиця 2.4 – Блокування маршрутів у варіанті 2

Умова	Маршрут	Маршрут закритий, якщо	Де чекає деталь	Блокування знімає подія
B_0	A → R, $S = 1$	до М1 уже призначено на одного робота більше, ніж там вільних місць	у черзі площадки A	робот завершив захоплення деталі з М1
B_1	R → М1	усі 3 місця М1 зайняті	у пристрої R: робот тримає деталь біля М1	робот завершив захоплення деталі з М1
B_2	М1 → R, $S = 2$	усі роботи зайняті або до М2 уже призначено на одного робота більше, ніж там вільних місць	на місці верстата М1	кінець вивільнення (звільнився робот) або кінець захоплення деталі з М2
B_3	R → М2	усі 3 місця М2 зайняті	у пристрої R: робот тримає деталь біля М2	робот завершив захоплення деталі з М2
B_4	М2 → R, $S = 3$	усі роботи зайняті	на місці верстата М2	будь-який робот завершив вивільнення

Відкритий вхід СМО R дозволяє деталі зайняти вільного робота. Правило, за яким запит обирає один із вільних пристроїв, задається відповідно до реальної системи [2, с. 63]. Якщо деталей на відкритих входах більше, ніж вільних роботів, порядок визначає правило вибору. Після обробки всіх подій поточного моменту формується множина допустимих пар «робот – тип деталі»:

$$P(t) = \{(r,S):\ K_r(t) = 0,\ \ q_S(t) > 0,\ \ B_{\text{вх}}(S,t) = 0\},$$
$$B_{\text{вх}}(1,t) = B_0, \qquad B_{\text{вх}}(2,t) = B_2, \qquad B_{\text{вх}}(3,t) = B_4, \tag{2.3}$$

де $K_r(t)$ – стан робота r; $q_S(t)$ – кількість деталей типу S, на які ще не призначено робота. З множини $P(t)$ обирається пара з найменшим значенням критеріїв, які порівнюються по черзі:

$$(r^*, S^*) = \arg\min_{(r,S)\in P(t)} (d(pos_r, src_S),\ \rho_S,\ t_S^{\text{гот}},\ \xi), \tag{2.4}$$

де $d(pos_r, src_S)$ – час холостого ходу робота r із точки pos_r, де він стоїть, до точки завантаження src_S; ρ_S – пріоритет маршруту (таблиця 1.1); $t_S^{\text{гот}}$ – момент, коли перша з деталей типу S, що чекають, стала готова до перевезення; ξ – рівномірно розподілена на $(0;\ 1)$ величина. Отже, спочатку обирається найближчий робот, при рівній відстані – маршрут, ближчий до складу, далі – деталь, що чекає довше, далі – випадковий вибір. Після кожного призначення значення $N_{\text{віл}}$, u_i і q_S змінюються, і вибір повторюється, доки множина $P(t)$ не стане порожньою.

Умова $u_i \le f_i$ в предикатах B_0 і B_2 обмежує кількість роботів, які одночасно їдуть до верстата: не більше, ніж там вільних місць, плюс один. Робот, який привіз деталь до заповненого верстата, тримає її і не може виконувати інших перевезень. У варіанті 1 до кожного верстата їздить лише свій робот, тому біля верстата чекає щонайбільше один робот. У варіанті 2 без обмеження всі три роботи можуть узяти деталі для одного заповненого верстата і стати біля нього. Тоді оброблені деталі на цьому верстаті чекають на вільного робота, якого вже немає, і виникає взаємне блокування. Оскільки умова перевіряється в момент призначення, завжди $u_i \le f_i + 1$, тобто біля кожного верстата чекає не більше одного робота. Заблокованими можуть бути щонайбільше два роботи, по одному біля М1 і М2. Третій робот заблокуватися не може: щойно на М2 з’явиться оброблена деталь, він вивезе її на склад, бо цей маршрут не блокується. Місце на М2 звільниться, і рух відновиться. Без цього правила в усіх п’яти пробних прогонах тривалістю 200 000 с остання деталь потрапляла на склад на 320-1172-й секунді, після чого всі три роботи стояли з деталями біля заповнених верстатів.

2.4 Час обслуговування роботів

Обслуговування деталі роботом складається з п’яти фаз, наведених у таблиці 2.5. Після вивільнення робот залишається в точці вивантаження. Коли його призначають на наступну деталь, він рухається порожнім до точки завантаження. Цей рух є першою фазою обслуговування.

Таблиця 2.5 – Фази обслуговування деталі роботом

Фаза	Дія робота	Тривалість	Де фізично деталь
1	холостий хід до точки завантаження src_S	$d^{\text{хол}}$	на площадці A або на місці верстата-джерела
2	захоплення деталі	$t_{\text{зх}}$	там само; місце звільняється в кінці фази
3	переміщення з деталлю $src_S \to dst_S$	d_S	у захваті робота
4	очікування вільного місця на верстаті-приймачі (лише якщо маршрут заблоковано)	$\tau_{\text{бл}} \geq 0$	у захваті робота
5	вивільнення деталі	$t_{\text{вив}}$	на місці верстата-приймача, яке вже зайняте (для $S = 3$ – на складі)

Часова затримка пристрою, тобто час обслуговування деталі роботом без блокування:

$$t_{\text{R}} = \underbrace{d^{\text{хол}}}_{\text{холостий хід}} + \underbrace{t_{\text{зх}}}_{\text{захоплення}} + \underbrace{d_S}_{\text{з деталлю}} + \underbrace{t_{\text{вив}}}_{\text{вивільнення}}, \qquad t_{\text{зх}},\ t_{\text{вив}} \sim \text{unif}(8,1). \tag{2.5}$$

У варіанті 1 закріплений робот повертається своїм маршрутом, тому $d^{\text{хол}} = d_S$ для всіх деталей, крім першої, для якої $d^{\text{хол}} = 0$:

$$\begin{cases} t_{\text{R1}} = 6 + t_{\text{зх}} + 6 + t_{\text{вив}}, & \bar{t}_{\text{R1}} = 28 \text{ с}, \\ t_{\text{R2}} = 7 + t_{\text{зх}} + 7 + t_{\text{вив}}, & \bar{t}_{\text{R2}} = 30 \text{ с}, \\ t_{\text{R3}} = 5 + t_{\text{зх}} + 5 + t_{\text{вив}}, & \bar{t}_{\text{R3}} = 26 \text{ с}, \end{cases} \tag{2.6}$$

де $\bar{t}$ – середнє значення. У варіанті 2 робот рухається порожнім із точки pos_r, де він вивільнив попередню деталь, тому $d^{\text{хол}} = d(pos_r, src_S)$. Час переміщення між пунктами задано матрицею

$$D = \begin{array}{c|cccc} & \text{A} & \text{M1} & \text{M2} & \text{W} \\ \text{A} & 0 & 6 & 13 & 18 \\ \text{M1} & 6 & 0 & 7 & 12, \\ \text{M2} & 13 & 7 & 0 & 5 \\ \text{W} & 18 & 12 & 5 & 0 \end{array} \tag{2.7}$$

з якої для кожного типу деталі отримуємо межі часу обслуговування:

$$\begin{cases} t_{\mathrm{R}}(1,r) = d(pos_r, \mathrm{A}) + t_{\text{зх}} + 6 + t_{\text{вив}}, & 20 < t_{\mathrm{R}}(1,r) < 42 \text{ с}, \\ t_{\mathrm{R}}(2,r) = d(pos_r, \mathrm{M1}) + t_{\text{зх}} + 7 + t_{\text{вив}}, & 21 < t_{\mathrm{R}}(2,r) < 37 \text{ с}, \\ t_{\mathrm{R}}(3,r) = d(pos_r, \mathrm{M2}) + t_{\text{зх}} + 5 + t_{\text{вив}}, & 19 < t_{\mathrm{R}}(3,r) < 36 \text{ с}. \end{cases} \quad (2.8)$$

Найменші значення досягаються, коли робот уже стоїть у точці завантаження: наприклад, робот, що вивільнив деталь на M2, одразу забирає з M2 оброблену деталь на склад. Варіант 1 є окремим випадком формули (2.5), коли $d^{\text{хол}} = d_S$. Середнє значення t_{R} у варіанті 2 заздалегідь невідоме, бо залежить від того, де стоять вільні роботи в момент призначення, тому його оцінюють за результатами імітації. Повний час, протягом якого робот зайнятий однією деталлю, дорівнює $t_{\mathrm{R}} + \tau_{\text{бл}}$. Час блокування $\tau_{\text{бл}}$ не задається законом розподілу: його визначає стан верстата-приймача. Для маршруту на склад завжди $\tau_{\text{бл}} = 0$.

2.5 Числові параметри та генератори випадкових величин

Числові значення параметрів і вхідних змінних мережі наведено в таблицях 1.1 і 1.2 та на рисунках 2.1 і 2.2: інтервал надходження $\exp(40)$, час обробки на M1 $\mathrm{norm}(60{,}10)$, на M2 $\exp(100)$, час захоплення і вивільнення $\mathrm{unif}(8{,}1)$, по три місця на верстатах, переміщення 6, 7 і 5 с, необмежена черга площадки A. Усі випадкові величини незалежні, $t_{\text{зх}}$ і $t_{\text{вив}}$ генеруються окремо для кожної дії робота. Значення випадкових величин отримуються з рівномірно розподіленої на $(0;\ 1)$ величини ζ за формулами [2, с. 157-161]:

$$\exp(d)\colon\ x = -d\ln\zeta; \qquad \mathrm{unif}(d,s)\colon\ x = d - s + 2s\zeta; \qquad \mathrm{norm}(d,s)\colon\ x = d + s\,z, \quad (2.9)$$

де z – стандартна нормальна величина, отримана полярним методом Марсальї – Брея:

$$v_1 = 2\zeta_1 - 1, \quad v_2 = 2\zeta_2 - 1, \quad w = v_1^2 + v_2^2, \quad z_{1,2} = v_{1,2}\sqrt{\frac{-2\ln w}{w}}. \quad (2.10)$$

Пара ζ_1, ζ_2 генерується знову, доки не виконається умова $0 < w \le 1$, а один розрахунок дає два значення z. Якщо $t_{\text{обр1}} \le 0$, значення генерується повторно.

2.6 Стан мережі, правила функціонування та початковий стан

Стан мережі $S(t)$ у момент t описують такі змінні:

– $q_S(t)$ – кількість деталей типу S, на які ще не призначено робота: q_1 – у черзі площадки A, q_2 і q_3 – оброблені деталі на місцях M1 і M2;

– $K_{ij}(t)$ – стан j-го місця верстата Mi ($i = 1,2$; $j = 1,2,3$): 0 – вільне; 1 – вивільнення (робот кладе деталь); 2 – обробка; 3 – блоковане (оброблена деталь чекає на робота); 4 – захоплення (робот знімає деталь);

– $K_r(t)$ – стан робота r: 0 – вільний; 1-5 – номер поточної фази з таблиці 2.5;

– $S_r(t)$ – тип деталі, яку обслуговує робот r;

– $pos_r(t)$ – точка, у якій стоїть вільний робот r; після вивільнення $pos_r = dst_{S_r}$.

Похідні величини, що входять в умови блокування і вихідні характеристики:

$$n_i(t) = \sum_{j=1}^{3} \mathbf{1}\{K_{ij}(t) \neq 0\}, \qquad f_i(t) = 3 - n_i(t), \qquad N_{\text{віл}}(t) = \sum_{r} \mathbf{1}\{K_r(t) = 0\}, \tag{2.11}$$

$$u_i(t) = \sum_{r} \mathbf{1}\{S_r(t) = i,\ K_r(t) \in \{1,2,3,4\}\},$$
$$L_A(t) = q_1(t) + \sum_{r} \mathbf{1}\{S_r(t) = 1,\ K_r(t) \in \{1,2\}\}, \tag{2.12}$$

де $L_A(t)$ – кількість деталей, що фізично лежать на площадці A: деталь залишається на площадці, поки робот їде по неї і захоплює її. Деталь типу S везуть на верстат MS, тому для u_i рахуються роботи з $S_r = i$.

Мережа функціонує за такими правилами:

1) деталь, що надійшла, стає в кінець черги площадки A; деталі одного типу обслуговуються в порядку готовності до перевезення (FIFO);

2) після обробки на верстаті Mi місце переходить у стан 3, а деталь стає запитом типу $S = i + 1$ і чекає на робота на цьому місці;

3) у варіанті 1 вільний робот RS починає обслуговування, щойно на початку його маршруту є деталь і маршрут відкритий; у варіанті 2 роботи призначаються за правилом (2.4);

4) кінець фази 2 звільняє місце, з якого знято деталь: на площадці L_A зменшується на 1, на верстаті $K_{ij} := 0$; якщо біля цього верстата чекає заблокований робот, він одразу займає звільнене місце;

5) вивільнення займає вільне місце з найменшим номером ($K_{ij} := 1$); після вивільнення починається обробка ($K_{ij} := 2$), а робот стає вільним у точці dst_S; вивільнення на складі завершує проходження деталі;

6) події, які настають в один момент часу, обробляються у випадковому порядку, причому завершення обслуговування обробляються раніше за надходження нової деталі; вільні роботи призначаються після обробки всіх подій цього моменту.

У початковий момент $t = 0$ черга площадки порожня, усі місця верстатів вільні, роботи вільні й стоять у точках завантаження маршрутів варіанта 1: R1 – в A, R2 – в M1, R3 – в M2. Для варіанта 2 прийнято той самий стан. Перша деталь надходить у момент $\tau_{A,1}$.

2.7 Обчислення вихідних характеристик та вимоги до точності

Статистика збирається на інтервалі $[T_{\text{п}};\ T_{\text{mod}}]$ тривалістю $T = T_{\text{mod}} - T_{\text{п}}$, де T_{mod} – час моделювання, $T_{\text{п}}$ – тривалість перехідного періоду. Інтеграли за часом обчислюються під час імітації як суми $\sum_k f\,(t_k)\,(t_{k+1} - t_k)$ за інтервалами між послідовними подіями. Формули обчислення вихідних характеристик наведено в таблиці 2.6.

Таблиця 2.6 – Формули обчислення вихідних характеристик

Характеристика	Формула обчислення
Пропускна здатність, дет./год	$\Lambda = 3600\,N_W/T$, де N_W – кількість деталей, вивільнених на складі за час T
Час проходження i-ї деталі, с	$\tau_i = t_i^W - t_i^A$ – від надходження на площадку A до кінця вивільнення на складі
Середній час проходження і його середнє квадратичне відхилення, с	$\bar{\tau} = \frac{1}{N_W}\sum_i \tau_i\,, \qquad s_\tau = \sqrt{\frac{1}{N_W - 1}\sum_i (\tau_i - \bar{\tau})^2}$

Характеристика	Формула обчислення
Розподіл часу проходження	гістограма відносних частот $h_k = N_k/N$, де N_k – кількість значень τ_i у k-му інтервалі; відповідність закону розподілу перевіряється за критерієм χ^2
Коефіцієнт використання робота r	$\rho_r = \frac{1}{T}\int_{T_п}^{T_{mod}} \mathbf{1}\{K_r(t) \neq 0\}\, dt$ – разом із холостим ходом і очікуванням з деталлю
Частка фази k у часі роботи робота	$\rho_r^{(k)} = \frac{1}{T}\int_{T_п}^{T_{mod}} \mathbf{1}\{K_r(t) = k\}\, dt, \qquad \rho_r = \sum_{k=1}^{5} \rho_r^{(k)}$
Коефіцієнт використання верстата Мi	$\rho_{Mi} = \frac{1}{3T}\int_{T_п}^{T_{mod}} n_i\,(t)\, dt$
Частка блокування верстата Мi	$\rho_{Mi}^{бл} = \frac{1}{3T}\int_{T_п}^{T_{mod}} \sum_{j=1}^{3} \mathbf{1}\{K_{ij}(t) = 3\}\, dt$
Максимальна місткість місця зберігання на ділянці прибуття, шт.	$L_{max} = \max_{t\in[T_п;\, T_{mod}]} L_A(t)$
Середня кількість деталей на площадці А, шт.	$\bar{L}_A = \frac{1}{T}\int_{T_п}^{T_{mod}} L_A\,(t)\, dt$

Інтеграли недоречні, стан змінюється дискретно, не буде будь-якого малого dt.

Кожна вихідна характеристика оцінюється як середнє $\bar{x}$ за серією з p незалежних прогонів моделі з різними початковими значеннями генераторів. Кількість прогонів визначається за нерівністю Чебишова [3, с. 81]:

$$p = \frac{\sigma^2}{\varepsilon^2(1-\beta)}, \qquad \beta = 0{,}95 \;\Rightarrow\; p = 20\left(\frac{\sigma}{\varepsilon}\right)^2, \tag{2.13}$$

де σ – середнє квадратичне відхилення характеристики за пілотною серією з 20 прогонів; ε – допустима похибка; β – довірча ймовірність. Для пропускної здатності, часу проходження і коефіцієнтів використання прийнято $\varepsilon = 0{,}02\,\bar{x}$, тобто 2% від середнього значення. Для максимальної місткості площадки, яка є цілим числом, точність задано абсолютною – 1-2 деталі. Тривалість перехідного періоду і кількість прогонів визначено в підрозділі 4.2.

2.8 Висновки до розділу

У розділі побудовано формалізовані моделі ГВС для обох способів закріплення роботів у термінах мережі масового обслуговування. У варіанті 1 кожен робот є окремою одноканальною СМО, у варіанті 2 роботи утворюють одну триканальну СМО, через яку деталь проходить тричі як запит різних типів.

Рух робота порожнім враховано в часовій затримці пристрою. Умови «немає місця на верстаті» і «немає вільного робота» описано предикатами блокування маршрутів. Для варіанта 2 сформульовано правило вибору найближчого робота й обмеження, що виключає взаємне блокування роботів. Визначено змінні стану, правила функціонування, початковий стан, формули обчислення вихідних характеристик і вимоги до точності їх оцінювання.

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ

У розділі описано алгоритм імітації, побудований за формалізованою моделлю розділу 2, і його програмну реалізацію: просування часу, просування стану моделі в часі, призначення роботів, генерування випадкових величин і збір статистики. Алгоритм імітації власний: спеціалізовані бібліотеки імітації не використовуються. Повний код наведено в додатках Б і В, а також у репозиторії проєкту.

3.1 Обґрунтування вибору мови програмування

Для програмної реалізації обрано мову Python [4]. Вона дає змогу компактно описати елементи мережі МО класами, а модуль `dataclasses` – задати параметри і структури даних без службового коду. Python також має розвинені бібліотеки для статистичної обробки результатів. Бібліотеки NumPy [6] і SciPy [7] використано лише на етапі експериментального дослідження: для обчислення квантилів розподілів Стьюдента, Фішера і χ^2, оцінювання параметрів законів розподілу і дисперсійного аналізу. Графіки побудовано бібліотекою Matplotlib [8]. Сам алгоритм імітації використовує лише стандартну бібліотеку Python.

Недоліком Python є менша швидкодія порівняно з компільованими мовами C++ і Java. Для цієї моделі вона достатня: один прогін тривалістю 200 000 с модельного часу виконується приблизно за 0,5 с. Серії незалежних прогонів виконуються паралельно в окремих процесах за допомогою модуля `concurrent.futures`. Коректність окремих частин алгоритму перевіряється автоматичними тестами (бібліотека pytest).

3.2 Структура програми

Програма є пакетом `fms` із модулями, наведеними в таблиці 3.1.

Таблиця 3.1 – Модулі програми

Модуль	Призначення
`rng.py`	генератори випадкових величин: окремі потоки чисел для кожного джерела випадковості, закони exp, unif, norm

Модуль	Призначення
`params.py`	вхідні змінні та параметри моделі, спосіб закріплення роботів
`elements.py`	елементи мережі МО: надходження, черга, верстат, СМО роботів, склад, а також деталь і заявка на перевезення
`model.py`	алгоритм імітації: просування часу, призначення роботів, збір і обчислення вихідних характеристик
`experiments/`	експериментальне дослідження: верифікація, перехідний період, кількість прогонів, порівняння варіантів, межа стійкості, розподіл часу проходження, пробні запуски пропозицій
`cli.py`	командний інтерфейс для запуску прогонів та експериментів
`report_plots.py`	побудова рисунків за результатами експериментів

Варіант закріплення роботів задається параметром `strategy`: `dedicated` – варіант 1, `nearest` – варіант 2. Обидва варіанти реалізовано тим самим алгоритмом. Різниця полягає лише в тому, які маршрути дозволено кожному роботу і як роботи згруповано в СМО.

3.3 Алгоритм просування часу

У програмі застосовано принцип просування часу до найближчої події. Подіями є надходження деталі, завершення обробки на місці верстата і завершення кожної фази обслуговування роботом. Кожен елемент мережі зберігає момент своєї найближчої події, а модель на кожному кроці обирає найменший із них. Псевдокод алгоритму імітації має вигляд:

Алгоритм 3.1 Імітаційне моделювання дискретної системи

вхід: вхідні змінні X, параметри P, поточний час t=0, час моделювання Tmod, перехідний період Tп

Ви мали розробити алгоритм на основі формалізму мережі МО. У цьому алгоритмі нічого від мережі МО немає

використовує метод переходу стану E(S,t)

ініціалізувати поточний стан моделі S: черга порожня, місця вільні, роботи в A, M1, M2

призначити вільних роботів на деталі, що чекають

поки t < Tmod

знайти момент найближчої події tnext := min(момент надходження, моменти кінця обробки, моменти кінця фаз роботів)

якщо Tп ∈ (t; tnext], **оновити** статистику на [t; Tп] і **обнулити** статистику

оновити статистику вихідних змінних Y на інтервалі [t; tnext]

перемістити поточний час t := tnext

викликати перетворення стану S := E(S,t):

для кожного елемента з подією в момент t (у випадковому порядку) **виконати** подію: кінець обробки або кінець фази робота
якщо в момент t надходить деталь, поставити її в чергу площадки A
поки існує допустима пара «вільний робот – деталь» (2.3):
призначити пару за правилом (2.4), почати фазу 1
кінець поки
обчислити статистику вихідних змінних Y за формулами таблиці 2.6
вивести вихідні змінні Y

Статистика оновлюється перед просуванням часу, тому кожен інтервал між подіями враховується зі станом, який був на ньому. Події, що настають одночасно, виконуються у випадковому порядку, бо це найбільш правильний підхід за відсутності явного пріоритету [3, с. 66]. Завершення обслуговування виконуються раніше за надходження нової деталі. Роботи призначаються після обробки всіх подій моменту, коли відомі всі деталі, що чекають, і всі вільні роботи. Програмна реалізація головного циклу (метод `Model.run`, спрощено: прибрано виклики спостерігача, що використовуються для аналізу перехідного періоду):

```
def run(self, t_end, t_warmup=0.0):
    pending_reset = t_warmup > 0
    self.dispatch(0.0)
    while True:
        t_next = self._next_event_time()       # найближча подія
        if pending_reset and t_warmup <= min(t_next, t_end):
            self._advance_stats(t_warmup)      # кінець перехідного періоду:
            self.reset_stats()                 # статистика обнуляється
            pending_reset = False
        if t_next > t_end:
            self._advance_stats(t_end)
            break
        self._advance_stats(t_next)            # статистика на [tcurr; t_next]
        t = self.tcurr                         # просування часу
        due = [e for e in self.event_elements if e.next_time() == t]
        self.streams.ties.shuffle(due)         # одночасні події – випадково
        for e in due:
            e.out_act(t)                       # завершення обслуговування
        if self.create.next_time() == t:
            self.create.out_act(t)             # надходження – останнім
        self.dispatch(t)                       # призначення вільних роботів
    return self
```

Наприкінці перехідного періоду обнуляються лише накопичені статистики. Стан мережі (деталі в черзі, на верстатах і в роботів) при цьому не змінюється.

3.4 Події та зміна стану моделі

Події моделі і зміни стану, які вони спричиняють, наведено в таблиці 3.2. Позначення станів відповідають підрозділу 2.6.

Таблиця 3.2 – Події моделі та зміна стану

немає такої події у формалізмі мереж МО

це однакова подія, якщо використовувати формалізм мережі МО

Подія	Елемент	Зміна стану
надходження деталі	`Create`	L_A і q_1 збільшуються на 1; створюється заявка на перевезення типу 1; планується наступне надходження в момент $t + \tau_A$
призначення робота	`Model.dispatch`	$K_r := 1$ або одразу $K_r := 2$, якщо робот уже в точці завантаження; q_S зменшується на 1; u_i верстата-приймача збільшується на 1
кінець холостого ходу (фаза 1)	`Transport`	$K_r := 2$; місце верстата-джерела переходить у стан 4; планується кінець захоплення $t + t_{зх}$
кінець захоплення (фаза 2)	`Transport`	місце-джерело звільняється: L_A зменшується на 1 або $K_{ij} := 0$; робот, що чекає біля цього верстата, займає місце; $K_r := 3$, планується прибуття $t + d_S$
кінець переміщення (фаза 3)	`Transport`	якщо приймач – склад або на верстаті є вільне місце: $K_{ij} := 1$, u_i зменшується на 1, $K_r := 5$; інакше $K_r := 4$, робот стає в чергу роботів, що чекають біля верстата
кінець вивільнення (фаза 5)	`Transport`	на верстаті $K_{ij} := 2$, планується кінець обробки $t + t_{обрі}$; на складі фіксується час проходження τ_i; робот вільний у точці dst_S
кінець обробки	`Machine`	$K_{ij} := 3$ (місце блоковане); створюється заявка на перевезення типу $S = i + 1$

Події завершення фаз обслуговування обробляє метод `Transport.out_act`. За поточною фазою робота він визначає наступну:

```
def out_act(self, t):
    due = [r for r in self.robots if r.tnext == t]
    self.model.streams.ties.shuffle(due)
    for robot in due:
        if robot.state == R_TO_PICKUP:      # кінець холостого ходу
            self._start_grab(robot, t)
        elif robot.state == R_GRAB:         # кінець захоплення
            self._start_carry(robot, t)
        elif robot.state == R_CARRY:        # прибуття до приймача
            self._arrive(robot, t)
        elif robot.state == R_RELEASE:      # кінець вивільнення
            self._finish(robot, t)
```

Блокування B_1 і B_3 перевіряються в момент прибуття робота до верстата. Якщо вільного місця немає, робот не планує наступної події ($t_{next} = \infty$) і стає в чергу роботів, що чекають біля верстата:

```
def _arrive(self, robot, t):
    route = robot.route
```

```
    robot.location = route.drop
    if isinstance(route.dest, Machine):
        slot = route.dest.take_for_release()
        if slot is None:                    # маршрут заблоковано
            robot.state = R_WAIT            # робот тримає деталь
            robot.tnext = INF
            route.dest.waiting_robots.append(robot)
            return
        self.start_release(robot, slot, t)
    else:                                   # склад приймає завжди
        self.start_release(robot, None, t)
```

і де ж перевірки виразів, визначених у розділі 2?

реалізація не відповідає формального опису з розділу 2

Блокування знімається подією «кінець захоплення» на цьому верстаті. Метод `Machine.grab_done` звільняє місце і, якщо біля верстата чекає робот, одразу віддає йому звільнене місце і починає вивільнення в той самий момент. Лічильник `incoming` верстата зберігає величину u_i, тобто кількість призначених на доставку сюди роботів, які ще не зайняли місця. Метод `can_dispatch_to` перевіряє умову $u_i \le f_i$ з предикатів B_0 і B_2:

```
def can_dispatch_to(self):              # u_i <= f_i
    return self.incoming <= self.count(FREE)

def take_for_release(self):             # місце для вивільнення або None
    ch = self._first(FREE)              # вільне місце з найменшим номером
    if ch is None:
        return None
    self.incoming -= 1
    ch.state = LOADING
    return ch

def grab_done(self, ch, t):             # деталь знято з місця
    ch.state = FREE
    ch.part = None
    if self.waiting_robots:             # розблокування робота, що чекає
        robot = self.waiting_robots.popleft()
        slot = self.take_for_release()
        robot.transport.start_release(robot, slot, t)
```

3.5 Призначення роботів

Призначення роботів виконує метод `Model.dispatch` після обробки всіх подій поточного моменту. Заявки на перевезення зберігаються в трьох чергах FIFO, по одній на кожен маршрут. Для кожного вільного робота і кожного дозволеного йому маршруту з непорожньою чергою і відкритим входом обчислюється ключ вибору за формулою (2.4). Обирається пара з найменшим ключем, і пошук повторюється, доки допустимі пари не закінчаться:

```
def dispatch(self, t):
    rank = self.params.route_rank
    while True:
        best = None
```

```
for tr in self.transports:
    for robot in tr.free_robots():
        for r in robot.allowed_routes:
            queue = self.tasks[r]
            if not queue or self.routes[r].blocked():
                continue
            route = self.routes[r]
            key = (self.params.distance(robot.location, route.pickup),
                   rank[r - 1], queue[0].t_ready,
                   self.streams.ties.random())
            if best is None or key < best[0]:
                best = (key, tr, robot, r)
if best is None:
    return
_, tr, robot, r = best
tr.assign(robot, self.tasks[r].popleft(), t)
```

У варіанті 1 кожному роботу дозволено один маршрут (`allowed_routes = (S,)`), і кожен робот утворює окрему СМО `Transport`. Тому той самий метод призначає роботу лише деталь його маршруту, а відстань холостого ходу в ключі нічого не обирає. У варіанті 2 всі роботи належать одній СМО `Transport` з дозволеними маршрутами `(1, 2, 3)`, і ключ реалізує правило найближчого робота. Метод `Route.blocked` перевіряє умову $u_i > f_i$ верстата-приймача (не більше одного зайвого робота на верстат), а маршрут на склад не блокується. Умова «немає вільного робота» в предикатах B_2 і B_4 виконується автоматично: якщо вільних роботів немає, допустимих пар немає, і заявки лишаються в чергах, а деталі – на місцях верстатів.

Метод `Transport.assign` починає обслуговування: збільшує лічильник u_i верстата-приймача і планує кінець холостого ходу $t + d(pos_r, src_s)$. Якщо робот уже стоїть у точці завантаження, захоплення починається одразу.

3.6 Генератори випадкових величин

Базову рівномірно розподілену на $(0; 1]$ величину ζ отримує генератор Mersenne Twister з модуля `random` стандартної бібліотеки. Усі інші закони обчислюються власними формулами (2.9) і (2.10). Кожне джерело випадковості моделі має окремий потік чисел із власним зерном: інтервали надходження, обробка на M1, обробка на M2, захоплення і вивільнення, а також вибір серед одночасних подій. Завдяки цьому варіанти 1 і 2 з однаковим зерном отримують однакові послідовності надходжень і часів обробки (спільні випадкові числа).

Різниця результатів тоді пояснюється способом закріплення роботів, а не випадковістю. Фрагмент класу потоку `Stream`:

```
def zeta(self):                          # рівномірна на (0; 1]
    return 1.0 - self._rnd.random()

def exponential(self, mean):             # exp(d): x = -d ln ζ
    return -mean * math.log(self.zeta())

def uniform(self, center, half_width):   # unif(d, s): x = d - s + 2sζ
    return center - half_width + 2.0 * half_width * self._rnd.random()

def standard_normal(self):               # полярний метод Марсальї – Брея
    if self._spare_normal is not None:   # друге значення попередньої пари
        value, self._spare_normal = self._spare_normal, None
        return value
    while True:
        a1 = 2.0 * self._rnd.random() - 1.0
        a2 = 2.0 * self._rnd.random() - 1.0
        s = a1 * a1 + a2 * a2
        if 0.0 < s <= 1.0:
            break
    beta = math.sqrt(-2.0 * math.log(s) / s)
    self._spare_normal = a2 * beta
    return a1 * beta
```

Час обробки на M1 генерується як norm(60,10) з повторним генеруванням від'ємних значень. Правильність генераторів перевірено автоматичними тестами: для вибірки з 200 000 значень середнє і середнє квадратичне відхилення збігаються з теоретичними з похибкою до 1 і 2% відповідно.

3.7 Збір статистики

Статистика збирається методом `update_stats(dt)` кожного елемента, який викликається перед кожним просуванням часу на інтервал dt. Елементи накопичують інтеграли за часом із таблиці 2.6:

– черга площадки А накопичує $\sum L_A \, \Delta t$, розподіл часу за кількістю деталей і найбільшу кількість деталей L_{max};

– верстат для кожного місця накопичує тривалість кожного стану (вільне, вивільнення, обробка, блоковане, захоплення);

– СМО роботів для кожного робота накопичує тривалість кожного стану: вільний, холостий хід, захоплення, переміщення з деталлю, очікування з деталлю, вивільнення. **забагато станів, у формалізмі мереж МО для пристрою стільки не визначено**

Для часу проходження кожна деталь зберігає момент надходження і момент завершення вивільнення на складі, а також моменти готовності до перевезення і початку захоплення на кожному маршруті. Наприкінці прогону функція `collect_results` за формулами таблиці 2.6 обчислює всі вихідні характеристики за період $[T_{\text{п}};\ T_{\text{mod}}]$: пропускну здатність, середнє, середнє квадратичне відхилення і квантилі часу проходження, коефіцієнти використання та частки станів верстатів і роботів, а також L_{max}, $\bar{L}_A$ і кількість деталей у системі в кінці прогону. Масив часів проходження використовується для побудови гістограми та перевірки закону розподілу.

3.8 Опис класів та UML-діаграма класів

Класи програми та їх призначення наведено в таблиці 3.3, UML-діаграму [9] класів – на рисунку 3.1.

Таблиця 3.3 – Класи програми

Клас	Призначення	Вміст класу
`Params`	вхідні змінні та параметри моделі	закони розподілу, кількість місць і роботів, розташування пунктів, спосіб закріплення; `distance`, `robot_routes`
`Dist`	закон розподілу в позначеннях методичних рекомендацій	вид закону і параметри; `sample(stream)`
`Stream`, `RandomStreams`	потоки випадкових чисел, окремий для кожного джерела випадковості	`exponential`, `uniform`, `normal`, `shuffle`
`Model`	мережа МО і алгоритм імітації	елементи мережі, маршрути, черги заявок, роботи; `run`, `dispatch`, `results`
`Element`	базовий клас елемента мережі	`next_time`, `out_act`, `update_stats`, `reset_stats`
`Create`	зовнішнє надходження деталей	момент наступного надходження
`Queue`	черга площадки А	кількість деталей, максимум, накопичені статистики
`Machine`, `MachineChannel`	верстат – СМО з трьох місць без черги, місце верстата	стани місць, лічильник u_i, роботи, що чекають; `can_dispatch_to`, `take_for_release`, `grab_done`
`Transport`, `Robot`	СМО роботів і робот як її пристрій	фаза робота, його положення, поточна заявка, дозволені маршрути; `assign`, `out_act`
`Warehouse`	склад – вихід з мережі	`accept` фіксує час проходження

Клас	Призначення	Вміст класу
`Part`, `Task`, `Route`	деталь (запит мережі), заявка на перевезення, маршрут	моменти подій деталі; тип і момент готовності заявки; точки і елементи маршруту, `blocked`

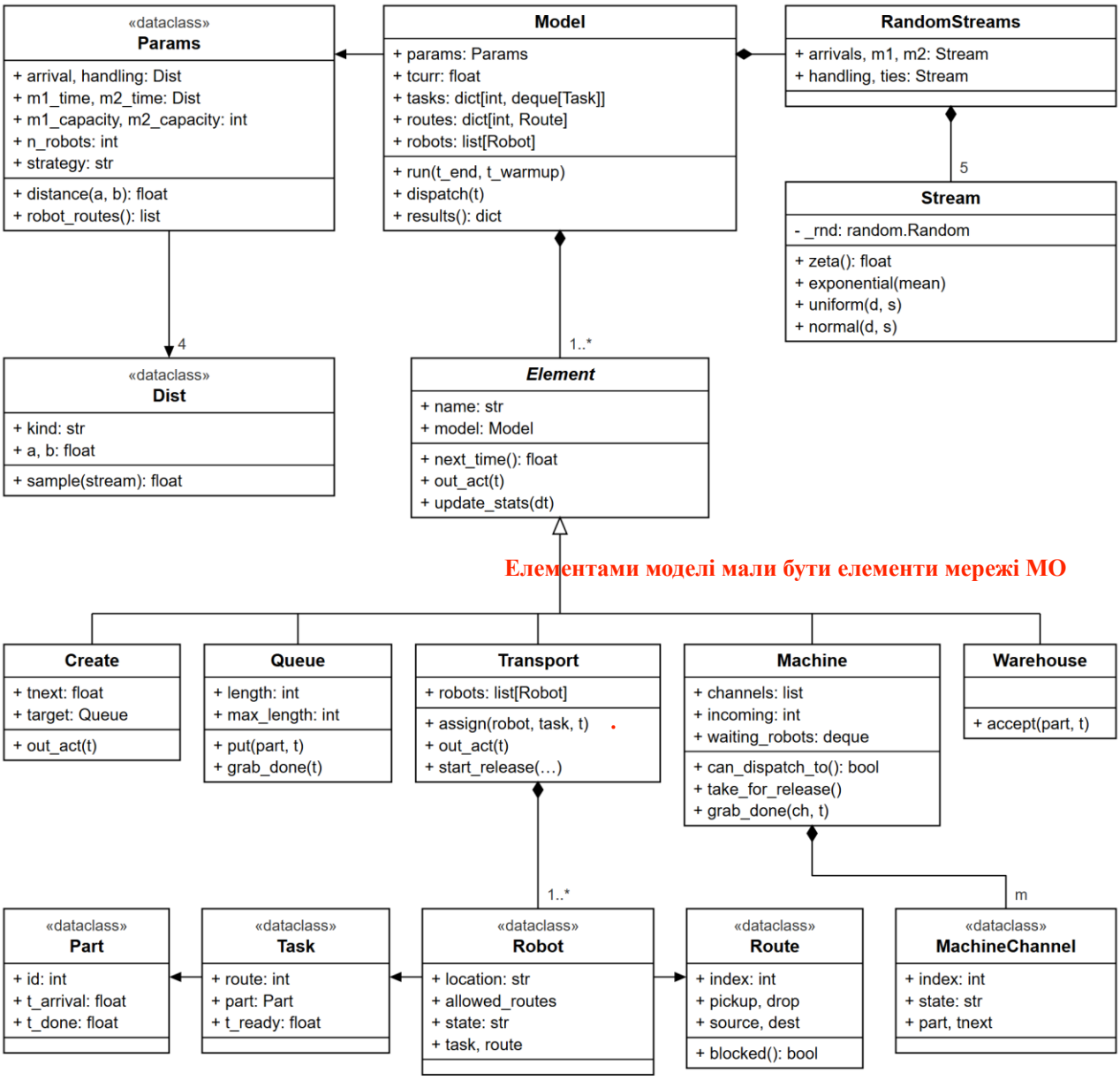


Рисунок 3.1 – UML-діаграма класів імітаційної моделі

Класи `Create`, `Queue`, `Machine`, `Transport` і `Warehouse` успадковують `Element`. Тому головний цикл однаково звертається до будь-якого елемента: запитує момент найближчої події, виконує подію і оновлює статистику.

3.9 Верифікація моделі

Коректність алгоритму імітації доведено верифікацією, яку виконують варіюванням вхідних змінних моделі. Розглянуто 9 сценаріїв, наведених у таблиці 3.4. Кожен сценарій змінює одну вхідну змінну або групу змінних відносно базових значень (таблиця 1.2), і для кожного сформульовано очікувану реакцію моделі. У таблиці k_d – множник часу переміщення роботів (базове значення 1), m_2 – кількість місць на верстаті М2.

Таблиця 3.4 – Сценарії верифікації: змінені вхідні змінні та очікувана реакція моделі

№	Змінена вхідна змінна	Очікувана реакція моделі
1	базові значення (таблиця 1.2)	базовий прогін для порівняння
2	$\tau_A = \exp(80)$	система недовантажена: $\Lambda \approx 3600/80 = 45$ дет./год, черга на площадці мала, коефіцієнти використання менші, ніж у базовому прогоні
3	$\tau_A = \exp(30)$	перевантаження сильніше: Λ не зростає, бо її обмежує М2; черга і час проходження більші, ніж у базовому прогоні
4	$t_{\text{обр2}} = \exp(50)$	М2 перестає бути вузьким місцем: $\Lambda \approx 90$ дет./год, черга не накопичується, час проходження різко менший
5	$t_{\text{обр1}} = \text{norm}(120{,}10)$	вузьким місцем стає М1: $\rho_{\text{M1}} \approx 1$, Λ менша, ніж у базовому прогоні
6	$m_2 = 6$	як у сценарії 4: $\Lambda \approx 90$ дет./год, черга не накопичується
7	$t_{\text{зх}},\ t_{\text{вив}} = \text{unif}(2;\ 0{,}5)$	роботи швидші: їх коефіцієнт використання менший, Λ більша
8	$k_d = 3$	роботи повільніші: їх коефіцієнт використання більший, Λ менша, час проходження більший
9	усі часи детерміновані: $\tau_A = 300$, $t_{\text{обр1}} = 60$, $t_{\text{обр2}} = 100$, $t_{\text{зх}} = t_{\text{вив}} = 8$ с	$\Lambda = 3600/300 = 12$ дет./год; перша деталь проходить систему за 226 с, наступні за 244 с (розрахунок у підрозділі 3.9)

Для кожного сценарію і кожного варіанта виконано 5 прогонів тривалістю $T_{\text{mod}} = 100\,000$ с модельного часу. Статистика збиралася після перехідного періоду $T_{\text{п}} = 10\,000$ с. Змінні, не зазначені в таблиці 3.4, мали базові значення, а для сценарію 9 базовими були час переміщення і кількість місць. Середні за 5 прогонами значення вихідних змінних наведено в таблицях 3.5 і 3.6, де $\bar{\rho}_{\text{R}}$ – середній коефіцієнт використання роботів, $N_{\text{кін}}$ – кількість деталей у системі в кінці прогону.

Таблиця 3.5 – Результати верифікації, варіант 1

№	Λ, дет./год	$\bar{\tau}$, с	L_{max}, шт.	$\bar{L}_A$, шт.	ρ_{M1}	ρ_{M2}	$\bar{\rho}_R$	$N_{кін}$, шт.
1	80,8	6128	286,4	148,6	0,999	0,962	0,858	288,0
2	44,6	278	6,2	0,3	0,390	0,516	0,374	2,6
3	80,8	18 214	1109,2	606,3	1,000	0,963	0,859	1116,8
4	90,9	310	17,4	2,6	0,804	0,685	0,749	8,0
5	68,9	13 063	614,4	326,7	1,000	0,814	0,745	619,2
6	90,8	333	15,8	1,9	0,771	0,576	0,726	8,2
7	90,8	956	55,8	17,0	0,945	0,932	0,721	21,2
8	57,8	19 783	920,8	497,2	1,000	0,807	0,913	926,4
9	12,0	244	1,0	0,0	0,092	0,134	0,093	1,0

Таблиця 3.6 – Результати верифікації, варіант 2

№	Λ, дет./год	$\bar{\tau}$, с	L_{max}, шт.	$\bar{L}_A$, шт.	ρ_{M1}	ρ_{M2}	$\bar{\rho}_R$	$N_{кін}$, шт.
1	76,6	8672	404,2	213,1	0,923	0,906	0,911	404,2
2	44,5	271	6,8	0,4	0,364	0,493	0,389	2,8
3	76,6	20 215	1228,8	672,9	0,923	0,906	0,911	1234,0
4	91,1	405	26,4	5,0	0,779	0,645	0,832	9,6
5	67,6	13 832	649,6	346,1	0,972	0,777	0,783	653,0
6	91,0	373	20,4	3,2	0,738	0,527	0,787	8,6
7	89,7	1436	70,8	29,2	0,943	0,918	0,782	51,6
8	54,2	21 910	1019,4	551,2	0,781	0,765	0,979	1023,6
9	12,0	244	1,0	0,1	0,084	0,129	0,093	1,0

Реакція моделі на зміну вхідних змінних в обох варіантах відповідає очікуваній:

1) у базовому прогоні черга на площадці накопичується: L_{max} і $N_{кін}$ досягають сотень деталей, а пропускна здатність (80,8 і 76,6 дет./год) менша за інтенсивність надходження 90 дет./год, тобто система перевантажена;

2) при $\exp(80)$ пропускна здатність дорівнює інтенсивності надходження 45 дет./год, черга мала, коефіцієнти використання зменшились приблизно вдвічі;

3) при $\exp(30)$ пропускна здатність не змінилась, а черга і час проходження зросли, тобто пропускну здатність обмежує обладнання, а не потік деталей;

4) прискорення обробки на М2 удвічі або 6 місць на М2 (сценарії 4 і 6) роблять систему стійкою: пропускна здатність близька до 90 дет./год, черга не накопичується; отже, вузьким місцем базової системи є верстат М2; незначне

перевищення 90 дет./год пояснюється тим, що за час спостереження на склад надходять і деталі, які були в системі на момент $T_{\text{п}}$;

5) при подвоєному часі обробки на M1 коефіцієнт використання M1 наближається до 1 (1,000 і 0,972), а пропускна здатність зменшується до 68,9 і 67,6 дет./год;

6) прискорення захоплення і вивільнення зменшує завантаження роботів і збільшує пропускну здатність, а потроєння часу переміщення діє навпаки; у варіанті 2 роботи при цьому завантажені майже повністю ($\bar{\rho}_{\text{R}} = 0{,}979$);

7) при детермінованих часах (сценарій 9) пропускна здатність дорівнює $3600/300 = 12$ дет./год, а час проходження – точно 244 с в обох варіантах.

Результат сценарію 9 збігається з розрахунком. Перша деталь проходить систему за $8 + 6 + 8 + 60 + 8 + 7 + 8 + 100 + 8 + 5 + 8 = 226$ с, бо роботи стоять у точках завантаження. Для наступних деталей у варіанті 1 кожен робот повертається своїм маршрутом, і додаються холості ходи $6 + 7 + 5 = 18$ с. У варіанті 2 друга і третя деталі проходять систему за 232 і 239 с, бо вільні роботи стоять ближче до точок завантаження. Далі всі роботи збираються на складі, і кожну деталь перевозить один робот: холостий хід зі складу до A триває 18 с, а перевезення M1 → M2 і M2 → W він виконує без холостого ходу. Тому час проходження також становить $226 + 18 = 244$ с. Ці значення перевіряються автоматичним тестом.

Додатково правильність алгоритму перевіряють 22 автоматичні тести, зокрема:

– закон збереження: кількість деталей, що надійшли, дорівнює сумі доставлених на склад і тих, що перебувають у системі;

– на верстаті ніколи не буває більше трьох зайнятих місць;

– система не зависає при сильному перевантаженні ($\exp(15)$);

– біля кожного верстата чекає не більше одного робота, а $u_i \le f_i + 1$;

– у варіанті 1 кожен робот виконує перевезення лише свого маршруту, у варіанті 2 – щонайменше двох маршрутів;

– однакове зерно генераторів дає однаковий результат.

Правильність послідовності подій підтверджує протокол подій, у якому для кожної події вказано момент часу, елемент, зміст події і стан мережі після неї. Фрагмент протоколу для варіанта 2 на початку моделювання:

```
 1.248  Create   надходження деталі #1
 1.248  Robot1   призначено на маршрут 1 (A->M1) для деталі #1,
                 холостий хід A->A 0
 1.248  Robot1   початок захоплення деталі #1 у A
 8.788  Robot1   деталь #1 захоплено, хід A->M1
14.788  Robot1   початок вивільнення деталі #1 у M1
16.321  Create   надходження деталі #2
16.321  Robot2   призначено на маршрут 1 (A->M1) для деталі #2,
                 холостий хід M1->A 6
        стан     A=1 | M1[1..] | M2[...] | R1:release@M1
                 R2:empty_move@M1 R3:free@M2
```

На момент надходження деталі №2 робот R1 зайнятий вивільненням, вільні R2 (у точці M1, холостий хід 6 с) і R3 (у точці M2, 13 с). Призначено найближчий робот R2, що відповідає правилу (2.4). Протоколи подій для обох варіантів наведено в додатку А.

3.10 Висновки до розділу

У розділі розроблено власний алгоритм імітації ГВС з просуванням часу до найближчої події, реалізовано його мовою Python. Описано обробку всіх подій моделі, реалізацію блокувань маршрутів, правила призначення роботів для обох варіантів, генератори випадкових величин і збір статистики. Обидва варіанти реалізовано тим самим алгоритмом, вони відрізняються лише дозволеними маршрутами роботів і групуванням роботів у СМО. Наведено опис класів і UML-діаграму класів. Верифікація варіюванням вхідних змінних у дев’яти сценаріях, детермінований сценарій з точно розрахованим результатом і автоматичні тести підтверджують коректність алгоритму.

4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ

У розділі визначено мету і план експериментального дослідження та умови проведення одного експерименту: тривалість перехідного періоду і кількість прогонів. Способи закріплення роботів порівняно методом дисперсійного аналізу. Досліджено межу стійкості системи, коефіцієнти використання обладнання, закон розподілу часу проходження деталей і місткість площадки прибуття. Фактором названо вхідну змінну або модифікацію моделі, вплив якої досліджується, а відгуком – вихідну змінну, що досліджується.

4.1 Мета і план експериментального дослідження

Мету використання моделі визначає завдання: знайти спосіб закріплення роботів до операцій переміщення, за якого пропускна здатність ГВС найбільша, і для кожного способу визначити розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів та максимальну місткість місця зберігання на ділянці прибуття. Головним відгуком моделі є пропускна здатність Λ. Додаткові відгуки – час проходження τ, коефіцієнти використання ρ і максимальна кількість деталей на площадці L_{max}.

Фактором є спосіб закріплення роботів. Це якісний фактор із двома рівнями: варіант 1 і варіант 2. Вплив такого фактора оцінюють дисперсійним аналізом, який показує, чи спричинена різниця відгуків зміною фактора, чи випадковістю [3, с. 82-85]. Результат одного експерименту є середнім значенням відгуку за серією з p незалежних прогонів при фіксованих вхідних змінних. Для часу проходження, крім середнього і середнього квадратичного відхилення, виконано ідентифікацію закону розподілу.

Умови експериментів обрано за результатами верифікації (підрозділ 3.9). За умови завдання, тобто при надходженні exp(40) або 90 дет./год, система перевантажена в обох варіантах: пропускна здатність менша за інтенсивність надходження, і черга на площадці А зростає необмежено. У такому режимі середня довжина черги і середній час проходження залежать від часу моделювання, тобто сталих значень не мають. Пропускна здатність, навпаки,

досягає найбільшого можливого значення і стабілізується. Тому способи закріплення порівнюються за пропускною здатністю і коефіцієнтами використання за умови завдання $\exp(40)$. Розподіл часу проходження і місткість площадки мають сенс лише в стійкому режимі. Для їх визначення інтервал надходження змінено на $\exp(60)$, тобто 60 дет./год, що менше за межу стійкості обох варіантів (підрозділ 4.4). Таку зміну числових характеристик моделі методичні рекомендації передбачають для випадку, коли сталий режим не досягається [3, с. 80]. Додатково як кількісний фактор досліджено інтервал надходження деталей. Залежність пропускної здатності від нього показує, за якого навантаження кожен варіант ще справляється з потоком деталей.

План експериментів наведено в таблиці 4.1. В усіх серіях прогін з номером i для обох варіантів має однакові початкові значення генераторів, тобто варіанти отримують спільні випадкові числа (підрозділ 3.6).

Таблиця 4.1 – План експериментального дослідження

№	Експеримент	Інтервал надходження	T_{mod}, с	$T_{\text{п}}$, с	Кількість прогонів	Відгук
1	тривалість перехідного періоду	$\exp(40)$; $\exp(60)$	300 000; 1 500 000	0	5	Λ, $\bar{\tau}$, $\bar{L}_A$, ρ_{M2}, $\bar{\rho}_{\text{R}}$
2	кількість прогонів (пілотна серія)	$\exp(40)$; $\exp(60)$	250 000; 1 500 000	50 000	20	Λ, $\bar{\tau}$, L_{max}, ρ_{M2}, $\bar{\rho}_{\text{R}}$
3	порівняння способів закріплення роботів	$\exp(40)$; $\exp(60)$	250 000; 1 500 000	50 000	30	усі вихідні змінні
4	межа стійкості	від $\exp(34)$ до $\exp(80)$, 17 значень	250 000	50 000	10	Λ, $N_{\text{кін}}$
5	розподіл часу проходження	$\exp(60)$	1 500 000	50 000	20	τ
6	пробні запуски пропозицій (підрозділ 5.2)	$\exp(40)$	250 000	50 000	20	Λ, $\bar{\tau}$, L_{max}, ρ

Час моделювання, перехідний період і кількість прогонів у серіях 2-6 обрано за результатами серій 1 і 2 (підрозділ 4.2).

4.2 Тривалість перехідного періоду та кількість прогонів

Перехідним є період, протягом якого модель виходить на сталі значення відгуків. На початку моделювання система порожня, тому значення, накопичені

в цей період, зміщують оцінки відгуків. Тривалість перехідного періоду визначено за п’ятьма пробними прогонами з різними початковими значеннями генераторів, тоді як методичні рекомендації вимагають щонайменше чотири. Для кожного прогону i з кроком 1000 с при $\exp(40)$ і 5000 с при $\exp(60)$ обчислювалось середнє значення відгуку $\bar{y}_i(t)$ на інтервалі $[0;\ t]$. Тривалістю перехідного періоду вважався найменший момент T, після якого середні значення всіх прогонів відрізняються менше ніж на задану точність ε [3, с. 80]:

$$T: \quad \forall i, j,\ \ \forall t \geq T:\ \ |\bar{y}_i(t) - \bar{y}_j(t)| < \varepsilon. \tag{4.1}$$

Умова перевіряється для всіх $t \geq T$, а не лише в момент T, щоб випадковий збіг кривих до виходу на сталий рівень не прийняти за кінець перехідного періоду. Для характеристик, які усереднюються за часом (ρ, $\bar{L}_A$), середнє обчислюється з урахуванням нерівних інтервалів просування часу:

$$\bar{y}_i(t) = \frac{\sum_{s=1}^{n} y_{is}\ \Delta t_s}{\sum_{s=1}^{n} \Delta\, t_s}, \qquad t = \sum_{s=1}^{n} \Delta\, t_s, \tag{4.2}$$

де Δt_s – інтервали, на які просувається час в алгоритмі імітації; y_{is} – значення величини на інтервалі s. Пропускна здатність на момент t дорівнює $3600\, N_W(t)/t$, середній час проходження – середньому за деталями, що надійшли на склад до моменту t. Точність ε задано як 5% від рівня відгуку, тобто від середнього за п’ятьма прогонами значення в кінці моделювання. Результати наведено в таблиці 4.2, динаміку пропускної здатності при $\exp(40)$ і середнього часу проходження при $\exp(60)$ – на рисунках 4.1 і 4.2.

Таблиця 4.2 – Тривалість перехідного періоду за умовою (4.1), 5 прогонів, ε = 5% від рівня відгуку

Відгук	Інтервал надходження	Варіант 1: рівень	Варіант 1: T, тис. с	Варіант 2: рівень	Варіант 2: T, тис. с
Λ, дет./год	$\exp(40)$	80,7	45	76,4	20
Λ, дет./год	$\exp(60)$	59,8	140	59,8	140
$\bar{\tau}$, с	$\exp(40)$	15 867	не досягається	22 939	не досягається
$\bar{\tau}$, с	$\exp(60)$	327	670	337	1480
$\bar{L}_A$, шт.	$\exp(40)$	389,0	не досягається	565,8	не досягається
$\bar{L}_A$, шт.	$\exp(60)$	0,89	не досягається	1,11	не досягається

Відгук	Інтервал надходження	Варіант 1: рівень	Варіант 1: T, тис. с	Варіант 2: рівень	Варіант 2: T, тис. с
ρ_{M2}	exp(40)	0,961	17	0,906	33
ρ_{M2}	exp(60)	0,702	220	0,679	230
$\bar{\rho}_R$	exp(40)	0,857	23	0,911	12
$\bar{\rho}_R$	exp(60)	0,542	245	0,587	280

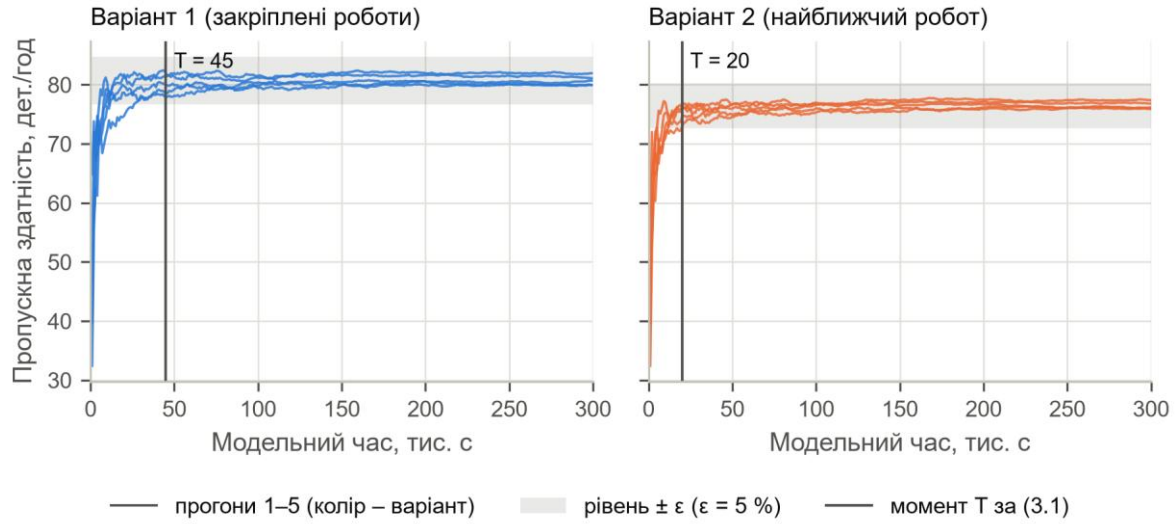


Рисунок 4.1 – Пропускна здатність ГВС, обчислена від початку моделювання, при надходженні exp(40): п’ять прогонів кожного варіанта

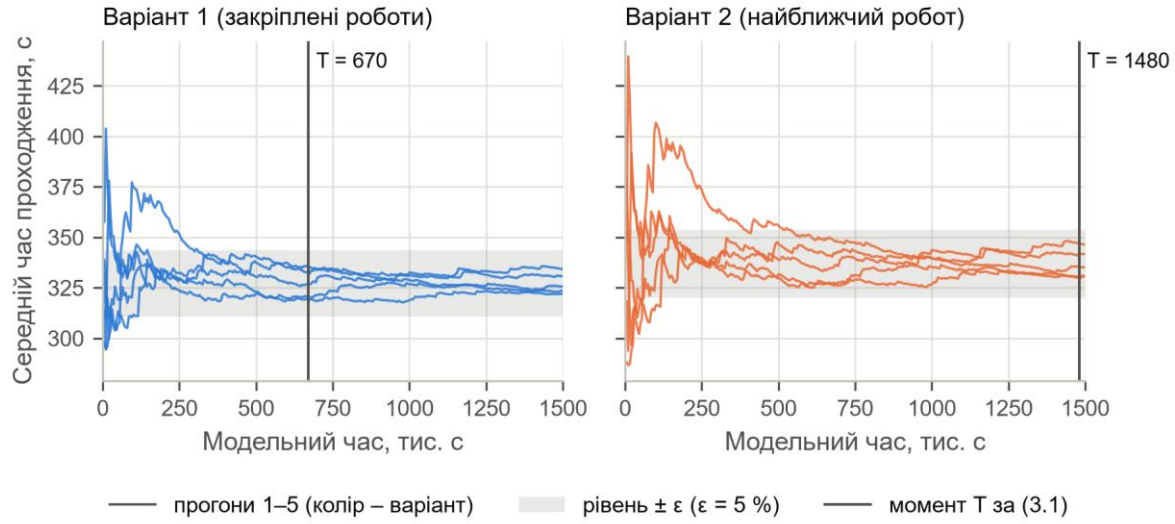


Рисунок 4.2 – Середній час проходження деталей, обчислений від початку моделювання, при надходженні exp(60): п’ять прогонів кожного варіанта

При exp(40) пропускна здатність і коефіцієнти використання стабілізуються, і умова (4.1) для них виконується не пізніше ніж через 45 000 с. Середній час проходження і середня кількість деталей на площадці умову (4.1)

не задовольняють. У другій половині прогону, від 150 000 до 300 000 с, їх середні значення зросли на 90-94%. Це ознака відсутності сталого режиму. Черга на площадці А зростає лінійно з часом моделювання (рисунок 4.3), тому середні значення на інтервалі $[0;\ t]$ зростають майже пропорційно t.

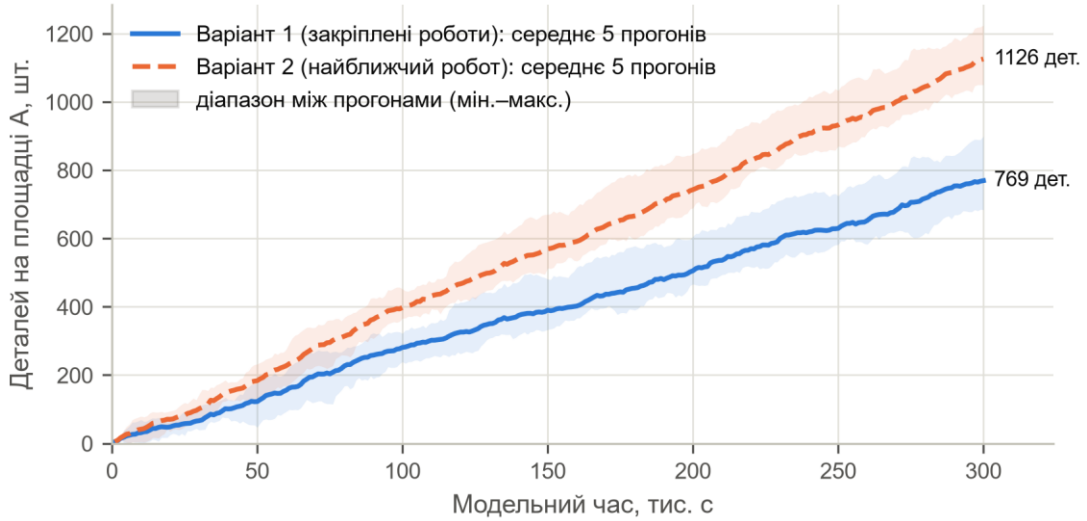


Рисунок 4.3 – Кількість деталей на площадці А при надходженні exp(40): середнє і діапазон значень за п’ятьма прогонами

При exp(60) система працює в стійкому режимі. Умова (4.1) виконується для всіх відгуків, крім $\bar{L}_A$: для пропускної здатності і коефіцієнтів використання T становить 140 0000-280 000 с, для середнього часу проходження – 670 000 с у варіанті 1 і 1 480 000 с у варіанті 2. Такі великі значення пояснюються переважно розкидом відгуку в окремому прогоні. Час обробки на М2 має експоненціальний закон, окремі деталі проходять систему за 1500-1900 с, і середнє за одним прогоном наближується до сталого рівня повільно (рисунок 4.2). Отже, при exp(60) умова (4.1) визначає час моделювання, потрібний для точності 5% уже в одному прогоні. Для $\bar{L}_A$ точність 5% від рівня становить лише 0,04-0,06 деталі і за 1 500 000 с не досягається. Проте в другій половині прогону середнє $\bar{L}_A$ не зростає, а зменшується на 1,5-2%, тобто сталий режим існує. Оскільки $\bar{L}_A$ є допоміжним відгуком, для нього меншу точність прийнято допустимою.

Вплив перехідного періоду на результат зменшено двома способами, які методичні рекомендації називають найбільш дієвими [3, с. 81]: статистика

збирається лише після перехідного періоду, а час моделювання в кілька разів перевищує перехідний період. Прийнято такі умови одного прогону:

– при $\exp(40)$ $T_{\text{п}} = 50\,000$ с, що більше за найбільше значення $T = 45\,000$ с для відгуків, які стабілізуються, а $T_{\text{mod}} = 250\,000$ с, тобто статистика збирається протягом 200 000 с, що в 4,4 раза довше за перехідний період;

– при $\exp(60)$ $T_{\text{mod}} = 1\,500\,000$ с, щоб умова (4.1) для середнього часу проходження виконувалась уже в одному прогоні, а $T_{\text{п}} = 50\,000$ с, щоб відкинути деталі, які пройшли систему, поки вона ще не заповнилась.

Кількість прогонів визначено за формулою (2.13), що випливає з нерівності Чебишова. Середнє квадратичне відхилення σ оцінено за пілотною серією з 20 прогонів з наведеними вище умовами, точність задано як $\varepsilon = 0{,}02\,\bar{x}$. Результати наведено в таблиці 4.3.

Таблиця 4.3 – Кількість прогонів за формулою (2.13), пілотна серія з 20 прогонів, ε = 2% від середнього

Відгук	Інтервал надходження	Варіант 1: $\bar{x}$	Варіант 1: σ	Варіант 1: p	Варіант 2: $\bar{x}$	Варіант 2: σ	Варіант 2: p
Λ, дет./год	$\exp(40)$	80,72	0,66	4	76,42	0,66	4
Λ, дет./год	$\exp(60)$	59,91	0,32	2	59,91	0,32	2
$\bar{\tau}$, с	$\exp(40)$	15 858,4	2228,5	—	22 868,8	2209,3	—
$\bar{\tau}$, с	$\exp(60)$	327,9	4,8	11	338,0	7,7	26
L_{max}, шт.	$\exp(40)$	653,6	78,3	—	946,1	78,1	—
L_{max}, шт.	$\exp(60)$	17,9	3,1	1461	20,7	3,9	1758
ρ_{M2}	$\exp(40)$	0,963	0,001	1	0,907	0,002	1
ρ_{M2}	$\exp(60)$	0,704	0,005	3	0,681	0,006	4
$\bar{\rho}_{\text{R}}$	$\exp(40)$	0,858	0,002	1	0,912	0,002	1
$\bar{\rho}_{\text{R}}$	$\exp(60)$	0,544	0,004	4	0,589	0,006	5

Для пропускної здатності і коефіцієнтів використання достатньо 1-5 прогонів, для середнього часу проходження при $\exp(60)$ – 11 у варіанті 1 і 26 у варіанті 2. Для $\bar{\tau}$ і L_{max} при $\exp(40)$ кількість прогонів не визначалась, бо ці відгуки сталих значень не мають. Для L_{max} при $\exp(60)$ відносна точність 2%, тобто 0,4 деталі, потребувала б 1461-1758 прогонів. Ціле значення L_{max} не потребує такої точності, тому для нього точність задано абсолютною – 1-2 деталі (підрозділ 2.7).

Для порівняння варіантів прийнято $p = 30$ прогонів, що більше за найбільшу потрібну кількість 26. Точність, яку забезпечує серія з p прогонів, випливає з формули (2.13):

$$\varepsilon = \sigma\sqrt{20/p}. \tag{4.3}$$

При $p = 30$ пропускна здатність при $\exp(40)$ оцінюється з точністю $\pm 0{,}54$ дет./год (0,7%), середній час проходження при $\exp(60)$ – $\pm 3{,}9$ с у варіанті 1 і $\pm 6{,}3$ с у варіанті 2 (1,2 і 1,9%), L_{max} – $\pm 2{,}5$ і $\pm 3{,}2$ деталі. Нерівність Чебишова справедлива за будь-якого закону розподілу відгуку, тому дає гарантовану, але завищену оцінку. Оцінка кількості прогонів за центральною граничною теоремою, $p = (1{,}96\,\sigma/\varepsilon)^2$, приблизно в п’ять разів менша [2, с. 226-228]. Середнє за 30 прогонами розподілене близько до нормального закону, тому 95%-й довірчий інтервал за розподілом Стьюдента для L_{max} становить $\pm 1{,}3$ і $\pm 1{,}7$ деталі (таблиця 4.4), що відповідає заданій точності 1-2 деталі. У серіях 4 і 6 головним відгуком є пропускна здатність, для якої достатньо 4 прогонів, тому 10 і 20 прогонів у цих серіях забезпечують задану точність.

4.3 Порівняння способів закріплення роботів

Порівняння виконано за планом однофакторного експерименту з якісним фактором [2, с. 238]. Фактор має $q = 2$ рівні, на кожному рівні виконано $p = 30$ прогонів. Позначимо y_{ij} значення відгуку в i-му прогоні на j-му рівні фактора, $\bar{y}_j$ – середнє значення на j-му рівні, $\bar{y}$ – загальне середнє. Факторна сума квадратів відхилень характеризує вплив фактора, залишкова – вплив випадкових причин:

$$S_{\text{факт}} = p\sum_{j=1}^{q}(\bar{y}_j - \bar{y})^2, \qquad S_{\text{залиш}} = \sum_{j=1}^{q}\sum_{i=1}^{p}(y_{ij} - \bar{y}_j)^2. \tag{4.4}$$

Факторну і залишкову дисперсії отримують діленням цих сум на кількість ступенів вільності і порівнюють за критерієм Фішера:

$$F = \frac{d_{\text{факт}}}{d_{\text{залиш}}} = \frac{S_{\text{факт}}/(q-1)}{S_{\text{залиш}}/(q(p-1))}. \tag{4.5}$$

Якщо $F > F_{\text{кр}}$, вплив фактора значущий. Критичне значення при рівні значущості $\alpha = 0{,}05$ і ступенях вільності $q - 1 = 1$ і $q(p-1) = 58$ дорівнює $F_{\text{кр}} = 4{,}01$. Знаменники у формулі (4.5) взято за посібником [2, с. 239]. У формулі (3.10) методичних рекомендацій факторну суму поділено на $qp - 1$, тобто на кількість ступенів вільності загальної дисперсії [3, с. 85]. Тоді значення F зменшилося б у $(qp-1)/(q-1) = 59$ разів, і його вже не можна було б порівнювати з квантилем розподілу Фішера з $(1;\ 58)$ ступенями вільності.

Варіанти отримують спільні випадкові числа, тому прогони з однаковим номером у двох варіантах залежні: у них однакові моменти надходження деталей і часи обробки. Дисперсійний аналіз цю залежність не враховує, і за спільних випадкових чисел його висновок про значущість стає обережнішим. Тому додатково обчислено середню різницю відгуків у парах прогонів $\bar{y}_2 - \bar{y}_1$ з 95%-м довірчим інтервалом за розподілом Стьюдента. Вихідні характеристики обох варіантів наведено в таблиці 4.4, результати дисперсійного аналізу – в таблиці 4.5.

Таблиця 4.4 – Вихідні характеристики ГВС для двох способів закріплення роботів (середнє ± півширина 95%-го довірчого інтервалу за 30 прогонами)

Характеристика	exp(40): варіант 1	exp(40): варіант 2	exp(60): варіант 1	exp(60): варіант 2
Λ, дет./год	80,73 ± 0,32	76,42 ± 0,29	59,91 ± 0,13	59,91 ± 0,13
$\bar{\tau}$, с	зростає	зростає	327,7 ± 1,6	337,8 ± 2,5
90%-квантиль τ, с	зростає	зростає	530,8 ± 4,0	567,6 ± 6,0
L_{max}, шт.	зростає	зростає	18,0 ± 1,3	20,9 ± 1,7
$\bar{L}_A$, шт.	зростає	зростає	0,90 ± 0,02	1,12 ± 0,03
99%-квантиль L_A, шт.	зростає	зростає	8,1 ± 0,3	10,1 ± 0,6
ρ_{M1}	1,000 ± 0,000	0,922 ± 0,001	0,593 ± 0,002	0,569 ± 0,003
ρ_{M2}	0,963 ± 0,001	0,907 ± 0,001	0,704 ± 0,002	0,681 ± 0,002
ρ_{R1}	1,000 ± 0,000	0,911 ± 0,001	0,553 ± 0,002	0,589 ± 0,003
ρ_{R2}	0,992 ± 0,000	0,913 ± 0,002	0,645 ± 0,002	0,589 ± 0,002
ρ_{R3}	0,583 ± 0,002	0,911 ± 0,002	0,433 ± 0,001	0,588 ± 0,002
$\bar{\rho}_{\text{R}}$	0,858 ± 0,001	0,912 ± 0,001	0,544 ± 0,002	0,589 ± 0,002

Таблиця 4.5 – Дисперсійний аналіз впливу способу закріплення роботів ($q = 2, p = 30, F_{\text{кр}} = 4{,}01$)

Відгук	Інтервал надходження	$\bar{y}_1$	$\bar{y}_2$	$\bar{y}_2 - \bar{y}_1$	F	Вплив фактора
Λ, дет./год	exp(40)	80,73	76,43	−4,30 ± 0,09	404,8	значущий
ρ_{M1}	exp(40)	1,000	0,922	−0,078 ± 0,001	75 836	значущий
ρ_{M2}	exp(40)	0,963	0,907	−0,056 ± 0,001	8435	значущий
$\bar{\rho}_{\text{R}}$	exp(40)	0,858	0,912	0,0533 ± 0,0004	10 341	значущий
Λ, дет./год	exp(60)	59,91	59,91	0,000 ± 0,001	$8{,}5 \cdot 10^{-7}$	незначущий
$\bar{\tau}$, с	exp(60)	327,7	337,8	10,1 ± 1,0	47,1	значущий
L_{max}, шт.	exp(60)	18,0	20,9	2,8 ± 0,8	7,29	значущий
$\bar{L}_{\text{A}}$, шт.	exp(60)	0,90	1,12	0,22 ± 0,02	129,6	значущий
ρ_{M1}	exp(60)	0,593	0,569	−0,0237 ± 0,0004	183,2	значущий
ρ_{M2}	exp(60)	0,704	0,681	−0,0234 ± 0,0002	260,7	значущий
$\bar{\rho}_{\text{R}}$	exp(60)	0,544	0,589	0,045 ± 0,001	1033	значущий

При exp(40) пропускна здатність варіанта 1 становить $80{,}73 \pm 0{,}32$ дет./год, варіанта 2 – $76{,}42 \pm 0{,}29$ дет./год. Значення $F = 404{,}8$ у сто разів перевищує критичне, тобто спосіб закріплення роботів істотно впливає на пропускну здатність. У парах прогонів пропускна здатність варіанта 2 менша на $4{,}30 \pm 0{,}09$ дет./год, тобто на 5,3%. Вплив фактора значущий і для коефіцієнтів використання. У варіанті 2 верстати завантажені менше, а роботи в середньому більше: $\bar{\rho}_{\text{R}} = 0{,}912$ проти 0,858.

При exp(60) обидва варіанти обробляють усі деталі, що надходять. Пропускна здатність дорівнює інтенсивності надходження і від способу закріплення не залежить ($F \approx 0$). Отже, у стійкому режимі пропускна здатність не розрізняє варіанти, і її порівнюють за перевантаження або за межею стійкості (підрозділ 4.4). Для решти відгуків вплив фактора значущий. Середній час проходження у варіанті 2 більший на $10{,}1 \pm 1{,}0$ с (3,1%), 90%-квантиль – на 37 с, L_{max} – на $2{,}8 \pm 0{,}8$ деталі. Роботи у варіанті 2 завантажені більше ($\bar{\rho}_{\text{R}} = 0{,}589$ проти 0,544), верстати – менше. Отже, за всіма відгуками, які стосуються мети моделювання, варіант 1 кращий.

4.4 Пропускна здатність залежно від інтенсивності надходження деталей

Щоб визначити, з яким потоком деталей справляється кожен варіант, як фактор розглянуто середній інтервал надходження τ_A. Досліджено 17 значень від 34 до 80 с, причому поблизу межі стійкості, від 44 до 50 с, інтервал змінювався з кроком 1 с. Для кожного значення виконано 10 прогонів (серія 4 таблиці 4.1). Режим вважався стійким, якщо пропускна здатність становить не менше 98% інтенсивності надходження $\lambda = 3600/\tau_A$, а в системі в кінці прогону менше 30 деталей. Результати наведено в таблиці 4.6 і на рисунку 4.4.

Таблиця 4.6 – Пропускна здатність ГВС залежно від інтервалу надходження деталей (10 прогонів)

τ_A, с	λ, дет./год	Варіант 1: Λ, дет./год	Варіант 1: $N_{кін}$, шт.	Варіант 1: режим	Варіант 2: Λ, дет./год	Варіант 2: $N_{кін}$, шт.	Варіант 2: режим
34	105,9	80,7	1750	нестійкий	76,4	2049	нестійкий
36	100,0	80,7	1344	нестійкий	76,4	1643	нестійкий
38	94,7	80,7	987	нестійкий	76,4	1286	нестійкий
40	90,0	80,7	661	нестійкий	76,4	958	нестійкий
42	85,7	80,7	376	нестійкий	76,4	669	нестійкий
44	81,8	80,6	127	нестійкий	76,4	408	нестійкий
45	80,0	79,9	54	нестійкий	76,4	290	нестійкий
46	78,3	78,6	29	стійкий	76,3	180	нестійкий
47	76,6	77,0	20	стійкий	76,1	86	нестійкий
48	75,0	75,4	13	стійкий	75,2	32	нестійкий
49	73,5	73,9	9	стійкий	73,8	16	стійкий
50	72,0	72,4	10	стійкий	72,3	14	стійкий
52	69,2	69,6	10	стійкий	69,5	13	стійкий
55	65,5	65,7	7	стійкий	65,7	7	стійкий
60	60,0	60,4	5	стійкий	60,4	6	стійкий
70	51,4	51,9	4	стійкий	51,9	4	стійкий
80	45,0	45,4	4	стійкий	45,4	4	стійкий

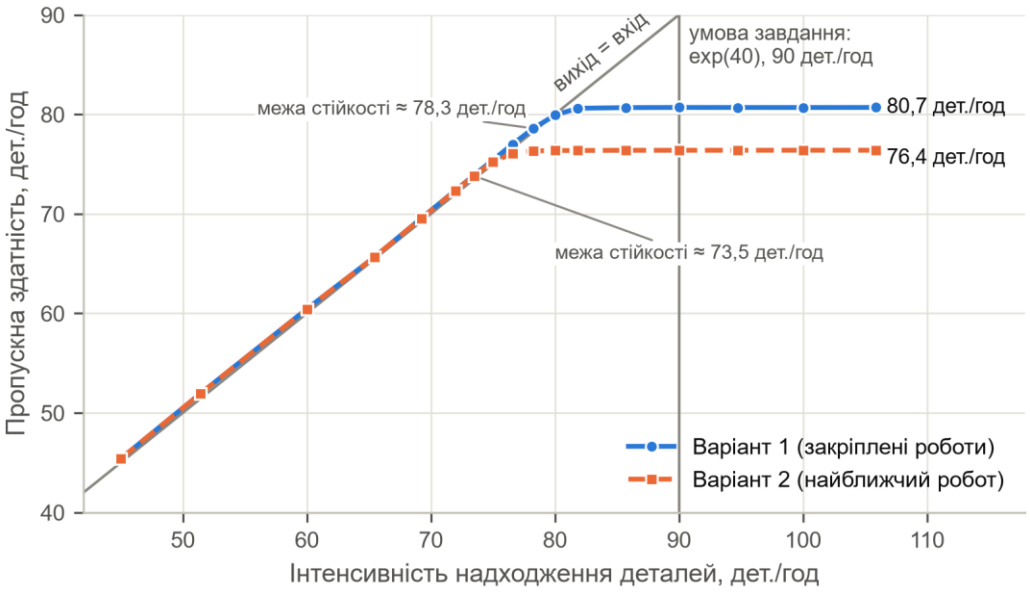


Рисунок 4.4 – Залежність пропускної здатності ГВС від інтенсивності надходження деталей

Поки інтенсивність надходження не перевищує 73,5 дет./год, обидва варіанти обробляють усі деталі, і $\Lambda \approx \lambda$. Подальше збільшення потоку пропускну здатність не збільшує: вона досягає граничного значення 80,7 дет./год у варіанті 1 і 76,4 дет./год у варіанті 2 і далі не змінюється. Тобто граничну пропускну здатність визначає обладнання, а не потік деталей. Межа стійкості варіанта 1 лежить між 78,3 дет./год (режим стійкий) і 80,0 дет./год (режим нестійкий), варіанта 2 – між 73,5 і 75,0 дет./год. За інтенсивності від 75 до 78 дет./год стійкий лише варіант 1. За умови завдання, 90 дет./год, обидва варіанти перевантажені. Щоб система справлялася з таким потоком, її граничну пропускну здатність треба збільшити щонайменше на 12% для варіанта 1 і на 18% для варіанта 2 (підрозділ 5.2).

4.5 Коефіцієнти використання роботів і верстатів

Коефіцієнти використання роботів і верстатів при exp(40) наведено на рисунку 4.5, розподіл часу роботи кожного робота за фазами обслуговування – на рисунку 4.6. Числові значення з довірчими інтервалами для обох інтервалів надходження наведено в таблиці 4.4.

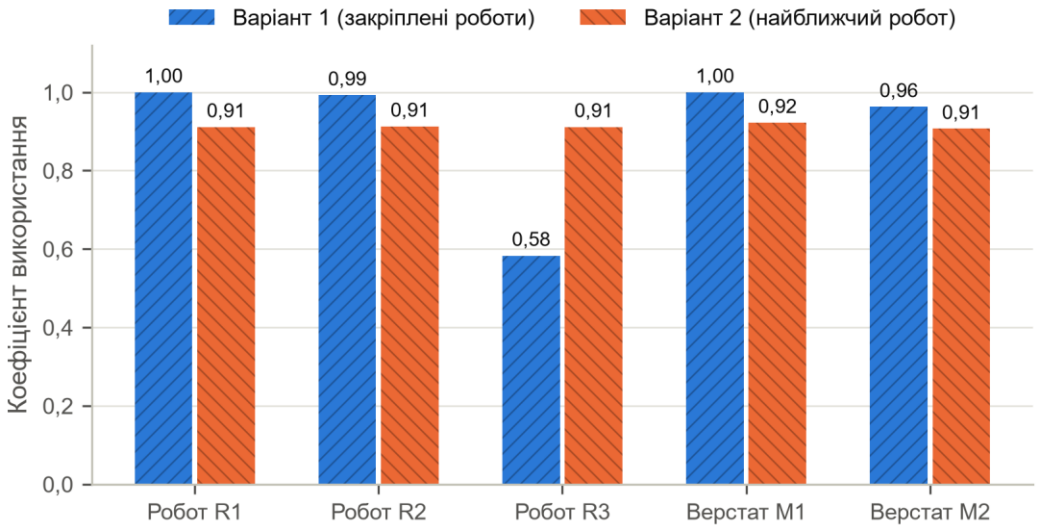


Рисунок 4.5 – Коефіцієнти використання роботів і верстатів при надходженні exp(40)

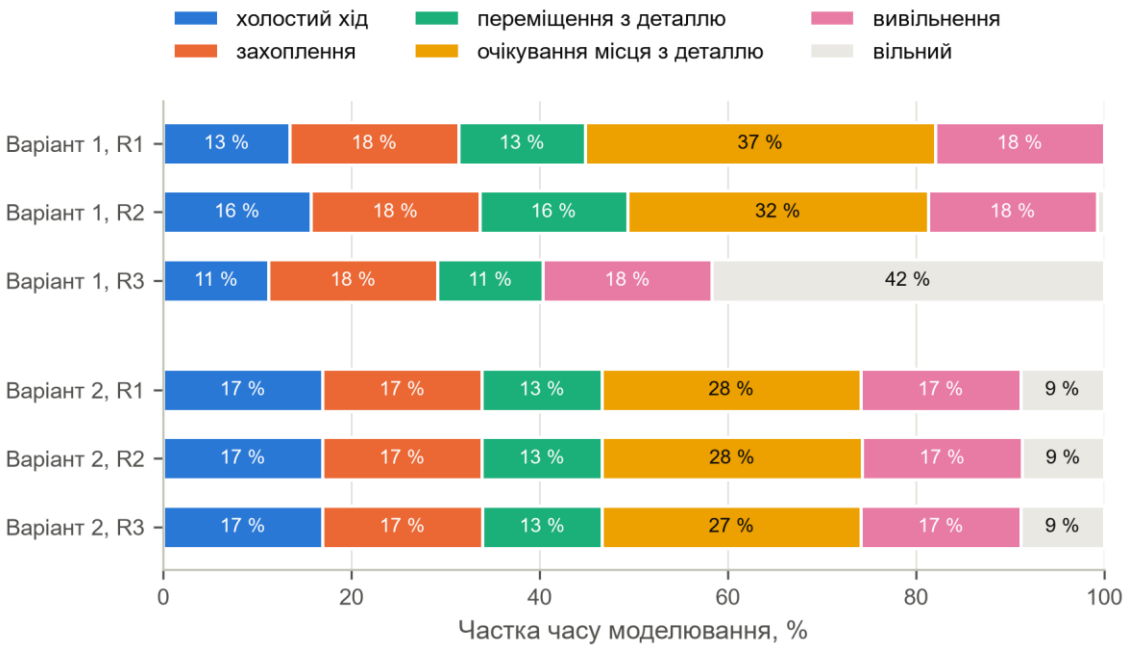


Рисунок 4.6 – Розподіл часу роботів за фазами обслуговування при надходженні exp(40)

У варіанті 1 роботи завантажені нерівномірно: $\rho_{R1} = 1{,}000$, $\rho_{R2} = 0{,}992$, $\rho_{R3} = 0{,}583$. Велику частку часу R1 і R2 не перевозять деталі, а тримають їх біля заповненого верстата: R1 – 37% часу біля M1, R2 – 32% біля M2. Робот R3 доставляє деталі на склад, який приймає їх завжди, тому R3 ніколи не чекає, а 42% часу вільний. Верстат M1 зайнятий весь час ($\rho_{M1} = 1{,}000$), але власне обробка займає лише 45% часу кожного місця, а 43% часу оброблена деталь

чекає на робота R2. Отже, M1 заповнений не через власну тривалість обробки, а через те, що R2 стоїть із деталлю біля заповненого M2. Верстат M2 зайнятий 96% часу: обробка займає 75%, блокування – 9,5%, вивільнення і захоплення – 12%.

У варіанті 2 всі три роботи завантажені однаково, на 91%. Холостий хід займає 17% часу кожного робота, тоді як у варіанті 1 – від 11 до 16%, очікування з деталлю біля заповненого верстата – 27-28%, ще 9% часу робот вільний. Коефіцієнти використання верстатів менші, ніж у варіанті 1: $\rho_{\mathrm{M1}} = 0{,}922$, $\rho_{\mathrm{M2}} = 0{,}907$. Причини цього пояснено в підрозділі 5.1.

При $\exp(60)$ завантаження всіх елементів менше, і найбільш завантаженим є верстат M2: $\rho_{\mathrm{M2}} = 0{,}704$ у варіанті 1 і 0,681 у варіанті 2. Коефіцієнти використання роботів у варіанті 1 становлять 0,553, 0,645 і 0,433, у варіанті 2 – по 0,589.

4.6 Розподіл часу проходження деталей

Закон розподілу часу проходження ідентифіковано в такій послідовності: формування масиву спостережуваних значень, побудова гістограми частот, формування гіпотези про вид закону, визначення параметрів закону і перевірка відповідності за критерієм згоди χ^2.

Масив значень сформовано за 20 прогонами при $\exp(60)$ з $T_{\mathrm{mod}} = 1\,500\,000$ с і $T_{\mathrm{п}} = 50\,000$ с (серія 5 таблиці 4.1). За один прогін на склад надходить близько 24 100 деталей. Деталі, які проходять систему одна за одною, затримуються в тих самих чергах, тому їх часи проходження залежні. Щоб зменшити цю залежність, з кожного прогону взято кожну п’яту деталь, разом 96 500 значень. На такій вибірці критерій χ^2 відхиляє всі розглянуті закони: найменші значення χ^2 становлять 952 і 1149 при критичному 38,9. При дуже великому обсязі вибірки критерій виявляє навіть незначні відхилення від теоретичного закону, які не мають практичного значення. Тому для перевірки з масиву рівномірно відібрано $n = 500$ значень. Сусідні значення в цій вибірці розділені приблизно 965 деталями, тому їх можна вважати незалежними. Статистичні характеристики вибірки наведено в таблиці 4.7.

Таблиця 4.7 – Статистичні характеристики вибірки часу проходження деталей, $\exp(60)$

Характеристика	Варіант 1	Варіант 2
обсяг вибірки n	500	500
середнє $\bar{\tau}$, с	317,7	333,6
середнє квадратичне відхилення s_τ, с	140,3	168,3
коефіцієнт варіації $s_\tau/\bar{\tau}$	0,442	0,505
найменше значення, с	132,9	119,4
медіана, с	278,9	291,7
90%-квантиль, с	489,8	546,7
95%-квантиль, с	618,2	664,2
найбільше значення, с	994,1	1472,9
коефіцієнт асиметрії	1,55	1,86

Гістограми відносних частот побудовано на 12 інтервалах однакової ширини між найменшим і найбільшим значеннями вибірки (рисунки 4.7 і 4.8). Гіпотези про вид закону сформульовано за виглядом гістограм і характеристиками вибірки:

– розподіл несиметричний, з довгим правим «хвостом»: коефіцієнт асиметрії 1,55 і 1,86, тому нормальний закон малоймовірний; його перевірено для порівняння;

– час проходження не може бути меншим за суму найменших тривалостей операцій, і найменше значення у вибірці становить 119-133 с, тому розглянуто закони зі зсувом c, тобто з нижньою межею;

– час проходження складається з кількох послідовних затримок, а найбільший розкид дає експоненціальна обробка на М2. Тому розглянуто експоненціальний закон зі зсувом, гамма-закон без зсуву і зі зсувом, а також логнормальний закон зі зсувом, який часто описує тривалості з правою асиметрією.

Щільності законів, що описують вибірку найкраще:

$$f(\tau) = \frac{(\tau - c)^{k-1}}{\Gamma(k)\,\theta^k}\, e^{-(\tau-c)/\theta}, \qquad \tau > c, \tag{4.6}$$

$$f(\tau) = \frac{1}{(\tau - c)\, s\sqrt{2\pi}} \exp\left(-\frac{(\ln(\tau - c) - \mu)^2}{2s^2}\right), \qquad \tau > c, \tag{4.7}$$

де (4.6) – гамма-закон зі зсувом c, параметром форми k і параметром масштабу θ; (4.7) – логнормальний закон зі зсувом c і параметрами μ, s логарифма величини $\tau - c$. Середнє значення гамма-закону дорівнює $c + k\theta$. Параметри законів оцінено методом найбільшої правдоподібності функцією `fit` бібліотеки SciPy [7]. Для гамма-закону зі зсувом пошук починається з оцінок методу моментів $k = (2/\gamma)^2$, $\theta = s_\tau/\sqrt{k}$, $c = \bar{\tau} - k\theta$, де γ – коефіцієнт асиметрії. Для експоненціального закону зі зсувом прийнято $c = \tau_{min}$, $m = \bar{\tau} - \tau_{min}$.

Відповідність закону перевірено за критерієм χ^2 [2, с. 27-28]:

$$\chi^2 = \sum_{l=1}^{h} \frac{(n_l - n\,p_l)^2}{n\,p_l}, \qquad \nu = h - 1 - r, \tag{4.8}$$

де n_l – кількість значень в l-му інтервалі; p_l – імовірність потрапляння в інтервал за гіпотетичним законом; h – кількість інтервалів; ν – кількість ступенів вільності; r – кількість оцінених параметрів закону (2 або 3). Крайні інтервали продовжено до $-\infty$ і $+\infty$, щоб сума p_l дорівнювала 1. Інтервали, для яких $n\,p_l < 5$, об'єднано із сусідніми [2, с. 28]. Гіпотеза приймається, якщо $\chi^2 < \chi^2_{\text{кр}}$ при рівні значущості 0,05. Результати перевірки наведено в таблиці 4.8, гістограми з кривими трьох найкращих законів – на рисунках 4.7 і 4.8.

Таблиця 4.8 – Перевірка гіпотез про закон розподілу часу проходження за критерієм χ^2 ($\alpha = 0{,}05$)

Варіант	Закон розподілу	Оцінка параметрів	χ^2	ν	$\chi^2_{\text{кр}}$	Гіпотеза
1	логнормальний зі зсувом	$s = 0{,}618;\ c = 99{,}2;\ \mu = 5{,}200$	5,68	6	12,59	приймається
1	гамма зі зсувом	$k = 1{,}857;\ c = 131{,}1;\ \theta = 100{,}5$	9,32	5	11,07	приймається
1	гамма без зсуву	$k = 6{,}259;\ \theta = 50{,}8$	37,71	5	11,07	відхиляється
1	експоненціальний зі зсувом	$c = 132{,}9;\ m = 184{,}7$	66,31	8	15,51	відхиляється
1	нормальний	$\mu = 317{,}7;\ \sigma = 140{,}1$	113,84	5	11,07	відхиляється
2	гамма зі зсувом	$k = 1{,}815;\ c = 118{,}1;\ \theta = 118{,}7$	1,32	3	7,81	приймається
2	логнормальний зі зсувом	$s = 0{,}679;\ c = 95{,}0;\ \mu = 5{,}252$	4,23	4	9,49	приймається
2	гамма без зсуву	$k = 4{,}951;\ \theta = 67{,}4$	15,13	3	7,81	відхиляється
2	експоненціальний зі зсувом	$c = 119{,}4;\ m = 214{,}2$	38,84	6	12,59	відхиляється

Варіант	Закон розподілу	Оцінка параметрів	χ^2	ν	$\chi^2_{кр}$	Гіпотеза
2	нормальний	$\mu = 333{,}6;\ \sigma = 168{,}2$	48,97	3	7,81	відхиляється

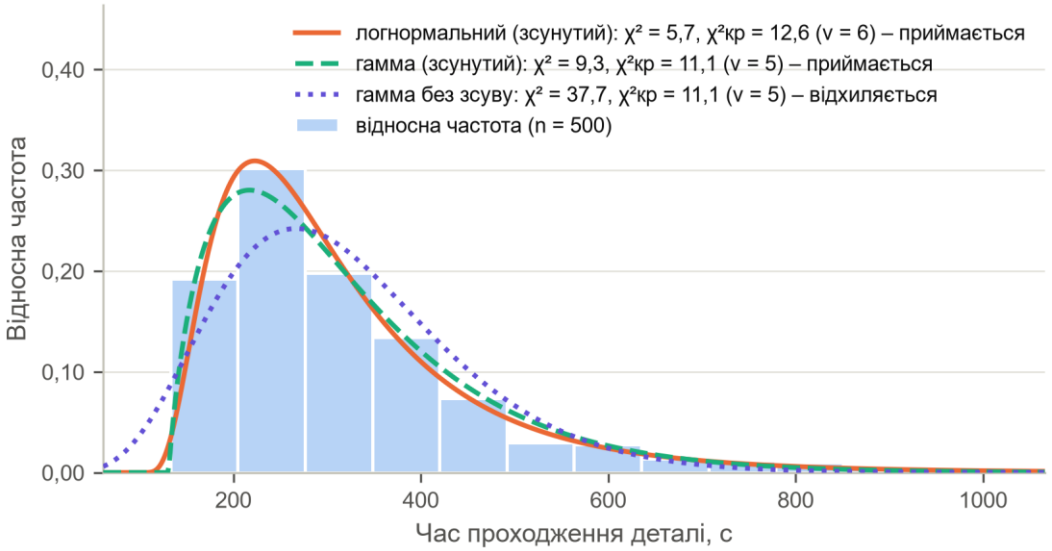


Рисунок 4.7 – Гістограма відносних частот часу проходження деталей і щільності законів розподілу, варіант 1

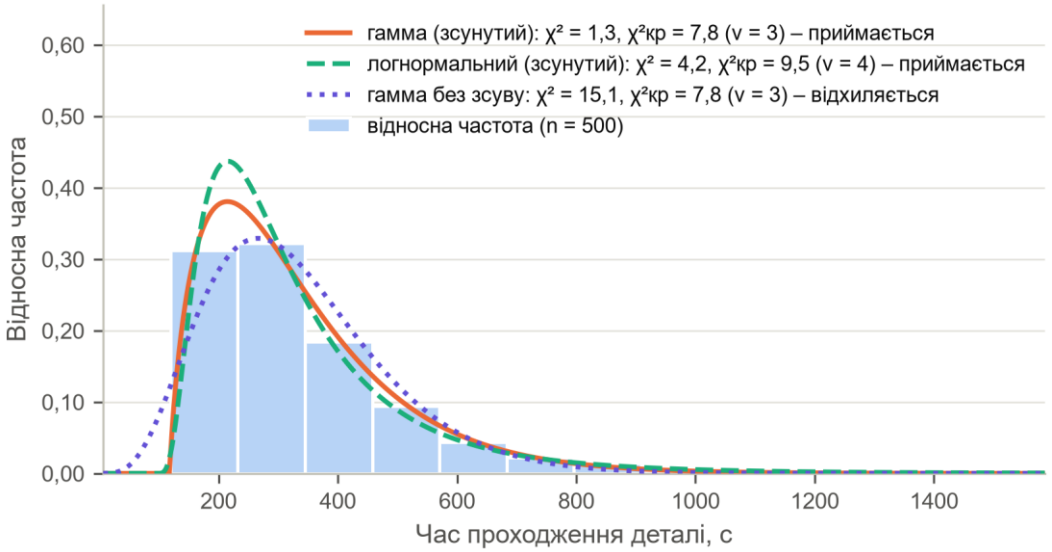


Рисунок 4.8 – Гістограма відносних частот часу проходження деталей і щільності законів розподілу, варіант 2

В обох варіантах приймаються гіпотези про гамма-закон зі зсувом і логнормальний закон зі зсувом, а нормальний, експоненціальний зі зсувом і гамма-закон без зсуву відхиляються. Відхилення гамма-закону без зсуву показує, що нижня межа часу проходження суттєва. Гамма-закон зі зсувом приймається

для обох варіантів, причому для варіанта 2 він найкращий. Тому його прийнято як закон розподілу часу проходження деталей в обох варіантах:

– варіант 1: $k = 1{,}857$, $c = 131{,}1$ с, $\theta = 100{,}5$ с, середнє $c + k\theta = 317{,}7$ с;

– варіант 2: $k = 1{,}815$, $c = 118{,}1$ с, $\theta = 118{,}7$ с, середнє $c + k\theta = 333{,}6$ с.

Параметри форми у варіантах майже однакові. У варіанті 2 зсув менший, тобто найшвидші деталі проходять систему швидше, бо вільний робот іноді вже стоїть у точці завантаження. Водночас параметр масштабу більший на 18%, тому більші середнє, розкид і частка довгих часів проходження: 90% деталей проходять систему не довше ніж за 531 с у варіанті 1 і 568 с у варіанті 2 (таблиця 4.4).

4.7 Максимальна місткість місця зберігання на ділянці прибуття

При $\exp(40)$ черга на площадці А зростає необмежено (рисунок 4.3): у середньому на $90 - 80{,}7 = 9{,}3$ деталі за годину у варіанті 1 і на $90 - 76{,}4 = 13{,}6$ деталі за годину у варіанті 2. Наприкінці прогону тривалістю 250 000 с на площадці в середньому 628 і 924 деталі. Отже, за умови завдання жодна скінченна місткість площадки не достатня, і L_{max} визначається лише тривалістю роботи системи.

При $\exp(60)$ найбільша кількість деталей на площадці за час спостереження 1 450 000 с (близько 400 год) становить $18{,}0 \pm 1{,}3$ у варіанті 1 і $20{,}9 \pm 1{,}7$ у варіанті 2. В окремих прогонах вона змінювалась від 14 до 29 деталей у варіанті 1 і від 17 до 38 у варіанті 2. Середня кількість деталей на площадці значно менша: 0,90 і 1,12, а 99% часу на ній не більше 8 і 10 деталей (таблиця 4.4). L_{max} є найбільшим значенням випадкового процесу, тому воно зростає зі збільшенням часу спостереження. Наприклад, у серії 4 з часом спостереження 200 000 с при $\exp(60)$ отримано L_{max} = 13,6 і 15,3 деталі. Місткість площадки також різко зростає з наближенням до межі стійкості: при $\exp(50)$ за 200 000 с L_{max} = 30,2 і 41,0 деталі.

4.8 Програмна реалізація експериментів

Експерименти реалізовано в пакеті `fms.experiments`. Кожна серія таблиці 4.1 відповідає окремому модулю і команді командного інтерфейсу. Результати зберігаються у файлах CSV у каталозі `results`, таблиці записки будуються з цих файлів окремим скриптом, а рисунки — модулем `report_plots.py`. Серії незалежних прогонів виконує функція `replicate`. Прогони з номером i мають зерно генераторів i, тому однакові для обох варіантів. Прогони виконуються паралельно в окремих процесах. Команди для відтворення всіх результатів розділу:

```
uv run fms warmup --arrival 40
uv run fms warmup --arrival 60 --t-end 1500000 --step 5000
uv run fms reps --arrival 40 --t-end 250000 --warmup 50000
uv run fms reps --arrival 60 --t-end 1500000 --warmup 50000
uv run fms compare --arrival 40 --reps 30 --t-end 250000 --warmup 50000
uv run fms compare --arrival 60 --reps 30 --t-end 1500000 --warmup 50000
uv run fms capacity --reps 10 --t-end 250000 --warmup 50000
uv run fms dist --arrival 60 --reps 20 --t-end 1500000 --warmup 50000 --max-n
500 --bins 12
uv run fms improve --reps 20 --t-end 250000 --warmup 50000
uv run fms plots
```

Тривалість перехідного періоду за умовою (4.1) визначає функція `transient_time`. Вона перебирає точки спостереження від кінця прогону до початку і зупиняється на першій точці, у якій розкид середніх значень прогонів не менший за ε:

```
def transient_time(series, metric, eps):
    """Найменший момент T, після якого max|y_i(t) - y_j(t)| < eps для всіх t >=
T."""
    runs = list(series.values())
    n_points = min(len(r) for r in runs)
    T = math.nan
    for k in range(n_points - 1, -1, -1):          # від кінця прогону до
початку
        vals = [r[k][metric] for r in runs]
        if any(math.isnan(v) for v in vals) or max(vals) - min(vals) >= eps:
            break
        T = runs[0][k]["t"]
    return T
```

Дисперсійний аналіз за формулами (4.4) і (4.5) виконує функція `anova` (спрощено: прибрано обчислення допоміжних величин):

```
def anova(groups, alpha=0.05):
    q, p = len(groups), len(groups[0])
    means = np.array([g.mean() for g in groups])
    grand = np.concatenate(groups).mean()
    s_f = p * ((means - grand) ** 2).sum()                    # (4.4)
```

```
    s_r = sum(((g - g.mean()) ** 2).sum() for g in groups)
    df1, df2 = q - 1, q * (p - 1)
    f = (s_f / df1) / (s_r / df2)                              # (4.5)
    f_crit = stats.f.ppf(1 - alpha, df1, df2)
    return f, f_crit, f > f_crit
```

Критерій χ^2 за формулою (4.8) з об'єднанням інтервалів обчислює функція `chi_square`:

```
def chi_square(data, dist, params, n_params, bins):
    counts, edges = np.histogram(data, bins=bins)
    cdf = dist.cdf(edges, *params)
    cdf[0], cdf[-1] = 0.0, 1.0              # крайні інтервали – до
нескінченності
    probs = np.diff(cdf)
    n = len(data)
    obs, exp_, acc_o, acc_e = [], [], 0.0, 0.0
    for o, p in zip(counts, probs):         # об'єднання інтервалів з n*p < 5
        acc_o += o
        acc_e += n * p
        if acc_e >= 5:
            obs.append(acc_o)
            exp_.append(acc_e)
            acc_o = acc_e = 0.0
    if acc_e > 0 and exp_:                  # залишок – до останнього інтервалу
        obs[-1] += acc_o
        exp_[-1] += acc_e
    obs, exp_ = np.array(obs), np.array(exp_)
    chi2 = float(((obs - exp_) ** 2 / exp_).sum())
    df = len(obs) - 1 - n_params
    return chi2, df, stats.chi2.ppf(0.95, df)
```

Повний код модулів експериментального дослідження наведено в додатку В.

4.9 Висновки до розділу

Експериментальне дослідження показало таке:

1) за умови завдання $\exp(40)$ система перевантажена в обох варіантах, тому способи закріплення роботів порівнюються за пропускною здатністю і коефіцієнтами використання, а розподіл часу проходження і місткість площадки визначено в стійкому режимі при $\exp(60)$;

2) перехідний період для відгуків, що стабілізуються, при $\exp(40)$ не перевищує 45 000 с. Прийнято $T_{\text{п}} = 50\,000$ с, $T_{\text{mod}} = 250\,000$ с при $\exp(40)$ і $T_{\text{mod}} = 1\,500\,000$ с при $\exp(60)$. Серія з 30 прогонів забезпечує точність не гірше 2% для пропускної здатності, часу проходження і коефіцієнтів використання та $\pm 1{,}3$–$1{,}7$ деталі для L_{max} за довірчим інтервалом;

3) спосіб закріплення роботів істотно впливає на пропускну здатність ($F = 404{,}8$ при $F_{\text{кр}} = 4{,}01$): у варіанті 1 вона становить 80,7 дет./год, у варіанті 2 – 76,4 дет./год. Межа стійкості варіанта 1 лежить у межах 78,3-80,0 дет./год, варіанта 2 – 73,5-75,0 дет./год;

4) у стійкому режимі варіант 1 також кращий: середній час проходження менший на 10 с, L_{max} – на 2,8 деталі;

5) час проходження деталей в обох варіантах розподілений за гамма-законом зі зсувом, середнє значення при $\exp(60)$ становить 328 с у варіанті 1 і 338 с у варіанті 2;

6) вузьким місцем системи є верстат М2. У варіанті 1 роботи R1 і R2 завантажені майже повністю, а R3 – на 58%. У варіанті 2 роботи завантажені рівномірно, на 91%, але більше часу витрачають на холостий хід.

Отже, найкращим з погляду пропускної здатності є варіант 1, закріплення по одному роботу за кожним шляхом переміщення деталей.

5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ СИСТЕМИ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ

У розділі сформульовано висновки щодо роботи ГВС і пропозиції щодо її поліпшення, перевірені пробними запусками моделі.

5.1 Інтерпретація результатів моделювання

Деяких умов роботи системи текст завдання не визначає, тому під час розробки моделі прийнято такі рішення:

1) що робить робот після вивільнення деталі, у завданні не сказано. Прийнято, що він залишається в пункті вивантаження і рухається порожнім, коли його призначають на наступне перевезення. Холостий хід є частиною часу обслуговування деталі роботом, а у варіанті 1 він збігається з поверненням робота своїм шляхом;

2) розташування пунктів у завданні не задано. Прийнято, що вони стоять на одній лінії A – M1 – M2 – W, а час переміщення між несусідніми пунктами дорівнює сумі часів між сусідніми;

3) правило вибору найближчого робота доповнено: при однаковій відстані перевагу має перевезення, ближче до складу, далі – деталь, що чекає довше;

4) у варіанті 2 до верстата одночасно призначається не більше роботів, ніж там вільних місць, плюс один. Без цього обмеження всі три роботи можуть узяти деталі для одного заповненого верстата, і робота системи зупиняється: у пробних прогонах це ставалось уже за перші 320-1172 с;

5) розподіл часу проходження і місткість площадки визначено при надходженні деталі в середньому раз на 60 с, а не раз на 40 с, як у завданні. Якщо деталі надходять раз на 40 с, система не встигає їх обробляти, черга зростає безмежно, і ці характеристики сталих значень не мають.

Перевірка моделі (підрозділ 3.9) показала, що вона правильно реагує на зміну вхідних змінних, а в сценарії з детермінованими часами результат моделювання точно збігається з ручним розрахунком. Отже, модель придатна для дослідження системи. Результати моделювання можна пояснити так.

Пропускна здатність. За умови завдання деталі надходять у середньому 90 разів на годину. Жоден варіант із таким потоком не справляється: у варіанті 1 на склад надходить 80,7 деталі за годину, у варіанті 2 – 76,4. Решта деталей, 9,3 і 13,6 за годину, накопичується на площадці прибуття. Через 70 годин роботи від порожнього стану там перебуває близько 630 деталей у варіанті 1 і 920 у варіанті 2, і черга продовжує зростати.

Найкращий спосіб закріплення роботів – варіант 1, по одному роботу на кожному шляху. Його пропускна здатність на 4,3 деталі за годину, тобто на 5,6%, більша, ніж у варіанті 2, і ця різниця не випадкова (підрозділ 4.3). Варіант 1 також справляється з більшим потоком: до 78 деталей за годину проти 73 у варіанті 2. Коли потік менший і обидва варіанти встигають обробити всі деталі, у варіанті 1 деталь проходить систему дещо швидше, а на площадці прибуття накопичується менше деталей. Лише при потоці до 51 деталі за годину варіанти практично рівноцінні: середній час проходження становить 273-296 с і відрізняється не більше ніж на 6 с, тобто приблизно на 2%.

Чому варіант 2 гірший. Здавалося б, роботи, які можуть виконувати будь-яке перевезення, мають працювати краще. Модель показує протилежне. Вузьким місцем системи є другий верстат М2. Обробка на ньому триває в середньому 100 с, а разом із вивільненням і захопленням деталі місце зайняте 116 с. Отже, навіть без жодних простоїв три місця М2 обробили б не більше $3 \cdot 3600/116 \approx 93$ деталей за годину, тобто майже стільки, скільки надходить. Тому пропускна здатність усієї системи залежить від того, як швидко звільнене місце М2 отримує нову деталь. Складові часу, за який одне місце М2 обробляє одну деталь, і показники роботи роботів наведено в таблиці 5.1. Тут T – час спостереження, N_W – кількість деталей, що надійшли на склад.

Таблиця 5.1 – Складові циклу місця верстата М2 і робота роботів при $\exp(40)$ (середнє за 30 прогонами)

Величина	Варіант 1	Варіант 2
Цикл місця М2 на одну деталь, с		
обробка	100,1	100,0
вивільнення і захоплення роботом	16,0	16,0

Величина	Варіант 1	Варіант 2
блокування: оброблена деталь чекає на робота	12,7	12,1
простій: місце вільне, нову деталь ще не доставлено	5,0	13,2
разом, $3T/N_W$	133,8	141,3
Холостий хід робота на одне перевезення, с		
шлях 1: до пункту прибуття A	6,0	12,6
шлях 2: до верстата M1	7,0	7,1
шлях 3: до верстата M2	5,0	4,2
разом на одну деталь	18,0	23,9
Частка часу, коли біля верстата чекає робот з деталлю		
біля M1	0,37	0,45
біля M2	0,32	0,38

У варіанті 1 робот R2 перевозить деталі лише з M1 на M2 і майже третину часу стоїть біля M2 з наступною деталлю, тому звільнене місце отримує деталь у середньому через 5,0 с. У варіанті 2 правило найближчого робота не враховує, яке перевезення важливіше для системи. По нову деталь до пункту прибуття часто їде робот, що стоїть біля M2 або на складі, тому холостий хід до A зростає вдвічі, з 6,0 до 12,6 с. Роботи також частіше стоять із деталями біля заповненого M1 (45% часу проти 37%), хоча M1 не є вузьким місцем. Унаслідок цього місце M2 на кожну деталь простоює 13,2 с замість 5,0 с, час обробки однієї деталі одним місцем M2 зростає зі 133,8 до 141,3 с, і пропускна здатність зменшується з 80,7 до 76,4 деталі за годину. Вищий коефіцієнт використання роботів у варіанті 2 (0,91 проти 0,86) не є перевагою: роботи більше часу витрачають на холості переїзди.

Завантаження обладнання. У варіанті 1 роботи R1 і R2 зайняті майже весь час, але третину часу вони не перевозять деталі, а тримають їх біля заповнених верстатів. Перший верстат заповнений постійно, хоча обробка займає лише 45% часу, бо оброблені деталі чекають на робота R2, який стоїть біля заповненого M2. Отже, нестача місць на M2 зупиняє і перший верстат, і роботи R1, R2, тому додатковий робот без нового місця на M2 майже не допомагає (підрозділ 5.2).

Час проходження при потоці 60 деталей за годину в середньому становить 5,5 хв у варіанті 1 і 5,6 хв у варіанті 2. Половина деталей проходить систему швидше ніж за 4,7-4,9 хв, 90% – швидше ніж за 8,8 і 9,5 хв, а окремі деталі

затримуються до 25-32 хв, коли обробка на М2 виявляється довгою. Розподіл описується гамма-законом зі зсувом близько 2 хв, що відповідає найменшому часу, за який деталь проходить усі операції без очікування.

Місткість площадки прибуття. При потоці 60 деталей за годину за 400 годин роботи на площадці одночасно буває в середньому до 18 деталей у варіанті 1 і до 21 у варіанті 2, а 99% часу – не більше 8 і 10 деталей. Найбільша кількість зростає з тривалістю роботи, тому площадку доцільно розраховувати із запасом: у жодному з 30 прогонів вона не перевищила 29 і 38 деталей відповідно. За умови завдання, 90 деталей за годину, жодна місткість площадки не достатня, бо черга зростає безмежно.

5.2 Пропозиції щодо поліпшення функціонування системи

Оскільки вузьким місцем є верстат М2, розглянуто три пропозиції:

1) додати четверте місце обробки на верстаті М2. Тоді без простоїв М2 міг би обробити до $4 \cdot 3600/116 \approx 124$ деталей за годину;

2) додати четвертий робот. У варіанті 1 він закріплюється за одним із шляхів, і перевірено всі три шляхи. У варіанті 2 він входить у загальну групу роботів;

3) поєднати обидві зміни. У варіанті 1 четвертий робот закріплено за шляхом М1 → М2, бо окремо саме він дає найбільший приріст.

Кожну пропозицію перевірено пробними запусками моделі за умови завдання $\exp(40)$, 20 прогонів по 250 000 с зі збором статистики після 50 000 с (серія 6 таблиці 4.1). Зерна генераторів такі самі, як для базової системи, тому відмінності пояснюються саме зміною системи. Результати наведено в таблиці 5.2 і на рисунку 5.1.

Таблиця 5.2 – Результати пробних запусків пропозицій, $\exp(40)$, 20 прогонів

Пропозиція	Варіант	Λ, дет./год	Режим	$\bar{\tau}$, с	L_{max}, шт.	ρ_{M2}	$\bar{\rho}_{\text{R}}$
базова система	1	80,7 ± 0,3	нестійкий	15 858*	653,6*	0,963	0,858
базова система	2	76,4 ± 0,3	нестійкий	22 869*	946,1*	0,907	0,912
4 місця на М2	1	90,1 ± 0,4	стійкий	489	32,9	0,833	0,796

Пропозиція	Варіант	Λ, дет./год	Режим	$\bar{\tau}$, с	L_{max}, шт.	ρ_{M2}	$\bar{\rho}_R$
4 місця на M2	2	90,1 ± 0,4	стійкий	1242	70,8	0,804	0,905
4-й робот на маршруті 1 (A → M1)	1	80,7 ± 0,3	нестійкий	15 855*	653,1*	0,963	0,689
4-й робот на маршруті 2 (M1 → M2)	1	82,8 ± 0,3	нестійкий	12 464*	511,8*	0,993	0,687
4-й робот на маршруті 3 (M2 → W)	1	80,7 ± 0,3	нестійкий	15 855*	653,5*	0,908	0,644
4-й робот	2	83,6 ± 0,3	нестійкий	11 232*	460,4*	0,940	0,744
4 місця на M2 і 4-й робот на маршруті 2	1	90,1 ± 0,5	стійкий	415	27,3	0,837	0,603
4 місця на M2 і 4-й робот	2	90,1 ± 0,5	стійкий	386	26,5	0,755	0,637

Значення, позначені «*», отримано в нестійкому режимі. Вони зростають зі збільшенням часу моделювання і наведені лише для порівняння пропозицій між собою.

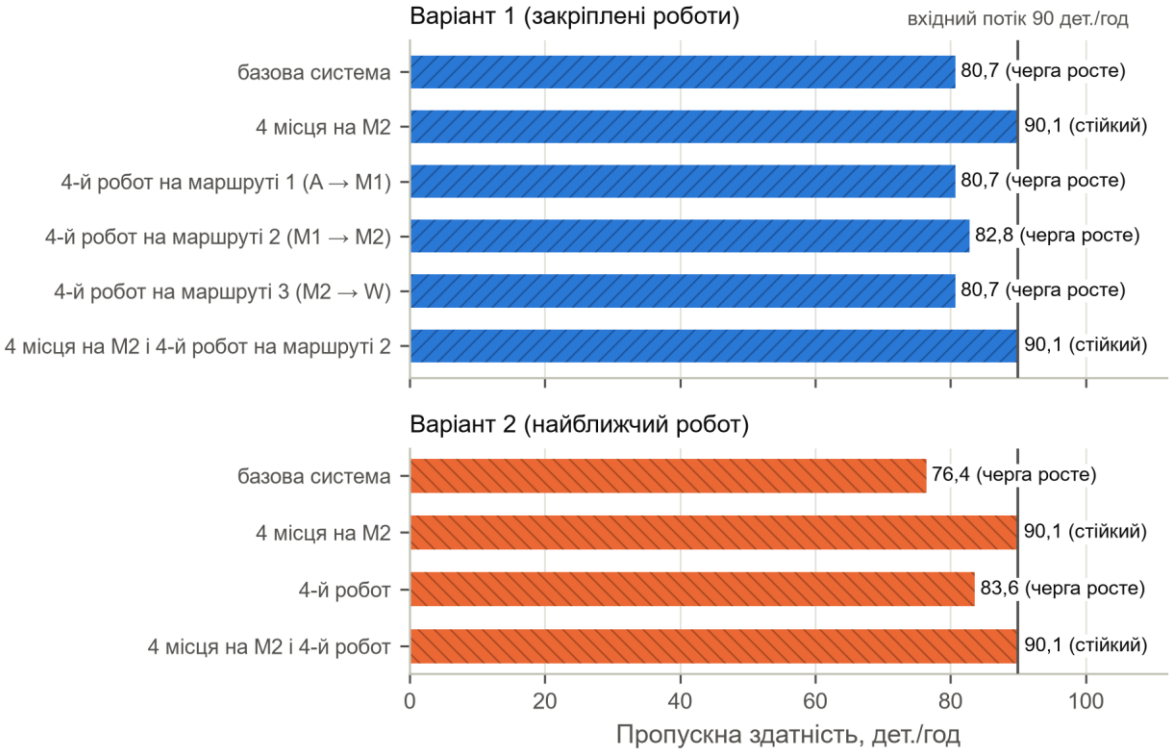


Рисунок 5.1 – Пропускна здатність ГВС для базової системи і пропозицій щодо її поліпшення при надходженні exp(40)

Четверте місце на M2 – єдина окрема зміна, після якої система справляється з потоком 90 деталей за годину в обох варіантах. Пропускна здатність зростає до 90,1 деталі за годину, тобто на 9,4 (11,6%) у варіанті 1 і на 13,6 (17,8%) у варіанті 2. Черга на площадці перестає накопичуватись: у варіанті

1 середній час проходження становить 489 с, а найбільша кількість деталей на площадці за 55 годин – 33. Варіант 1 і тут кращий за варіант 2, у якому час проходження 1242 с, а на площадці буває до 71 деталі. Коефіцієнт використання M2 зменшується з 0,96 до 0,83, тобто з'являється запас продуктивності. Погіршується лише одна характеристика: у варіанті 1 частка часу, коли оброблена деталь на M2 чекає на робота, зростає з 0,095 до 0,106, бо через M2 проходить більше деталей, а вивозить їх той самий робот R3. Витрати на цю пропозицію – доопрацювання верстата M2 або заміна його верстатом із чотирма місцями обробки.

Четвертий робот без зміни M2 проблему не розв'язує. У варіанті 1 робот на шляху A → M1 або M2 → W пропускної здатності не змінює, бо ці шляхи не обмежують роботу системи. Робот на шляху M1 → M2 швидше підвозить деталі до M2 і збільшує пропускну здатність лише до 82,8 деталі за годину. У варіанті 2 четвертий робот збільшує її до 83,6 деталі за годину. В усіх випадках черга продовжує зростати, а середній коефіцієнт використання роботів зменшується до 0,64-0,74, тобто роботи частину часу простоюють.

Поєднання обох змін дає найменший час проходження: 415 с у варіанті 1 і 386 с у варіанті 2, а найбільша кількість деталей на площадці – близько 27. Порівняно з самим четвертим місцем на M2 час проходження зменшується на 15% у варіанті 1 і на 69% у варіанті 2. За наявності четвертого робота варіант 2 стає трохи кращим за варіант 1: коли роботів вистачає, гнучкість призначення переважає втрати на холості переїзди. Водночас коефіцієнт використання роботів зменшується до 0,60-0,64, тобто четвертий робот використовується неповністю.

Отже, доцільно додати четверте місце обробки на верстаті M2 і зберегти закріплення роботів за шляхами (варіант 1). Це найменша зміна, після якої система обробляє всі деталі за умови завдання. Купувати четвертий робот доцільно, лише якщо потрібно додатково скоротити час проходження деталей. Тоді варто перейти до призначення найближчого робота (варіант 2), що зменшує середній час проходження з 489 до 386 с.

5.3 Пропозиції щодо подальшого дослідження

Модель можна використати для таких подальших досліджень:

– правила призначення роботів, які враховують вузьке місце системи, наприклад перевагу перевезенням на М2 або змішане закріплення, коли один робот закріплений за шляхом М1 → М2, а решта виконують будь-які перевезення. Для цього в моделі достатньо змінити множини дозволених маршрутів роботів і ключ вибору в диспетчері;

– межа стійкості і запас продуктивності системи з чотирма місцями на М2, а також найвигідніше поєднання кількості місць на верстатах і кількості роботів з урахуванням їх вартості;

– вплив реального розташування обладнання, відмов верстатів і роботів, а також законів розподілу часу обробки на М2, відмінних від експоненціального.

5.4 Висновки до розділу

Найкращим з погляду пропускної здатності є закріплення по одному роботу за кожним шляхом переміщення деталей (варіант 1): його пропускна здатність на 5,6% більша, а межа стійкості вища, ніж у варіанті 2. Правило призначення найближчого робота програє через довші холості переїзди і несвоєчасну доставку деталей на верстат М2, який є вузьким місцем системи. За умови завдання обидва варіанти перевантажені. Пробні запуски показали, що четверте місце обробки на М2 робить систему стійкою при потоці 90 деталей за годину, а четвертий робот без цієї зміни недостатній.

ВИСНОВКИ

У курсовій роботі розроблено імітаційну модель експериментальної роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування і досліджено два способи закріплення роботів до операцій переміщення деталей.

Формалізовані моделі обох варіантів побудовано як розімкнені мережі масового обслуговування. Верстати представлено триканальними СМО без черги, роботів – трьома одноканальними СМО у варіанті 1 і однією триканальною СМО з вибором найближчого пристрою у варіанті 2. Умови «на верстаті немає місця» і «немає вільного робота» описано предикатами блокування маршрутів. Рух робота порожнім враховано як складову часу обслуговування деталі роботом. Алгоритм імітації власний: просування часу до найближчої події, елементи мережі описано класами мовою Python. Статистичну обробку результатів експериментів виконано з використанням бібліотек NumPy і SciPy, рисунки побудовано бібліотекою Matplotlib. Повний код моделі та експериментального дослідження розміщено в репозиторії GitHub: https://github.com/tannnnnny/course_work_csdes.git. Обидва варіанти реалізовано тим самим алгоритмом, вони відрізняються лише дозволеними маршрутами роботів. Коректність алгоритму підтверджено верифікацією в дев’яти сценаріях, точним збігом із ручним розрахунком у детермінованому сценарії, автоматичними тестами і протоколом подій.

Під час розробки моделі прийнято такі припущення: пункти розташовані на одній лінії, робот після вивільнення деталі залишається в пункті вивантаження, час обробки на першому верстаті має усічений нормальний закон, одночасні події виконуються у випадковому порядку. Для варіанта 2 введено обмеження кількості роботів, які одночасно їдуть до верстата, без якого система переходила у стан взаємного блокування роботів. Розподіл часу проходження і місткість площадки визначено при надходженні $\exp(60)$, бо за умови завдання $\exp(40)$ система перевантажена, і ці характеристики сталих значень не мають. Усі інші умови завдання відтворено в моделі без спрощень.

Результати моделювання такі:

1) найкращим з погляду пропускної здатності є варіант 1, по одному роботу на кожному шляху: 80,7 дет./год проти 76,4 дет./год у варіанті 2 при надходженні exp(40); вплив способу закріплення значущий за критерієм Фішера;

2) варіант 1 залишається стійким до 78,3 дет./год, варіант 2 – до 73,5 дет./год; за умови завдання, 90 дет./год, перевантажені обидва варіанти, і черга на площадці прибуття зростає безмежно;

3) вузьким місцем системи є верстат М2. Варіант 2 програє через довші холості переїзди роботів і несвоєчасну доставку деталей на М2;

4) коефіцієнти використання при exp(40): у варіанті 1 роботів – 1,00, 0,99 і 0,58, верстатів М1 і М2 – 1,00 і 0,96; у варіанті 2 роботів – по 0,91, верстатів – 0,92 і 0,91;

5) час проходження деталей в обох варіантах розподілений за гамма-законом зі зсувом; при exp(60) середнє значення становить 328 с у варіанті 1 і 338 с у варіанті 2;

6) максимальна кількість деталей на площадці прибуття при exp(60) за 400 годин роботи становить у середньому 18 у варіанті 1 і 21 у варіанті 2; за умови завдання жодна скінченна місткість не достатня.

Запропоновано додати четверте місце обробки на верстаті М2 і зберегти закріплення роботів за шляхами. Пробні запуски показали, що після цієї зміни система обробляє всі 90 деталей за годину, середній час проходження становить 489 с. Четвертий робот без зміни М2 проблему не розв’язує. Отже, імітаційне моделювання системи виконано успішно: мету роботи досягнуто, на всі питання завдання отримано відповіді із заданою точністю.

Формалізм мережі масового обслуговування добре відповідає системі, у якій деталі послідовно проходять обробку і транспортування. Найскладнішими в розробці виявилися блокування верстатів, коли робот тримає деталь біля заповненого верстата, і правило вибору найближчого робота у варіанті 2. Їх

довелося описувати додатковими предикатами і правилом диспетчеризації, які в стандартних позначеннях мережі МО відсутні. Перевагою імітаційного методу є те, що він враховує ці особливості без спрощень, недоступних аналітичним методам, і дає змогу перевірити пропозиції без змін у реальній системі. Перевагою розробленого алгоритму є те, що обидва варіанти і всі пропозиції реалізовано однією програмою зі зміною лише параметрів, а спільні випадкові числа зменшують вплив випадковості на порівняння варіантів. Подальші дослідження доцільно спрямувати на правила призначення роботів, що враховують вузьке місце системи, і на пошук найвигіднішого поєднання кількості місць на верстатах і кількості роботів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Томашевський В. М. Моделювання систем. Київ : Видавнича група BHV, 2005. 352 с.
   URL: https://pdf.lib.vntu.edu.ua/books/2016/Tomashev_2005_352.pdf (дата звернення: 28.10.2026).
2. Стеценко І. В. Моделювання систем : навч. посіб. Черкаси : ЧДТУ, 2010. 399 с.
3. Стеценко І. В., Дифучина О. Ю., Дифучин А. Ю. Моделювання систем. Курсова робота : рекомендації до виконання курсової роботи : навч. посіб. Київ : КПІ ім. Ігоря Сікорського, 2024. 109 с .
4. Python 3 Documentation. Python Software Foundation. URL: https://docs.python.org/3/ (дата звернення: 28.10.2026).
5. Business Process Model and Notation (BPMN). Version 2.0.2. Object Management Group, 2013. URL: https://www.omg.org/spec/BPMN/2.0.2/(дата звернення: 28.10.2026).
6. NumPy Documentation. URL: https://numpy.org/doc/stable/ (дата звернення: 28.10.2026).
7. SciPy Documentation. URL: https://docs.scipy.org/doc/scipy/(дата звернення: 28.10.2026).
8. Matplotlib: Visualization with Python. URL: https://matplotlib.org/stable/ (дата звернення: 28.10.2026).
9. Unified Modeling Language (UML). Version 2.5.1. Object Management Group, 2017. URL: https://www.omg.org/spec/UML/2.5.1/(дата звернення: 28.10.2026).
10. Tanya Ponzel. course_work_csdes. GitHub.
    URL: https://github.com/tannnnnny/course_work_csdes.git (дата звернення: 28.10.2026).

ДОДАТОК А. ПРОТОКОЛИ ПОДІЙ

Протоколи отримано командою `uv run fms trace --until 600` при значеннях параметрів і вхідних змінних з таблиць 1.1 і 1.2 та початковому значенні генераторів 1. Наведено події до моменту 300 с. Кожен рядок містить момент події в секундах, елемент мережі і зміст події. Після кожної групи одночасних подій виведено стан мережі: кількість деталей на площадці A, стан кожного місця верстатів M1 і M2 і фазу кожного робота з точкою, де він перебуває. Стани місць верстата: «.» – вільне, «l» – вивільнення деталі роботом, «B» – обробка, «x» – блоковане (оброблена деталь чекає на робота), «u» – захоплення. Фази роботів: free – вільний, empty_move – холостий хід, grab – захоплення, loaded_move – переміщення з деталлю, wait_slot – очікування вільного місця, release – вивільнення.

з нової сторінки

А.1 Протокол подій для варіанта 1

```
       стан      A=0 | M1[...] | M2[...] | R1:free@A R2:free@M1 R3:free@M2
  1.248 Create    надходження деталі #1
  1.248 Robot1    призначено на маршрут 1 (A->M1) для деталі #1, холостий хід A->A 0
  1.248 Robot1    початок захоплення деталі #1 у A
       стан      A=1 | M1[...] | M2[...] | R1:grab@A R2:free@M1 R3:free@M2
  8.788 Robot1    деталь #1 захоплено, хід A->M1
       стан      A=0 | M1[...] | M2[...] | R1:loaded_move@A R2:free@M1 R3:free@M2
 14.788 Robot1    початок вивільнення деталі #1 у M1
       стан      A=0 | M1[l..] | M2[...] | R1:release@M1 R2:free@M1 R3:free@M2
 16.321 Create    надходження деталі #2
       стан      A=1 | M1[l..] | M2[...] | R1:release@M1 R2:free@M1 R3:free@M2
 23.597 M1        початок обробки деталі #1 (місце 0), кінець о 78.547
 23.597 Robot1    вивільнив деталь #1, вільний у M1
 23.597 Robot1    призначено на маршрут 1 (A->M1) для деталі #2, холостий хід M1->A 6
       стан      A=1 | M1[B..] | M2[...] | R1:empty_move@M1 R2:free@M1 R3:free@M2
 29.597 Robot1    початок захоплення деталі #2 у A
       стан      A=1 | M1[B..] | M2[...] | R1:grab@A R2:free@M1 R3:free@M2
 37.877 Robot1    деталь #2 захоплено, хід A->M1
       стан      A=0 | M1[B..] | M2[...] | R1:loaded_move@A R2:free@M1 R3:free@M2
 43.877 Robot1    початок вивільнення деталі #2 у M1
       стан      A=0 | M1[Bl.] | M2[...] | R1:release@M1 R2:free@M1 R3:free@M2
 52.057 M1        початок обробки деталі #2 (місце 1), кінець о 98.115
 52.057 Robot1    вивільнив деталь #2, вільний у M1
       стан      A=0 | M1[BB.] | M2[...] | R1:free@M1 R2:free@M1 R3:free@M2
 78.547 M1        кінець обробки деталі #1 (місце 0) -> блокування
 78.547 Robot2    призначено на маршрут 2 (M1->M2) для деталі #1, холостий хід M1->M1 0
 78.547 Robot2    початок захоплення деталі #1 у M1
       стан      A=0 | M1[uB.] | M2[...] | R1:free@M1 R2:grab@M1 R3:free@M2
 87.494 Robot2    деталь #1 захоплено, хід M1->M2
       стан      A=0 | M1[.B.] | M2[...] | R1:free@M1 R2:loaded_move@M1 R3:free@M2
 94.494 Robot2    початок вивільнення деталі #1 у M2
       стан      A=0 | M1[.B.] | M2[l..] | R1:free@M1 R2:release@M2 R3:free@M2
 97.633 Create    надходження деталі #3
 97.633 Robot1    призначено на маршрут 1 (A->M1) для деталі #3, холостий хід M1->A 6
       стан      A=1 | M1[.B.] | M2[l..] | R1:empty_move@M1 R2:release@M2 R3:free@M2
 98.115 M1        кінець обробки деталі #2 (місце 1) -> блокування
       стан      A=1 | M1[.x.] | M2[l..] | R1:empty_move@M1 R2:release@M2 R3:free@M2
102.497 M2        початок обробки деталі #1 (місце 0), кінець о 112.780
102.497 Robot2    вивільнив деталь #1, вільний у M2
```

```
 102.497 Robot2    призначено на маршрут 2 (M1->M2) для деталі #2, холостий хід M2->M1 7
         стан      A=1 | M1[.x.] | M2[B..] | R1:empty_move@M1 R2:empty_move@M2 R3:free@M2
 103.633 Robot1    початок захоплення деталі #3 у A
         стан      A=1 | M1[.x.] | M2[B..] | R1:grab@A R2:empty_move@M2 R3:free@M2
 109.497 Robot2    початок захоплення деталі #2 у M1
         стан      A=1 | M1[.u.] | M2[B..] | R1:grab@A R2:grab@M1 R3:free@M2
 112.150 Robot1    деталь #3 захоплено, хід A->M1
         стан      A=0 | M1[.u.] | M2[B..] | R1:loaded_move@A R2:grab@M1 R3:free@M2
 112.780 M2        кінець обробки деталі #1 (місце 0) -> блокування
 112.780 Robot3    призначено на маршрут 3 (M2->W) для деталі #1, холостий хід M2->M2 0
 112.780 Robot3    початок захоплення деталі #1 у M2
         стан      A=0 | M1[.u.] | M2[u..] | R1:loaded_move@A R2:grab@M1 R3:grab@M2
 117.172 Robot2    деталь #2 захоплено, хід M1->M2
         стан      A=0 | M1[...] | M2[u..] | R1:loaded_move@A R2:loaded_move@M1
R3:grab@M2
 118.150 Robot1    початок вивільнення деталі #3 у M1
         стан      A=0 | M1[l..] | M2[u..] | R1:release@M1 R2:loaded_move@M1 R3:grab@M2
 120.099 Robot3    деталь #1 захоплено, хід M2->W
         стан      A=0 | M1[l..] | M2[...] | R1:release@M1 R2:loaded_move@M1
R3:loaded_move@M2
 124.172 Robot2    початок вивільнення деталі #2 у M2
         стан      A=0 | M1[l..] | M2[l..] | R1:release@M1 R2:release@M2
R3:loaded_move@M2
 125.099 Robot3    початок вивільнення деталі #1 у W
         стан      A=0 | M1[l..] | M2[l..] | R1:release@M1 R2:release@M2 R3:release@W
 125.614 M1        початок обробки деталі #3 (місце 0), кінець о 190.789
 125.614 Robot1    вивільнив деталь #3, вільний у M1
         стан      A=0 | M1[B..] | M2[l..] | R1:free@M1 R2:release@M2 R3:release@W
 131.245 M2        початок обробки деталі #2 (місце 0), кінець о 161.581
 131.245 Robot2    вивільнив деталь #2, вільний у M2
         стан      A=0 | M1[B..] | M2[B..] | R1:free@M1 R2:free@M2 R3:release@W
 133.741 Warehouse деталь #1 на складі, час проходження 132.493
 133.741 Robot3    вивільнив деталь #1, вільний у W
         стан      A=0 | M1[B..] | M2[B..] | R1:free@M1 R2:free@M2 R3:free@W
 161.581 M2        кінець обробки деталі #2 (місце 0) -> блокування
 161.581 Robot3    призначено на маршрут 3 (M2->W) для деталі #2, холостий хід W->M2 5
         стан      A=0 | M1[B..] | M2[x..] | R1:free@M1 R2:free@M2 R3:empty_move@W
 166.581 Robot3    початок захоплення деталі #2 у M2
         стан      A=0 | M1[B..] | M2[u..] | R1:free@M1 R2:free@M2 R3:grab@M2
 169.390 Create    надходження деталі #4
 169.390 Robot1    призначено на маршрут 1 (A->M1) для деталі #4, холостий хід M1->A 6
         стан      A=1 | M1[B..] | M2[u..] | R1:empty_move@M1 R2:free@M2 R3:grab@M2
 174.294 Robot3    деталь #2 захоплено, хід M2->W
         стан      A=1 | M1[B..] | M2[...] | R1:empty_move@M1 R2:free@M2
R3:loaded_move@M2
 175.390 Robot1    початок захоплення деталі #4 у A
         стан      A=1 | M1[B..] | M2[...] | R1:grab@A R2:free@M2 R3:loaded_move@M2
 179.294 Robot3    початок вивільнення деталі #2 у W
         стан      A=1 | M1[B..] | M2[...] | R1:grab@A R2:free@M2 R3:release@W
 182.873 Robot1    деталь #4 захоплено, хід A->M1
         стан      A=0 | M1[B..] | M2[...] | R1:loaded_move@A R2:free@M2 R3:release@W
 188.205 Warehouse деталь #2 на складі, час проходження 171.884
 188.205 Robot3    вивільнив деталь #2, вільний у W
         стан      A=0 | M1[B..] | M2[...] | R1:loaded_move@A R2:free@M2 R3:free@W
 188.873 Robot1    початок вивільнення деталі #4 у M1
         стан      A=0 | M1[Bl.] | M2[...] | R1:release@M1 R2:free@M2 R3:free@W
 190.789 M1        кінець обробки деталі #3 (місце 0) -> блокування
 190.789 Robot2    призначено на маршрут 2 (M1->M2) для деталі #3, холостий хід M2->M1 7
         стан      A=0 | M1[xl.] | M2[...] | R1:release@M1 R2:empty_move@M2 R3:free@W
 195.939 Create    надходження деталі #5
         стан      A=1 | M1[xl.] | M2[...] | R1:release@M1 R2:empty_move@M2 R3:free@W
 196.024 M1        початок обробки деталі #4 (місце 1), кінець о 243.916
 196.024 Robot1    вивільнив деталь #4, вільний у M1
 196.024 Robot1    призначено на маршрут 1 (A->M1) для деталі #5, холостий хід M1->A 6
         стан      A=1 | M1[xB.] | M2[...] | R1:empty_move@M1 R2:empty_move@M2 R3:free@W
 197.789 Robot2    початок захоплення деталі #3 у M1
         стан      A=1 | M1[uB.] | M2[...] | R1:empty_move@M1 R2:grab@M1 R3:free@W
 202.024 Robot1    початок захоплення деталі #5 у A
         стан      A=1 | M1[uB.] | M2[...] | R1:grab@A R2:grab@M1 R3:free@W
 205.128 Robot2    деталь #3 захоплено, хід M1->M2
```

```
         стан      A=1 | M1[.B.] | M2[...] | R1:grab@A R2:loaded_move@M1 R3:free@W
 210.675 Robot1    деталь #5 захоплено, хід A->M1
         стан      A=0 | M1[.B.] | M2[...] | R1:loaded_move@A R2:loaded_move@M1 R3:free@W
 212.128 Robot2    початок вивільнення деталі #3 у M2
         стан      A=0 | M1[.B.] | M2[l..] | R1:loaded_move@A R2:release@M2 R3:free@W
 216.675 Robot1    початок вивільнення деталі #5 у M1
         стан      A=0 | M1[lB.] | M2[l..] | R1:release@M1 R2:release@M2 R3:free@W
 219.433 M2        початок обробки деталі #3 (місце 0), кінець о 232.428
 219.433 Robot2    вивільнив деталь #3, вільний у M2
         стан      A=0 | M1[lB.] | M2[B..] | R1:release@M1 R2:free@M2 R3:free@W
 225.339 M1        початок обробки деталі #5 (місце 0), кінець о 291.635
 225.339 Robot1    вивільнив деталь #5, вільний у M1
         стан      A=0 | M1[BB.] | M2[B..] | R1:free@M1 R2:free@M2 R3:free@W
 230.992 Create    надходження деталі #6
 230.992 Robot1    призначено на маршрут 1 (A->M1) для деталі #6, холостий хід M1->A 6
         стан      A=1 | M1[BB.] | M2[B..] | R1:empty_move@M1 R2:free@M2 R3:free@W
 232.428 M2        кінець обробки деталі #3 (місце 0) -> блокування
 232.428 Robot3    призначено на маршрут 3 (M2->W) для деталі #3, холостий хід W->M2 5
         стан      A=1 | M1[BB.] | M2[x..] | R1:empty_move@M1 R2:free@M2 R3:empty_move@W
 236.992 Robot1    початок захоплення деталі #6 у A
         стан      A=1 | M1[BB.] | M2[x..] | R1:grab@A R2:free@M2 R3:empty_move@W
 237.428 Robot3    початок захоплення деталі #3 у M2
         стан      A=1 | M1[BB.] | M2[u..] | R1:grab@A R2:free@M2 R3:grab@M2
 243.916 M1        кінець обробки деталі #4 (місце 1) -> блокування
 243.916 Robot2    призначено на маршрут 2 (M1->M2) для деталі #4, холостий хід M2->M1 7
         стан      A=1 | M1[Bx.] | M2[u..] | R1:grab@A R2:empty_move@M2 R3:grab@M2
 245.001 Robot3    деталь #3 захоплено, хід M2->W
         стан      A=1 | M1[Bx.] | M2[...] | R1:grab@A R2:empty_move@M2 R3:loaded_move@M2
 245.178 Create    надходження деталі #7
         стан      A=2 | M1[Bx.] | M2[...] | R1:grab@A R2:empty_move@M2 R3:loaded_move@M2
 245.611 Robot1    деталь #6 захоплено, хід A->M1
         стан      A=1 | M1[Bx.] | M2[...] | R1:loaded_move@A R2:empty_move@M2
R3:loaded_move@M2
 250.001 Robot3    початок вивільнення деталі #3 у W
         стан      A=1 | M1[Bx.] | M2[...] | R1:loaded_move@A R2:empty_move@M2
R3:release@W
 250.916 Robot2    початок захоплення деталі #4 у M1
         стан      A=1 | M1[Bu.] | M2[...] | R1:loaded_move@A R2:grab@M1 R3:release@W
 251.611 Robot1    початок вивільнення деталі #6 у M1
         стан      A=1 | M1[Bul] | M2[...] | R1:release@M1 R2:grab@M1 R3:release@W
 258.026 Warehouse деталь #3 на складі, час проходження 160.393
 258.026 Robot3    вивільнив деталь #3, вільний у W
         стан      A=1 | M1[Bul] | M2[...] | R1:release@M1 R2:grab@M1 R3:free@W
 259.184 Robot2    деталь #4 захоплено, хід M1->M2
         стан      A=1 | M1[B.l] | M2[...] | R1:release@M1 R2:loaded_move@M1 R3:free@W
 260.020 M1        початок обробки деталі #6 (місце 2), кінець о 326.782
 260.020 Robot1    вивільнив деталь #6, вільний у M1
 260.020 Robot1    призначено на маршрут 1 (A->M1) для деталі #7, холостий хід M1->A 6
         стан      A=1 | M1[B.B] | M2[...] | R1:empty_move@M1 R2:loaded_move@M1 R3:free@W
 266.020 Robot1    початок захоплення деталі #7 у A
         стан      A=1 | M1[B.B] | M2[...] | R1:grab@A R2:loaded_move@M1 R3:free@W
 266.184 Robot2    початок вивільнення деталі #4 у M2
         стан      A=1 | M1[B.B] | M2[l..] | R1:grab@A R2:release@M2 R3:free@W
 272.731 Create    надходження деталі #8
         стан      A=2 | M1[B.B] | M2[l..] | R1:grab@A R2:release@M2 R3:free@W
 273.437 Robot1    деталь #7 захоплено, хід A->M1
         стан      A=1 | M1[B.B] | M2[l..] | R1:loaded_move@A R2:release@M2 R3:free@W
 273.920 M2        початок обробки деталі #4 (місце 0), кінець о 307.648
 273.920 Robot2    вивільнив деталь #4, вільний у M2
         стан      A=1 | M1[B.B] | M2[B..] | R1:loaded_move@A R2:free@M2 R3:free@W
 279.437 Robot1    початок вивільнення деталі #7 у M1
         стан      A=1 | M1[BlB] | M2[B..] | R1:release@M1 R2:free@M2 R3:free@W
 287.831 M1        початок обробки деталі #7 (місце 1), кінець о 340.931
 287.831 Robot1    вивільнив деталь #7, вільний у M1
 287.831 Robot1    призначено на маршрут 1 (A->M1) для деталі #8, холостий хід M1->A 6
         стан      A=1 | M1[BBB] | M2[B..] | R1:empty_move@M1 R2:free@M2 R3:free@W
 291.635 M1        кінець обробки деталі #5 (місце 0) -> блокування
 291.635 Robot2    призначено на маршрут 2 (M1->M2) для деталі #5, холостий хід M2->M1 7
         стан      A=1 | M1[xBB] | M2[B..] | R1:empty_move@M1 R2:empty_move@M2 R3:free@W
 293.831 Robot1    початок захоплення деталі #8 у A
```

```
         стан      A=1 | M1[xBB] | M2[B..] | R1:grab@A R2:empty_move@M2 R3:free@W
 298.635 Robot2    початок захоплення деталі #5 у M1
         стан      A=1 | M1[uBB] | M2[B..] | R1:grab@A R2:grab@M1 R3:free@W
```

A.2 Протокол подій для варіанта 2

```
         стан      A=0 | M1[...] | M2[...] | R1:free@A R2:free@M1 R3:free@M2
   1.248 Create    надходження деталі #1
   1.248 Robot1    призначено на маршрут 1 (A->M1) для деталі #1, холостий хід A->A 0
   1.248 Robot1    початок захоплення деталі #1 у A
         стан      A=1 | M1[...] | M2[...] | R1:grab@A R2:free@M1 R3:free@M2
   8.788 Robot1    деталь #1 захоплено, хід A->M1
         стан      A=0 | M1[...] | M2[...] | R1:loaded_move@A R2:free@M1 R3:free@M2
  14.788 Robot1    початок вивільнення деталі #1 у M1
         стан      A=0 | M1[l..] | M2[...] | R1:release@M1 R2:free@M1 R3:free@M2
  16.321 Create    надходження деталі #2
  16.321 Robot2    призначено на маршрут 1 (A->M1) для деталі #2, холостий хід M1->A 6
         стан      A=1 | M1[l..] | M2[...] | R1:release@M1 R2:empty_move@M1 R3:free@M2
  22.321 Robot2    початок захоплення деталі #2 у A
         стан      A=1 | M1[l..] | M2[...] | R1:release@M1 R2:grab@A R3:free@M2
  23.597 M1        початок обробки деталі #1 (місце 0), кінець о 78.547
  23.597 Robot1    вивільнив деталь #1, вільний у M1
         стан      A=1 | M1[B..] | M2[...] | R1:free@M1 R2:grab@A R3:free@M2
  30.601 Robot2    деталь #2 захоплено, хід A->M1
         стан      A=0 | M1[B..] | M2[...] | R1:free@M1 R2:loaded_move@A R3:free@M2
  36.601 Robot2    початок вивільнення деталі #2 у M1
         стан      A=0 | M1[Bl.] | M2[...] | R1:free@M1 R2:release@M1 R3:free@M2
  44.781 M1        початок обробки деталі #2 (місце 1), кінець о 90.839
  44.781 Robot2    вивільнив деталь #2, вільний у M1
         стан      A=0 | M1[BB.] | M2[...] | R1:free@M1 R2:free@M1 R3:free@M2
  78.547 M1        кінець обробки деталі #1 (місце 0) -> блокування
  78.547 Robot2    призначено на маршрут 2 (M1->M2) для деталі #1, холостий хід M1->M1 0
  78.547 Robot2    початок захоплення деталі #1 у M1
         стан      A=0 | M1[uB.] | M2[...] | R1:free@M1 R2:grab@M1 R3:free@M2
  87.494 Robot2    деталь #1 захоплено, хід M1->M2
         стан      A=0 | M1[.B.] | M2[...] | R1:free@M1 R2:loaded_move@M1 R3:free@M2
  90.839 M1        кінець обробки деталі #2 (місце 1) -> блокування
  90.839 Robot1    призначено на маршрут 2 (M1->M2) для деталі #2, холостий хід M1->M1 0
  90.839 Robot1    початок захоплення деталі #2 у M1
         стан      A=0 | M1[.u.] | M2[...] | R1:grab@M1 R2:loaded_move@M1 R3:free@M2
  94.494 Robot2    початок вивільнення деталі #1 у M2
         стан      A=0 | M1[.u.] | M2[l..] | R1:grab@M1 R2:release@M2 R3:free@M2
  97.633 Create    надходження деталі #3
  97.633 Robot3    призначено на маршрут 1 (A->M1) для деталі #3, холостий хід M2->A 13
         стан      A=1 | M1[.u.] | M2[l..] | R1:grab@M1 R2:release@M2 R3:empty_move@M2
  98.842 Robot1    деталь #2 захоплено, хід M1->M2
         стан      A=1 | M1[...] | M2[l..] | R1:loaded_move@M1 R2:release@M2
R3:empty_move@M2
 103.011 M2        початок обробки деталі #1 (місце 0), кінець о 113.294
 103.011 Robot2    вивільнив деталь #1, вільний у M2
         стан      A=1 | M1[...] | M2[B..] | R1:loaded_move@M1 R2:free@M2
R3:empty_move@M2
 105.842 Robot1    початок вивільнення деталі #2 у M2
         стан      A=1 | M1[...] | M2[Bl.] | R1:release@M2 R2:free@M2 R3:empty_move@M2
 110.633 Robot3    початок захоплення деталі #3 у A
         стан      A=1 | M1[...] | M2[Bl.] | R1:release@M2 R2:free@M2 R3:grab@A
 113.294 M2        кінець обробки деталі #1 (місце 0) -> блокування
 113.294 Robot2    призначено на маршрут 3 (M2->W) для деталі #1, холостий хід M2->M2 0
 113.294 Robot2    початок захоплення деталі #1 у M2
         стан      A=1 | M1[...] | M2[ul.] | R1:release@M2 R2:grab@M2 R3:grab@A
 113.517 M2        початок обробки деталі #2 (місце 1), кінець о 143.854
 113.517 Robot1    вивільнив деталь #2, вільний у M2
         стан      A=1 | M1[...] | M2[uB.] | R1:free@M2 R2:grab@M2 R3:grab@A
 117.952 Robot3    деталь #3 захоплено, хід A->M1
         стан      A=0 | M1[...] | M2[uB.] | R1:free@M2 R2:grab@M2 R3:loaded_move@A
 120.758 Robot2    деталь #1 захоплено, хід M2->W
         стан      A=0 | M1[...] | M2[.B.] | R1:free@M2 R2:loaded_move@M2
R3:loaded_move@A
 123.952 Robot3    початок вивільнення деталі #3 у M1
         стан      A=0 | M1[l..] | M2[.B.] | R1:free@M2 R2:loaded_move@M2 R3:release@M1
```

```
 125.758 Robot2    початок вивільнення деталі #1 у W
         стан      A=0 | M1[l..] | M2[.B.] | R1:free@M2 R2:release@W R3:release@M1
 131.025 M1        початок обробки деталі #3 (місце 0), кінець о 196.200
 131.025 Robot3    вивільнив деталь #3, вільний у M1
         стан      A=0 | M1[B..] | M2[.B.] | R1:free@M2 R2:release@W R3:free@M1
 134.400 Warehouse деталь #1 на складі, час проходження 133.152
 134.400 Robot2    вивільнив деталь #1, вільний у W
         стан      A=0 | M1[B..] | M2[.B.] | R1:free@M2 R2:free@W R3:free@M1
 143.854 M2        кінець обробки деталі #2 (місце 1) -> блокування
 143.854 Robot1    призначено на маршрут 3 (M2->W) для деталі #2, холостий хід M2->M2 0
 143.854 Robot1    початок захоплення деталі #2 у M2
         стан      A=0 | M1[B..] | M2[.u.] | R1:grab@M2 R2:free@W R3:free@M1
 151.567 Robot1    деталь #2 захоплено, хід M2->W
         стан      A=0 | M1[B..] | M2[...] | R1:loaded_move@M2 R2:free@W R3:free@M1
 156.567 Robot1    початок вивільнення деталі #2 у W
         стан      A=0 | M1[B..] | M2[...] | R1:release@W R2:free@W R3:free@M1
 164.050 Warehouse деталь #2 на складі, час проходження 147.728
 164.050 Robot1    вивільнив деталь #2, вільний у W
         стан      A=0 | M1[B..] | M2[...] | R1:free@W R2:free@W R3:free@M1
 169.390 Create    надходження деталі #4
 169.390 Robot3    призначено на маршрут 1 (A->M1) для деталі #4, холостий хід M1->A 6
         стан      A=1 | M1[B..] | M2[...] | R1:free@W R2:free@W R3:empty_move@M1
 175.390 Robot3    початок захоплення деталі #4 у A
         стан      A=1 | M1[B..] | M2[...] | R1:free@W R2:free@W R3:grab@A
 184.301 Robot3    деталь #4 захоплено, хід A->M1
         стан      A=0 | M1[B..] | M2[...] | R1:free@W R2:free@W R3:loaded_move@A
 190.301 Robot3    початок вивільнення деталі #4 у M1
         стан      A=0 | M1[Bl.] | M2[...] | R1:free@W R2:free@W R3:release@M1
 195.939 Create    надходження деталі #5
 195.939 Robot1    призначено на маршрут 1 (A->M1) для деталі #5, холостий хід W->A 18
         стан      A=1 | M1[Bl.] | M2[...] | R1:empty_move@W R2:free@W R3:release@M1
 196.200 M1        кінець обробки деталі #3 (місце 0) -> блокування
 196.200 Robot2    призначено на маршрут 2 (M1->M2) для деталі #3, холостий хід W->M1 12
         стан      A=1 | M1[xl.] | M2[...] | R1:empty_move@W R2:empty_move@W
R3:release@M1
 197.452 M1        початок обробки деталі #4 (місце 1), кінець о 245.344
 197.452 Robot3    вивільнив деталь #4, вільний у M1
         стан      A=1 | M1[xB.] | M2[...] | R1:empty_move@W R2:empty_move@W R3:free@M1
 208.200 Robot2    початок захоплення деталі #3 у M1
         стан      A=1 | M1[uB.] | M2[...] | R1:empty_move@W R2:grab@M1 R3:free@M1
 213.939 Robot1    початок захоплення деталі #5 у A
         стан      A=1 | M1[uB.] | M2[...] | R1:grab@A R2:grab@M1 R3:free@M1
 215.539 Robot2    деталь #3 захоплено, хід M1->M2
         стан      A=1 | M1[.B.] | M2[...] | R1:grab@A R2:loaded_move@M1 R3:free@M1
 222.539 Robot2    початок вивільнення деталі #3 у M2
         стан      A=1 | M1[.B.] | M2[l..] | R1:grab@A R2:release@M2 R3:free@M1
 222.590 Robot1    деталь #5 захоплено, хід A->M1
         стан      A=0 | M1[.B.] | M2[l..] | R1:loaded_move@A R2:release@M2 R3:free@M1
 228.590 Robot1    початок вивільнення деталі #5 у M1
         стан      A=0 | M1[lB.] | M2[l..] | R1:release@M1 R2:release@M2 R3:free@M1
 229.844 M2        початок обробки деталі #3 (місце 0), кінець о 242.839
 229.844 Robot2    вивільнив деталь #3, вільний у M2
         стан      A=0 | M1[lB.] | M2[B..] | R1:release@M1 R2:free@M2 R3:free@M1
 230.992 Create    надходження деталі #6
 230.992 Robot3    призначено на маршрут 1 (A->M1) для деталі #6, холостий хід M1->A 6
         стан      A=1 | M1[lB.] | M2[B..] | R1:release@M1 R2:free@M2 R3:empty_move@M1
 236.992 Robot3    початок захоплення деталі #6 у A
         стан      A=1 | M1[lB.] | M2[B..] | R1:release@M1 R2:free@M2 R3:grab@A
 237.253 M1        початок обробки деталі #5 (місце 0), кінець о 303.549
 237.253 Robot1    вивільнив деталь #5, вільний у M1
         стан      A=1 | M1[BB.] | M2[B..] | R1:free@M1 R2:free@M2 R3:grab@A
 242.839 M2        кінець обробки деталі #3 (місце 0) -> блокування
 242.839 Robot2    призначено на маршрут 3 (M2->W) для деталі #3, холостий хід M2->M2 0
 242.839 Robot2    початок захоплення деталі #3 у M2
         стан      A=1 | M1[BB.] | M2[u..] | R1:free@M1 R2:grab@M2 R3:grab@A
 245.178 Create    надходження деталі #7
 245.178 Robot1    призначено на маршрут 1 (A->M1) для деталі #7, холостий хід M1->A 6
         стан      A=2 | M1[BB.] | M2[u..] | R1:empty_move@M1 R2:grab@M2 R3:grab@A
 245.344 M1        кінець обробки деталі #4 (місце 1) -> блокування
         стан      A=2 | M1[Bx.] | M2[u..] | R1:empty_move@M1 R2:grab@M2 R3:grab@A
```

```
 245.611 Robot3    деталь #6 захоплено, хід A->M1
         стан      A=1 | M1[Bx.] | M2[u..] | R1:empty_move@M1 R2:grab@M2 R3:loaded_move@A
 250.412 Robot2    деталь #3 захоплено, хід M2->W
         стан      A=1 | M1[Bx.] | M2[...] | R1:empty_move@M1 R2:loaded_move@M2
R3:loaded_move@A
 251.178 Robot1    початок захоплення деталі #7 у A
         стан      A=1 | M1[Bx.] | M2[...] | R1:grab@A R2:loaded_move@M2 R3:loaded_move@A
 251.611 Robot3    початок вивільнення деталі #6 у M1
         стан      A=1 | M1[Bxl] | M2[...] | R1:grab@A R2:loaded_move@M2 R3:release@M1
 255.412 Robot2    початок вивільнення деталі #3 у W
         стан      A=1 | M1[Bxl] | M2[...] | R1:grab@A R2:release@W R3:release@M1
 259.204 Robot1    деталь #7 захоплено, хід A->M1
         стан      A=0 | M1[Bxl] | M2[...] | R1:loaded_move@A R2:release@W R3:release@M1
 259.879 M1        початок обробки деталі #6 (місце 2), кінець о 326.641
 259.879 Robot3    вивільнив деталь #6, вільний у M1
 259.879 Robot3    призначено на маршрут 2 (M1->M2) для деталі #4, холостий хід M1->M1 0
 259.879 Robot3    початок захоплення деталі #4 у M1
         стан      A=0 | M1[BuB] | M2[...] | R1:loaded_move@A R2:release@W R3:grab@M1
 263.821 Warehouse деталь #3 на складі, час проходження 166.187
 263.821 Robot2    вивільнив деталь #3, вільний у W
         стан      A=0 | M1[BuB] | M2[...] | R1:loaded_move@A R2:free@W R3:grab@M1
 265.204 Robot1    на M1 немає місця, очікування з деталлю #7
         стан      A=0 | M1[BuB] | M2[...] | R1:wait_slot@M1 R2:free@W R3:grab@M1
 267.296 Robot1    початок вивільнення деталі #7 у M1
 267.296 Robot3    деталь #4 захоплено, хід M1->M2
         стан      A=0 | M1[BlB] | M2[...] | R1:release@M1 R2:free@W R3:loaded_move@M1
 272.731 Create    надходження деталі #8
 272.731 Robot2    призначено на маршрут 1 (A->M1) для деталі #8, холостий хід W->A 18
         стан      A=1 | M1[BlB] | M2[...] | R1:release@M1 R2:empty_move@W
R3:loaded_move@M1
 274.296 Robot3    початок вивільнення деталі #4 у M2
         стан      A=1 | M1[BlB] | M2[l..] | R1:release@M1 R2:empty_move@W R3:release@M2
 275.032 M1        початок обробки деталі #7 (місце 1), кінець о 328.132
 275.032 Robot1    вивільнив деталь #7, вільний у M1
         стан      A=1 | M1[BBB] | M2[l..] | R1:free@M1 R2:empty_move@W R3:release@M2
 282.691 M2        початок обробки деталі #4 (місце 0), кінець о 316.419
 282.691 Robot3    вивільнив деталь #4, вільний у M2
         стан      A=1 | M1[BBB] | M2[B..] | R1:free@M1 R2:empty_move@W R3:free@M2
 290.731 Robot2    початок захоплення деталі #8 у A
         стан      A=1 | M1[BBB] | M2[B..] | R1:free@M1 R2:grab@A R3:free@M2
 298.705 Robot2    деталь #8 захоплено, хід A->M1
         стан      A=0 | M1[BBB] | M2[B..] | R1:free@M1 R2:loaded_move@A R3:free@M2
```

ДОДАТОК Б. ЛІСТИНГ КОДУ АЛГОРИТМУ ІМІТАЦІЇ

Наведено модулі пакета `fms`, що реалізують алгоритм імітації (таблиця 3.1). Повний код програми, зокрема командний інтерфейс, автоматичні тести і побудову рисунків, розміщено в репозиторії проєкту.

Б.1 Модуль rng.py – генератори випадкових величин

```
from __future__ import annotations

import math
import random
from dataclasses import dataclass


class Stream:

    def __init__(self, seed: int | str):
        self._rnd = random.Random(seed)
        self._spare_normal: float | None = None

    def zeta(self) -> float:
        """Рівномірна величина на (0, 1]."""
        return 1.0 - self._rnd.random()

    def exponential(self, mean: float) -> float:
        return -mean * math.log(self.zeta())

    def uniform(self, center: float, half_width: float) -> float:
        return center - half_width + 2.0 * half_width * self._rnd.random()

    def standard_normal(self) -> float:
        if self._spare_normal is not None:
            value, self._spare_normal = self._spare_normal, None
            return value
        while True:
            a1 = 2.0 * self._rnd.random() - 1.0
            a2 = 2.0 * self._rnd.random() - 1.0
            s = a1 * a1 + a2 * a2
            if 0.0 < s <= 1.0:
                break
        beta = math.sqrt(-2.0 * math.log(s) / s)
        self._spare_normal = a2 * beta
        return a1 * beta

    def normal(self, mean: float, std: float) -> float:
        return mean + std * self.standard_normal()

    def shuffle(self, items: list) -> None:
        self._rnd.shuffle(items)

    def random(self) -> float:
        return self._rnd.random()


@dataclass(frozen=True)
class Dist:

    kind: str
    a: float
    b: float = 0.0

    def sample(self, stream: Stream) -> float:
        if self.kind == "const":
            return self.a
        if self.kind == "exp":
```

```
            return stream.exponential(self.a)
        if self.kind == "unif":
            return stream.uniform(self.a, self.b)
        if self.kind == "norm":

            while True:
                value = stream.normal(self.a, self.b)
                if value > 0.0:
                    return value
        raise ValueError(f"Невідомий закон розподілу: {self.kind}")

    @property
    def mean(self) -> float:
        return self.a

    def __str__(self) -> str:
        if self.kind == "const":
            return f"{self.a:g}"
        if self.kind == "exp":
            return f"exp({self.a:g})"
        return f"{self.kind}({self.a:g},{self.b:g})"


def const(d: float) -> Dist:
    return Dist("const", d)


def exp(mean: float) -> Dist:
    return Dist("exp", mean)


def unif(center: float, half_width: float) -> Dist:
    return Dist("unif", center, half_width)


def norm(mean: float, std: float) -> Dist:
    return Dist("norm", mean, std)


STREAM_NAMES = ("arrivals", "m1", "m2", "handling", "ties")


class RandomStreams:


    def __init__(self, seed: int):
        self.seed = seed
        for name in STREAM_NAMES:
            setattr(self, name, Stream(f"{seed}:{name}"))

    arrivals: Stream
    m1: Stream
    m2: Stream
    handling: Stream
    ties: Stream
```

Б.2 Модуль params.py – вхідні змінні та параметри моделі

```
from __future__ import annotations

from dataclasses import dataclass, field, replace

from .rng import Dist, exp, norm, unif

# Стратегії закріплення роботів
DEDICATED = "dedicated"  # 1) по одному роботу на кожний шлях переміщення
NEAREST = "nearest"      # 2) будь-який робот на будь-якому шляху, займається найближчий
```

```
DEFAULT_POSITIONS = {"A": 0.0, "M1": 6.0, "M2": 13.0, "W": 18.0}


@dataclass(frozen=True)
class Params:
    # вхідні змінні
    arrival: Dist = exp(40.0)        # інтервал надходження деталей
    m1_time: Dist = norm(60.0, 10.0)  # час обробки на першому верстаті
    m2_time: Dist = exp(100.0)        # час обробки на другому верстаті
    handling: Dist = unif(8.0, 1.0)   # час захоплення або вивільнення деталі роботом

    # параметри
    m1_capacity: int = 3
    m2_capacity: int = 3
    n_robots: int = 3
    positions: dict[str, float] = field(default_factory=lambda: dict(DEFAULT_POSITIONS))


    dedicated_routes: tuple[int, ...] | None = None
    route_rank: tuple[int, int, int] = (3, 2, 1)

    def distance(self, a: str, b: str) -> float:
        return abs(self.positions[a] - self.positions[b])

    def with_(self, **changes) -> "Params":
        return replace(self, **changes)

    def scaled_travel(self, factor: float) -> "Params":
        return replace(self, positions={k: v * factor for k, v in
self.positions.items()})

    def robot_routes(self) -> list[tuple[int, ...]]:
        if self.strategy == NEAREST:
            return [(1, 2, 3)] * self.n_robots
        if self.strategy == DEDICATED:
            routes = self.dedicated_routes or tuple(k % 3 + 1 for k in
range(self.n_robots))
            if len(routes) != self.n_robots or set(routes) != {1, 2, 3}:
                raise ValueError(f"dedicated_routes має покривати маршрути 1–3 для
кожного з "
                                 f"{self.n_robots} роботів: {routes}")
            return [(r,) for r in routes]
        raise ValueError(f"Невідома стратегія: {self.strategy}")

    def describe(self) -> str:
        pos = ", ".join(f"{k}={v:g}" for k, v in self.positions.items())
        return (
            f"strategy={self.strategy}, "
            f"arrival={self.arrival}, M1={self.m1_time}, M2={self.m2_time}, "
            f"handling={self.handling}, cap=({self.m1_capacity},{self.m2_capacity}), "
            f"robots={self.n_robots}"
            + (f", routes={self.dedicated_routes}" if self.dedicated_routes else "")
            + f", positions[{pos}]"
        )
```

Б.3 Модуль elements.py – елементи мережі масового обслуговування

```
from __future__ import annotations

import math
from collections import deque
from dataclasses import dataclass, field
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from .model import Model, Route

INF = math.inf
```

```
@dataclass
class Part:

    id: int
    t_arrival: float
    t_grab: dict[int, float] = field(default_factory=dict)   # маршрут -> початок
захоплення
    t_ready: dict[int, float] = field(default_factory=dict)  # маршрут -> момент
готовності до перевезення
    t_done: float = INF


@dataclass
class Task:

    route: int
    part: Part
    t_ready: float
    channel: "MachineChannel | None" = None  # канал верстата-джерела (для маршрутів 2 і
3)


class Element:
    def __init__(self, name: str, model: "Model"):
        self.name = name
        self.model = model

    def next_time(self) -> float:
        return INF

    def out_act(self, t: float) -> None:
        pass

    def update_stats(self, dt: float) -> None:
        pass

    def reset_stats(self) -> None:
        pass

    def state_str(self) -> str:
        return ""


class Create(Element):
    """Зовнішнє надходження деталей на пункт прибуття."""

    def __init__(self, name: str, model: "Model", target: "Queue"):
        super().__init__(name, model)
        self.target = target
        self.tnext = model.params.arrival.sample(model.streams.arrivals)
        self.created = 0

    def next_time(self) -> float:
        return self.tnext

    def out_act(self, t: float) -> None:
        self.created += 1
        part = Part(id=self.created, t_arrival=t)
        self.model.log(t, self.name, f"надходження деталі #{part.id}")
        self.target.put(part, t)
        self.tnext = t + self.model.params.arrival.sample(self.model.streams.arrivals)


class Queue(Element):

    def __init__(self, name: str, model: "Model", route_out: int):
        super().__init__(name, model)
        self.route_out = route_out
        self.length = 0
        self.max_length = 0
```

```
        self._sum_length = 0.0
        self._hist: dict[int, float] = {}  # довжина -> сумарний час перебування в цьому
стані

    def put(self, part: Part, t: float) -> None:
        self.length += 1
        self.max_length = max(self.max_length, self.length)
        part.t_ready[self.route_out] = t
        self.model.add_task(Task(route=self.route_out, part=part, t_ready=t))

    def grab_done(self, t: float) -> None:
        self.length -= 1

    def update_stats(self, dt: float) -> None:
        self._sum_length += self.length * dt
        self._hist[self.length] = self._hist.get(self.length, 0.0) + dt

    def reset_stats(self) -> None:
        self._sum_length = 0.0
        self._hist = {}
        self.max_length = self.length

    def mean_length(self, period: float) -> float:
        return self._sum_length / period if period > 0 else 0.0

    def length_distribution(self, period: float) -> dict[int, float]:
        return {k: v / period for k, v in sorted(self._hist.items())} if period > 0 else
{}

    def state_str(self) -> str:
        return f"A={self.length}"


FREE, LOADING, BUSY, BLOCKED, UNLOADING = (
    "free", "loading", "busy", "blocked", "unloading",
)
MACHINE_STATES = (FREE, LOADING, BUSY, BLOCKED, UNLOADING)
_STATE_CHAR = {FREE: ".", LOADING: "l", BUSY: "B", BLOCKED: "x", UNLOADING: "u"}


@dataclass
class MachineChannel:
    index: int
    state: str = FREE
    part: Part | None = None
    tnext: float = INF


class Machine(Element):

    def __init__(self, name: str, model: "Model", capacity: int, dist, stream, route_out:
int):
        super().__init__(name, model)
        self.channels = [MachineChannel(i) for i in range(capacity)]
        self.dist = dist
        self.stream = stream
        self.route_out = route_out
        self.incoming = 0  # роботи, призначені на доставку сюди, але ще без місця (u_i)
        self.waiting_robots: deque = deque()  # роботи з деталлю, що чекають вільного
місця
        self.processed = 0
        self._state_time = {s: 0.0 for s in MACHINE_STATES}
        self.robot_wait_time = 0.0  # час, коли біля верстата чекає робот з деталлю

    @property
    def capacity(self) -> int:
        return len(self.channels)

    def count(self, state: str) -> int:
```

```
        return sum(1 for ch in self.channels if ch.state == state)

    def _first(self, state: str) -> MachineChannel | None:
        return next((ch for ch in self.channels if ch.state == state), None)

    def next_time(self) -> float:
        return min(ch.tnext for ch in self.channels)

    def out_act(self, t: float) -> None:
        due = [ch for ch in self.channels if ch.tnext == t]
        self.model.streams.ties.shuffle(due)
        for ch in due:
            ch.state = BLOCKED
            ch.tnext = INF
            self.processed += 1
            ch.part.t_ready[self.route_out] = t
            self.model.log(t, self.name, f"кінець обробки деталі #{ch.part.id} (місце
{ch.index}) -> блокування")
            self.model.add_task(Task(route=self.route_out, part=ch.part, t_ready=t,
channel=ch))


    def can_dispatch_to(self) -> bool:
        """u_i <= f_i: не більше одного «зайвого» робота на верстат."""
        return self.incoming <= self.count(FREE)

    def on_dispatch(self) -> None:
        self.incoming += 1

    def take_for_release(self) -> MachineChannel | None:
        ch = self._first(FREE)
        if ch is None:
            return None
        self.incoming -= 1
        ch.state = LOADING
        return ch

    def release_done(self, ch: MachineChannel, part: Part, t: float) -> None:
        ch.state = BUSY
        ch.part = part
        ch.tnext = t + self.dist.sample(self.stream)
        self.model.log(t, self.name, f"початок обробки деталі #{part.id} (місце
{ch.index}), кінець о {ch.tnext:.3f}")

    def start_grab(self, ch: MachineChannel) -> None:
        ch.state = UNLOADING

    def grab_done(self, ch: MachineChannel, t: float) -> None:
        ch.state = FREE
        ch.part = None
                if self.waiting_robots:
            robot = self.waiting_robots.popleft()
            slot = self.take_for_release()
            robot.transport.start_release(robot, slot, t)


    def update_stats(self, dt: float) -> None:
        for ch in self.channels:
            self._state_time[ch.state] += dt
        if self.waiting_robots:
            self.robot_wait_time += dt

    def reset_stats(self) -> None:
        self._state_time = {s: 0.0 for s in MACHINE_STATES}
        self.processed = 0
        self.robot_wait_time = 0.0

    def utilization(self, period: float) -> dict[str, float]:
        denom = period * self.capacity
        share = {s: v / denom for s, v in self._state_time.items()} if denom > 0 else {}
```

```
        occupied = 1.0 - share.get(FREE, 1.0)
        return {
            "occupied": occupied,
            "processing": share.get(BUSY, 0.0),
            "blocked": share.get(BLOCKED, 0.0),
            "handling": share.get(LOADING, 0.0) + share.get(UNLOADING, 0.0),
        }

    def state_str(self) -> str:
        return f"{self.name}[{''.join(_STATE_CHAR[ch.state] for ch in self.channels)}]"


class Warehouse(Element):
    """Склад оброблених деталей – вихід з мережі."""

    def accept(self, part: Part, t: float) -> None:
        part.t_done = t
        self.model.log(t, self.name, f"деталь #{part.id} на складі, час проходження {t -
part.t_arrival:.3f}")
        self.model.record_completion(part)


R_FREE, R_TO_PICKUP, R_GRAB, R_CARRY, R_WAIT, R_RELEASE = (
    "free", "empty_move", "grab", "loaded_move", "wait_slot", "release",
)
ROBOT_STATES = (R_FREE, R_TO_PICKUP, R_GRAB, R_CARRY, R_WAIT, R_RELEASE)


@dataclass(eq=False)
class Robot:

    id: int
    location: str
    allowed_routes: tuple[int, ...]
    transport: "Transport" = None
    state: str = R_FREE
    tnext: float = INF
    task: Task | None = None
    route: "Route | None" = None
    dest_slot: MachineChannel | None = None
    jobs: dict[int, int] = field(default_factory=dict)
    empty_travel: dict[int, float] = field(default_factory=dict)  # сумарний холостий хід
за маршрутами
    state_time: dict[str, float] = field(default_factory=lambda: {s: 0.0 for s in
ROBOT_STATES})


class Transport(Element):

    def __init__(self, name: str, model: "Model", robots: list[Robot]):
        super().__init__(name, model)
        self.robots = robots
        for r in robots:
            r.transport = self

    def next_time(self) -> float:
        return min(r.tnext for r in self.robots)

    def _handling(self) -> float:
        return self.model.params.handling.sample(self.model.streams.handling)

    def assign(self, robot: Robot, task: Task, t: float) -> None:
        route = self.model.routes[task.route]
        robot.task = task
        robot.route = route
        robot.jobs[route.index] = robot.jobs.get(route.index, 0) + 1
        if isinstance(route.dest, Machine):
            route.dest.on_dispatch()
        travel = self.model.params.distance(robot.location, route.pickup)
        robot.empty_travel[route.index] = robot.empty_travel.get(route.index, 0.0) +
```

```
travel
        self.model.log(
            t, f"Robot{robot.id}",
            f"призначено на маршрут {route.index} ({route.pickup}->{route.drop}) для
деталі #{task.part.id}, "
            f"холостий хід {robot.location}->{route.pickup} {travel:g}",
        )
        if travel > 0:
            robot.state = R_TO_PICKUP
            robot.tnext = t + travel
        else:
            self._start_grab(robot, t)

    def _start_grab(self, robot: Robot, t: float) -> None:
        robot.location = robot.route.pickup
        robot.state = R_GRAB
        robot.tnext = t + self._handling()
        robot.task.part.t_grab[robot.route.index] = t
        if robot.task.channel is not None:
            robot.route.source.start_grab(robot.task.channel)
        self.model.log(t, f"Robot{robot.id}", f"початок захоплення деталі
#{robot.task.part.id} у {robot.location}")

    def _start_carry(self, robot: Robot, t: float) -> None:
        route = robot.route
        # деталь знята з місця-джерела
        if robot.task.channel is not None:
            route.source.grab_done(robot.task.channel, t)
        else:
            route.source.grab_done(t)
        robot.state = R_CARRY
        robot.tnext = t + self.model.params.distance(route.pickup, route.drop)
        self.model.log(t, f"Robot{robot.id}", f"деталь #{robot.task.part.id} захоплено,
хід {route.pickup}->{route.drop}")

    def _arrive(self, robot: Robot, t: float) -> None:
        route = robot.route
        robot.location = route.drop
        if isinstance(route.dest, Machine):
            slot = route.dest.take_for_release()
            if slot is None:
                robot.state = R_WAIT
                robot.tnext = INF
                route.dest.waiting_robots.append(robot)
                self.model.log(t, f"Robot{robot.id}", f"на {route.drop} немає місця,
очікування з деталлю #{robot.task.part.id}")
                return
            self.start_release(robot, slot, t)
        else:
            self.start_release(robot, None, t)

    def start_release(self, robot: Robot, slot: MachineChannel | None, t: float) -> None:
        robot.dest_slot = slot
        robot.state = R_RELEASE
        robot.tnext = t + self._handling()
        self.model.log(t, f"Robot{robot.id}", f"початок вивільнення деталі
#{robot.task.part.id} у {robot.location}")

    def _finish(self, robot: Robot, t: float) -> None:
        route, part = robot.route, robot.task.part
        if isinstance(route.dest, Machine):
            route.dest.release_done(robot.dest_slot, part, t)
        else:
            route.dest.accept(part, t)
        self.model.log(t, f"Robot{robot.id}", f"вивільнив деталь #{part.id}, вільний у
{robot.location}")
        robot.state = R_FREE
        robot.tnext = INF
        robot.task = robot.route = robot.dest_slot = None
```

```
    def out_act(self, t: float) -> None:
        due = [r for r in self.robots if r.tnext == t]
        self.model.streams.ties.shuffle(due)
        for robot in due:
            if robot.state == R_TO_PICKUP:
                self._start_grab(robot, t)
            elif robot.state == R_GRAB:
                self._start_carry(robot, t)
            elif robot.state == R_CARRY:
                self._arrive(robot, t)
            elif robot.state == R_RELEASE:
                self._finish(robot, t)

    def free_robots(self) -> list[Robot]:
        return [r for r in self.robots if r.state == R_FREE]


    def update_stats(self, dt: float) -> None:
        for r in self.robots:
            r.state_time[r.state] += dt

    def reset_stats(self) -> None:
        for r in self.robots:
            r.state_time = {s: 0.0 for s in ROBOT_STATES}
            r.jobs = {}
            r.empty_travel = {}

    def state_str(self) -> str:
        return " ".join(f"R{r.id}:{r.state}@{r.location}" for r in self.robots)
```

Б.4 Модуль model.py – алгоритм імітації

```
from __future__ import annotations

import math
from collections import deque
from dataclasses import dataclass
from typing import Callable

from .elements import (
    INF, Create, Element, Machine, Part, Queue, Robot, Task, Transport, Warehouse,
)
from .params import DEDICATED, Params
from .rng import RandomStreams


@dataclass
class Route:
    index: int
    pickup: str
    drop: str
    source: Element
    dest: Element

    def blocked(self) -> bool:

        return isinstance(self.dest, Machine) and not self.dest.can_dispatch_to()


class Model:
    def __init__(self, params: Params, seed: int = 1, trace: bool = False,
                 trace_until: float = INF):
        self.params = params
        self.streams = RandomStreams(seed)
        self.seed = seed
        self.tcurr = 0.0
        self.trace = trace
        self.trace_until = trace_until
        self.trace_lines: list[str] = []
```

```
        self.tasks: dict[int, deque[Task]] = {1: deque(), 2: deque(), 3: deque()}
        self.completed: list[Part] = []
        self.completed_total = 0
        self.t_stats_start = 0.0

        p = params
        self.buffer = Queue("Arrival", self, route_out=1)
        self.create = Create("Create", self, self.buffer)
        self.m1 = Machine("M1", self, p.m1_capacity, p.m1_time, self.streams.m1,
route_out=2)
        self.m2 = Machine("M2", self, p.m2_capacity, p.m2_time, self.streams.m2,
route_out=3)
        self.warehouse = Warehouse("Warehouse", self)
        self.routes = {
            1: Route(1, "A", "M1", self.buffer, self.m1),
            2: Route(2, "M1", "M2", self.m1, self.m2),
            3: Route(3, "M2", "W", self.m2, self.warehouse),
        }

        self.robots: list[Robot] = []
        for k, allowed in enumerate(p.robot_routes()):
            home = self.routes[allowed[0]].pickup if len(allowed) == 1 else ("A", "M1",
"M2")[k % 3]
            self.robots.append(Robot(id=k + 1, location=home, allowed_routes=allowed))

        if p.strategy == DEDICATED:
            # окрема одноканальна СМО для кожного маршруту
            self.transports = [Transport(f"T{r.id}", self, [r]) for r in self.robots]
        else:
            self.transports = [Transport("T", self, self.robots)]

        self.event_elements: list[Element] = [self.m1, self.m2, *self.transports]
        self.stat_elements: list[Element] = [self.buffer, self.m1, self.m2,
*self.transports]

    def log(self, t: float, who: str, what: str) -> None:
        if self.trace and t <= self.trace_until:
            self.trace_lines.append(f"{t:11.3f}  {who:<9} {what}")

    def log_state(self, t: float) -> None:
        if self.trace and t <= self.trace_until:
            parts = [self.buffer.state_str(), self.m1.state_str(), self.m2.state_str(),
                     " ".join(tr.state_str() for tr in self.transports)]
            self.trace_lines.append(f"{'':11}  {'стан':<9} " + " | ".join(parts))

    def add_task(self, task: Task) -> None:
        self.tasks[task.route].append(task)

    def record_completion(self, part: Part) -> None:
        self.completed_total += 1
        if self.tcurr >= self.t_stats_start:
            self.completed.append(part)

    def wip(self) -> int:
        return self.create.created - self.completed_total

    def dispatch(self, t: float) -> None:
        rank = self.params.route_rank
        while True:
            best = None
            for tr in self.transports:
                for robot in tr.free_robots():
                    for r in robot.allowed_routes:
                        queue = self.tasks[r]
                        if not queue:
                            continue
                        route = self.routes[r]
                        if route.blocked():
                            continue
```

```
                        key = (self.params.distance(robot.location, route.pickup), rank[r
- 1],
                               queue[0].t_ready, self.streams.ties.random())
                        if best is None or key < best[0]:
                            best = (key, tr, robot, r)
            if best is None:
                return
            _, tr, robot, r = best
            tr.assign(robot, self.tasks[r].popleft(), t)

    def _next_event_time(self) -> float:
        return min(self.create.next_time(), *(e.next_time() for e in
self.event_elements))

    def _advance_stats(self, t_to: float) -> None:
        dt = t_to - self.tcurr
        if dt > 0:
            for e in self.stat_elements:
                e.update_stats(dt)
        self.tcurr = t_to

    def reset_stats(self) -> None:
        self.t_stats_start = self.tcurr
        self.completed = []
        for e in self.stat_elements:
            e.reset_stats()
        self._created_at_reset = self.create.created

    def run(self, t_end: float, t_warmup: float = 0.0,
            observe_every: float | None = None,
            observer: Callable[["Model"], None] | None = None) -> "Model":

        self.t_end = t_end
        self._created_at_reset = 0
        pending_reset = t_warmup > 0
        next_obs = observe_every if observe_every else INF
        self.dispatch(0.0)
        self.log_state(0.0)
        while True:
            t_next = self._next_event_time()
            while True:
                t_service = min(t_warmup if pending_reset else INF, next_obs)
                if t_service > min(t_next, t_end):
                    break
                self._advance_stats(t_service)
                if pending_reset and t_service == t_warmup:
                    self.reset_stats()
                    pending_reset = False
                if t_service == next_obs:
                    observer(self)
                    next_obs += observe_every
            if t_next > t_end:
                self._advance_stats(t_end)
                break
            self._advance_stats(t_next)
            t = self.tcurr
            due = [e for e in self.event_elements if e.next_time() == t]
            self.streams.ties.shuffle(due)
            for e in due:
                e.out_act(t)
            if self.create.next_time() == t:
                self.create.out_act(t)
            self.dispatch(t)
            self.log_state(t)
        return self

    @property
    def period(self) -> float:
        return self.tcurr - self.t_stats_start
```

```
    def sojourn_times(self) -> list[float]:
        return [p.t_done - p.t_arrival for p in self.completed]

    def results(self) -> dict:
        return collect_results(self)


def _mean(xs: list[float]) -> float:
    return sum(xs) / len(xs) if xs else math.nan


def _std(xs: list[float]) -> float:
    if len(xs) < 2:
        return math.nan
    m = _mean(xs)
    return math.sqrt(sum((x - m) ** 2 for x in xs) / (len(xs) - 1))


def _quantile(sorted_xs: list[float], q: float) -> float:
    if not sorted_xs:
        return math.nan
    pos = q * (len(sorted_xs) - 1)
    lo, hi = math.floor(pos), math.ceil(pos)
    return sorted_xs[lo] + (sorted_xs[hi] - sorted_xs[lo]) * (pos - lo)


def _time_quantile(dist: dict[int, float], q: float) -> float:
    acc = 0.0
    for value, share in dist.items():
        acc += share
        if acc >= q - 1e-12:
            return value
    return max(dist) if dist else math.nan


def collect_results(m: Model) -> dict:
    T = m.period
    soj = sorted(m.sojourn_times())
    parts = m.completed
    res: dict = {
        "strategy": m.params.strategy,
        "seed": m.seed,
        "t_end": m.tcurr,
        "t_warmup": m.t_stats_start,
        "arrived": m.create.created - m._created_at_reset,
        "completed": len(parts),
        "wip_end": m.wip(),
        "throughput_per_hour": len(parts) / T * 3600 if T > 0 else math.nan,
        "sojourn_mean": _mean(soj),
        "sojourn_std": _std(soj),
        "sojourn_min": soj[0] if soj else math.nan,
        "sojourn_p50": _quantile(soj, 0.5),
        "sojourn_p90": _quantile(soj, 0.9),
        "sojourn_p95": _quantile(soj, 0.95),
        "sojourn_max": soj[-1] if soj else math.nan,
        "wait_route1_mean": _mean([p.t_grab[1] - p.t_ready[1] for p in parts]),
        "wait_route2_mean": _mean([p.t_grab[2] - p.t_ready[2] for p in parts]),
        "wait_route3_mean": _mean([p.t_grab[3] - p.t_ready[3] for p in parts]),
        "buffer_max": m.buffer.max_length,
        "buffer_mean": m.buffer.mean_length(T),
        "buffer_end": m.buffer.length,
        "buffer_q95": _time_quantile(m.buffer.length_distribution(T), 0.95),
        "buffer_q99": _time_quantile(m.buffer.length_distribution(T), 0.99),
    }
    for mach in (m.m1, m.m2):
        for k, v in mach.utilization(T).items():
            res[f"{mach.name}_{k}"] = v
        res[f"{mach.name}_robot_waiting"] = mach.robot_wait_time / T if T > 0 else
math.nan
    for route in (1, 2, 3):
```

```
        n = sum(r.jobs.get(route, 0) for r in m.robots)
        travel = sum(r.empty_travel.get(route, 0.0) for r in m.robots)
        res[f"empty_route{route}_mean"] = travel / n if n else math.nan
    utils = []
    for r in m.robots:
        busy = sum(v for s, v in r.state_time.items() if s != "free")
        u = busy / T if T > 0 else math.nan
        utils.append(u)
        res[f"robot{r.id}_util"] = u
        for s, v in r.state_time.items():
            if s != "free":
                res[f"robot{r.id}_{s}"] = v / T if T > 0 else math.nan
        for route in (1, 2, 3):
            res[f"robot{r.id}_jobs_route{route}"] = r.jobs.get(route, 0)
    res["robots_util_mean"] = _mean(utils)
    return res


def simulate(params: Params, seed: int, t_end: float, t_warmup: float = 0.0) ->
tuple[dict, list[float]]:
    m = Model(params, seed=seed).run(t_end, t_warmup)
    return m.results(), m.sojourn_times()
```

ДОДАТОК В. ЛІСТИНГ КОДУ ЕКСПЕРИМЕНТАЛЬНОГО ДОСЛІДЖЕННЯ

Наведено модулі пакета `fms.experiments`, які виконують серії експериментів з таблиці 4.1. Команди для їх запуску наведено в підрозділі 4.8.

В.1 Модуль common.py – спільні засоби для серій прогонів

```
from __future__ import annotations

import csv
import json
import math
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

import numpy as np
from scipy import stats

from ..model import simulate
from ..params import DEDICATED, NEAREST, Params

KEY_OUTPUTS = (
    "throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
    "M1_occupied", "M1_processing", "M1_blocked", "M2_occupied", "M2_processing",
"M2_blocked",
    "robot1_util", "robot2_util", "robot3_util", "robots_util_mean",
    "M1_robot_waiting", "M2_robot_waiting", "empty_route1_mean", "empty_route2_mean",
"empty_route3_mean",
)

STRATEGY_LABEL = {DEDICATED: "1) закріплені", NEAREST: "2) найближчий"}


def variants(strategy: str | None) -> list[str]:
    return [strategy] if strategy else [DEDICATED, NEAREST]


def _one(args):
    params, seed, t_end, t_warmup, keep_sojourn = args
    res, soj = simulate(params, seed, t_end, t_warmup)
    return res, (soj if keep_sojourn else None)


def run_batch(tasks: list[tuple], jobs: int = 1) -> list[tuple[dict, list[float] |
None]]:
    if jobs > 1 and len(tasks) > 1:
        with ProcessPoolExecutor(max_workers=jobs) as ex:
            return list(ex.map(_one, tasks, chunksize=max(1, len(tasks) // (4 * jobs))))
    return [_one(t) for t in tasks]


def replicate(params: Params, n: int, t_end: float, t_warmup: float,
              seed0: int = 1, jobs: int = 1, keep_sojourn: bool = False):
    tasks = [(params, seed0 + i, t_end, t_warmup, keep_sojourn) for i in range(n)]
    out = run_batch(tasks, jobs)
    results = [r for r, _ in out]
    sojourns = [s for _, s in out] if keep_sojourn else None
    return results, sojourns


def summarize(values, beta: float = 0.95) -> dict:
    x = np.asarray([v for v in values if not (isinstance(v, float) and math.isnan(v))],
dtype=float)
    n = len(x)
    if n == 0:
```

```
        return {"n": 0, "mean": math.nan, "std": math.nan, "ci": math.nan}
    mean = float(x.mean())
    std = float(x.std(ddof=1)) if n > 1 else math.nan
    ci = float(stats.t.ppf((1 + beta) / 2, n - 1) * std / math.sqrt(n)) if n > 1 else
math.nan
    return {"n": n, "mean": mean, "std": std, "ci": ci}


def out_dir(path: str | Path) -> Path:
    p = Path(path)
    p.mkdir(parents=True, exist_ok=True)
    return p


def write_csv(path: Path, rows: list[dict]) -> None:
    if not rows:
        return
    keys: list[str] = []
    for r in rows:
        for k in r:
            if k not in keys:
                keys.append(k)
    with path.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=keys)
        w.writeheader()
        w.writerows(rows)


def write_json(path: Path, data) -> None:
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2, default=float),
encoding="utf-8")


def fmt(v, digits: int = 3) -> str:
    if isinstance(v, float):
        if math.isnan(v):
            return "—"
        return f"{v:.{digits}f}"
    return str(v)


def print_table(rows: list[dict], columns: list[str], headers: list[str] | None = None) -
> None:
    headers = headers or columns
    cells = [[fmt(r.get(c, "")) for c in columns] for r in rows]
    widths = [max(len(h), *(len(c[i]) for c in cells)) if cells else len(h) for i, h in
enumerate(headers)]
    print("  ".join(h.ljust(w) for h, w in zip(headers, widths)))
    print("  ".join("-" * w for w in widths))
    for c in cells:
        print("  ".join(v.ljust(w) for v, w in zip(c, widths)))
```

B.2 Модуль verification.py – верифікація моделі

```
from __future__ import annotations

from ..params import Params
from ..rng import const, exp, norm, unif
from .common import replicate, summarize

SCENARIOS = [
    ("базовий", {}, "—"),
    ("рідше надходження exp(80)", {"arrival": exp(80.0)},
     "пропускна ≈ 45/год, черга A мала, завантаження ↓"),
    ("частіше надходження exp(30)", {"arrival": exp(30.0)},
     "перевантаження: черга A росте, пропускна обмежена M2"),
    ("швидший M2 exp(50)", {"m2_time": exp(50.0)},
     "M2 не вузьке місце: пропускна ≈ 90/год, час проходження ↓"),
    ("повільніший M1 norm(120,10)", {"m1_time": norm(120.0, 10.0)},
```

```
     "M1 стає вузьким місцем, зайнятість M1 ≈ 1"),
    ("M2 на 6 місць", {"m2_capacity": 6},
     "пропускна ≈ 90/год, блокування M1 зникає"),
    ("швидке захоплення unif(2,0.5)", {"handling": unif(2.0, 0.5)},
     "завантаження роботів ↓, пропускна ↑"),
    ("переїзди ×3", "travel3",
     "завантаження роботів ↑, час проходження ↑"),
    ("детерміновані часи, exp→const(300)", {"arrival": const(300.0), "m1_time":
const(60.0),
                                            "m2_time": const(100.0), "handling":
const(8.0)},
     "перша деталь 226 с, решта 244 с (226 + холості ходи 6+7+5)"),
]

COLUMNS = ["throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
           "M1_occupied", "M2_occupied", "M2_processing", "robots_util_mean", "wip_end"]
INPUT_COLUMNS = ["in_arrival", "in_m1", "in_m2", "in_handling", "in_m2_capacity",
"in_travel"]


def inputs_of(p: Params, travel: float) -> dict:
    return {"in_arrival": str(p.arrival), "in_m1": str(p.m1_time), "in_m2":
str(p.m2_time),
            "in_handling": str(p.handling), "in_m2_capacity": p.m2_capacity, "in_travel":
f"×{travel:g}"}


def run(strategy: str, reps: int, t_end: float, t_warmup: float, jobs: int) ->
list[dict]:
    rows = []
    for name, change, expected in SCENARIOS:
        p = Params(strategy=strategy)
        travel = 3.0 if change == "travel3" else 1.0
        p = p.scaled_travel(travel) if change == "travel3" else p.with_(**change)
        results, _ = replicate(p, reps, t_end, t_warmup, jobs=jobs)
        row = {"scenario": name, "strategy": strategy, **inputs_of(p, travel),
"expected": expected}
        for c in COLUMNS:
            row[c] = summarize([r[c] for r in results])["mean"]
        rows.append(row)
    return rows
```

B.3 Модуль warmup.py – тривалість перехідного періоду

```
from __future__ import annotations

import math

from ..model import Model
from ..params import Params

METRICS = ("throughput_per_hour", "sojourn_mean", "buffer_mean", "M2_occupied",
"robots_util_mean")


def _snapshot(m: Model) -> dict:
    t = m.tcurr
    soj = [p.t_done - p.t_arrival for p in m.completed]
    busy = [sum(v for s, v in r.state_time.items() if s != "free") / t for r in m.robots]
    return {
        "t": t,
        "throughput_per_hour": m.completed_total / t * 3600,
        "sojourn_mean": sum(soj) / len(soj) if soj else math.nan,
        "buffer_mean": m.buffer.mean_length(t),
        "M2_occupied": m.m2.utilization(t)["occupied"],
        "robots_util_mean": sum(busy) / len(busy),
        "buffer_now": m.buffer.length,
        "buffer_max": m.buffer.max_length,
    }
```

```
def observe(params: Params, seeds: list[int], t_end: float, step: float) -> dict[int,
list[dict]]:
    series = {}
    for seed in seeds:
        snaps: list[dict] = []
        Model(params, seed=seed).run(t_end, observe_every=step, observer=lambda m:
snaps.append(_snapshot(m)))
        series[seed] = snaps
    return series


def transient_time(series: dict[int, list[dict]], metric: str, eps: float) -> float:
    runs = list(series.values())
    n_points = min(len(r) for r in runs)
    T = math.nan
    for k in range(n_points - 1, -1, -1):
        vals = [r[k][metric] for r in runs]
        if any(math.isnan(v) for v in vals) or max(vals) - min(vals) >= eps:
            break
        T = runs[0][k]["t"]
    return T


def run(params: Params, n_runs: int, t_end: float, step: float, eps_rel: float):
    seeds = list(range(1, n_runs + 1))
    series = observe(params, seeds, t_end, step)
    rows = [{"strategy": params.strategy, "arrival_mean": params.arrival.mean, "seed": s,
**snap}
            for s, snaps in series.items() for snap in snaps]
    summary = []
    for metric in METRICS:
        final = [snaps[-1][metric] for snaps in series.values()]
        level = sum(final) / len(final)
        eps = eps_rel * abs(level)
        summary.append({
            "strategy": params.strategy, "arrival_mean": params.arrival.mean, "metric":
metric,
            "final_mean": level, "eps": eps, "T_transient": transient_time(series,
metric, eps),
            "growth_last_half": _growth(series, metric),
        })
    return rows, summary


def _growth(series: dict[int, list[dict]], metric: str) -> float:
    ratios = []
    for snaps in series.values():
        mid, last = snaps[len(snaps) // 2][metric], snaps[-1][metric]
        if mid and not math.isnan(mid):
            ratios.append(last / mid - 1.0)
    return sum(ratios) / len(ratios) if ratios else math.nan
```

B.4 Модуль replications.py – кількість прогонів

```
from __future__ import annotations

import math

from ..params import Params
from .common import replicate, summarize

RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "M2_occupied",
"robots_util_mean")


def run(params: Params, pilot: int, t_end: float, t_warmup: float, eps_rel: float,
        beta: float, jobs: int) -> list[dict]:
```

```
    results, _ = replicate(params, pilot, t_end, t_warmup, jobs=jobs)
    rows = []
    for resp in RESPONSES:
        s = summarize([r[resp] for r in results], beta)
        eps = eps_rel * abs(s["mean"])
        sigma = s["std"]
        p_cheb = math.ceil(sigma ** 2 / (eps ** 2 * (1 - beta))) if eps > 0 else math.nan
        p_clt = math.ceil((1.959964 * sigma / eps) ** 2) if eps > 0 else math.nan
        rows.append({
            "strategy": params.strategy, "arrival_mean": params.arrival.mean, "response":
resp,
            "pilot_runs": pilot, "mean": s["mean"], "std": sigma, "eps": eps,
            "p_chebyshev": p_cheb, "p_clt": p_clt,
        })
    return rows
```

B.5 Модуль compare.py – порівняння способів закріплення роботів

```
from __future__ import annotations

import numpy as np
from scipy import stats

from ..params import DEDICATED, NEAREST, Params
from .common import KEY_OUTPUTS, replicate, summarize

RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
             "M1_occupied", "M2_occupied", "robots_util_mean")


def anova(groups: list[np.ndarray], alpha: float = 0.05) -> dict:
    q = len(groups)
    p = len(groups[0])
    means = np.array([g.mean() for g in groups])
    grand = np.concatenate(groups).mean()
    s_f = p * ((means - grand) ** 2).sum()
    s_r = sum(((g - g.mean()) ** 2).sum() for g in groups)
    d_f_guide = s_f / (q * p - 1)
    d_r = s_r / (q * (p - 1))
    df1, df2 = q - 1, q * (p - 1)
    f_guide = d_f_guide / d_r if d_r > 0 else np.inf
    f_classic = (s_f / df1) / d_r if d_r > 0 else np.inf
    f_crit = stats.f.ppf(1 - alpha, df1, df2)
    return {
        "S_f": s_f, "S_r": s_r, "D_f(3.10)": d_f_guide, "D_r(3.11)": d_r,
        "F(3.10/3.11)": f_guide, "F_classic": f_classic, "F_crit": f_crit,
        "p_value": float(stats.f.sf(f_classic, df1, df2)),
        "significant": bool(f_classic > f_crit),
    }


def run(reps: int, t_end: float, t_warmup: float, jobs: int,
        base: Params | None = None) -> tuple[list[dict], list[dict], list[dict]]:
    base = base or Params()
    runs: dict[str, list[dict]] = {}
    for strategy in (DEDICATED, NEAREST):
        runs[strategy], _ = replicate(base.with_(strategy=strategy), reps, t_end,
t_warmup, jobs=jobs)

    summary = []
    for strategy, results in runs.items():
        row = {"strategy": strategy, "arrival_mean": base.arrival.mean}
        for k in KEY_OUTPUTS:
            s = summarize([r[k] for r in results])
            row[k] = s["mean"]
            row[k + "_ci"] = s["ci"]
        summary.append(row)

    tests = []
```

```
    for resp in RESPONSES:
        a = np.array([r[resp] for r in runs[DEDICATED]], dtype=float)
        b = np.array([r[resp] for r in runs[NEAREST]], dtype=float)
        res = {"response": resp, "arrival_mean": base.arrival.mean,
               "mean_dedicated": a.mean(), "mean_nearest": b.mean(), **anova([a, b])}
        diff = b - a
        t_stat, t_p = stats.ttest_rel(b, a)
        res.update({"paired_diff_mean": diff.mean(), "paired_diff_ci":
summarize(diff)["ci"],
                    "paired_t": float(t_stat), "paired_p": float(t_p)})
        tests.append(res)

    raw = [r for results in runs.values() for r in results]
    return summary, tests, raw
```

B.6 Модуль capacity.py – межа стійкості

```
from __future__ import annotations

from ..params import Params
from ..rng import exp
from .common import run_batch, summarize, variants

DEFAULT_ARRIVALS = (34, 36, 38, 40, 42, 44, 45, 46, 47, 48, 49, 50, 52, 55, 60, 70, 80)
RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
"buffer_q99",
             "wip_end", "M1_occupied", "M2_occupied", "M2_processing", "M2_blocked",
"robots_util_mean",
             "robot1_util", "robot2_util", "robot3_util")


def run(arrivals, strategy, reps: int, t_end: float, t_warmup: float, jobs: int,
        base: Params | None = None) -> list[dict]:
    base = base or Params()
    points = [(s, a) for s in variants(strategy) for a in arrivals]
    tasks = [(base.with_(strategy=s, arrival=exp(float(a))), seed, t_end, t_warmup,
False)
             for s, a in points for seed in range(1, reps + 1)]
    out = run_batch(tasks, jobs)
    rows = []
    for i, (s, a) in enumerate(points):
        results = [r for r, _ in out[i * reps:(i + 1) * reps]]
        row = {"strategy": s, "arrival_mean": a, "input_per_hour": 3600 / a}
        for k in RESPONSES:
            st = summarize([r[k] for r in results])
            row[k] = st["mean"]
            row[k + "_ci"] = st["ci"]
        row["output_input_ratio"] = row["throughput_per_hour"] / row["input_per_hour"]
        row["stable"] = row["output_input_ratio"] >= 0.98 and row["wip_end"] < 30
        rows.append(row)
    return rows


def max_stable(rows: list[dict]) -> list[dict]:
    out = {}
    for r in rows:
        key = r["strategy"]
        cur = out.setdefault(key, {"strategy": key,
                                   "max_stable_input_per_hour": 0.0,
"min_stable_arrival_mean": None,
                                   "max_throughput_per_hour": 0.0})
        cur["max_throughput_per_hour"] = max(cur["max_throughput_per_hour"],
r["throughput_per_hour"])
        if r["stable"] and r["input_per_hour"] > cur["max_stable_input_per_hour"]:
            cur["max_stable_input_per_hour"] = r["input_per_hour"]
            cur["min_stable_arrival_mean"] = r["arrival_mean"]
    return list(out.values())
```

В.7 Модуль distribution.py – розподіл часу проходження деталей

```
from __future__ import annotations

import math

import numpy as np
from scipy import stats

from ..params import Params
from .common import replicate

CANDIDATES = {
    "normal": (stats.norm, {}, 2),
    "exponential (shifted)": (stats.expon, {}, 2),
    "gamma (shifted)": (stats.gamma, {}, 3),
    "lognormal (shifted)": (stats.lognorm, {}, 3),
    "erlang/gamma (loc=0)": (stats.gamma, {"floc": 0.0}, 2),
}


def histogram(data: np.ndarray, bins: int) -> tuple[np.ndarray, np.ndarray]:
    counts, edges = np.histogram(data, bins=bins)
    return counts, edges


def chi_square(data: np.ndarray, dist, params: tuple, n_params: int, bins: int) -> dict:
    counts, edges = histogram(data, bins)
    # крайні інтервали розширюємо до нескінченності, щоб Σ p_i = 1
    cdf = dist.cdf(edges, *params)
    cdf[0], cdf[-1] = 0.0, 1.0
    probs = np.diff(cdf)
    n = len(data)
    obs, exp_ = [], []
    acc_o = acc_e = 0.0
    for o, p in zip(counts, probs):
        acc_o += o
        acc_e += n * p
        if acc_e >= 5:
            obs.append(acc_o)
            exp_.append(acc_e)
            acc_o = acc_e = 0.0
    if acc_e > 0 and exp_:
        obs[-1] += acc_o
        exp_[-1] += acc_e
    obs, exp_ = np.array(obs), np.array(exp_)
    chi2 = float(((obs - exp_) ** 2 / exp_).sum())
    df = len(obs) - 1 - n_params
    crit = float(stats.chi2.ppf(0.95, df)) if df > 0 else math.nan
    return {"chi2": chi2, "df": df, "chi2_crit_0.05": crit, "intervals": len(obs),
            "accepted": bool(df > 0 and chi2 < crit)}


def gamma_shifted_moments(data: np.ndarray) -> tuple[float, float, float]:
    mean, std = float(data.mean()), float(data.std(ddof=1))
    skew = max(float(stats.skew(data)), 1e-3)
    k = (2.0 / skew) ** 2
    theta = std / np.sqrt(k)
    loc = min(mean - k * theta, float(data.min()) - 1e-6)
    return k, loc, theta


def fit_gamma_shifted(data: np.ndarray) -> tuple:
    k0, loc0, theta0 = gamma_shifted_moments(data)
    k, loc, theta = stats.gamma.fit(data, k0, loc=loc0, scale=theta0)
    if k < 1.0 or loc >= data.min():
        return k0, loc0, theta0
    return k, loc, theta
```

```
def fit_all(data: np.ndarray, bins: int) -> list[dict]:
    rows = []
    for name, (dist, fixed, k) in CANDIDATES.items():
        if name.startswith("exponential"):
            params = (float(data.min()), float(data.mean() - data.min()))
        elif name == "gamma (shifted)":
            params = fit_gamma_shifted(data)
        else:
            params = dist.fit(data, **fixed)
        res = chi_square(data, dist, params, k, bins)
        rows.append({"law": name, "params": ", ".join(f"{v:.4g}" for v in params),
                     "params_raw": [float(v) for v in params], **res})
    rows.sort(key=lambda r: r["chi2"])
    return rows


def run(params: Params, reps: int, t_end: float, t_warmup: float, jobs: int,
        bins: int, thin: int, max_n: int = 0) -> dict:
    _, sojourns = replicate(params, reps, t_end, t_warmup, jobs=jobs, keep_sojourn=True)
    data = np.concatenate([np.asarray(s[::thin], dtype=float) for s in sojourns])
    if max_n and len(data) > max_n:
        data = data[np.linspace(0, len(data) - 1, max_n).astype(int)]
    counts, edges = histogram(data, bins)
    hist = [{"left": float(edges[i]), "right": float(edges[i + 1]), "count":
int(counts[i]),
             "freq": float(counts[i] / len(data))} for i in range(len(counts))]
    describe = {
        "n": int(len(data)), "mean": float(data.mean()), "std": float(data.std(ddof=1)),
        "cv": float(data.std(ddof=1) / data.mean()), "min": float(data.min()),
        "p50": float(np.quantile(data, 0.5)), "p90": float(np.quantile(data, 0.9)),
        "p95": float(np.quantile(data, 0.95)), "max": float(data.max()),
        "skew": float(stats.skew(data)),
    }
    return {"describe": describe, "histogram": hist, "fits": fit_all(data, bins), "data":
data}
```

В.8 Модуль improvement.py – пробні запуски пропозицій

```
from __future__ import annotations

from ..params import DEDICATED, NEAREST, Params
from .common import replicate, summarize

SCENARIOS = [
    ("базова система", {}, {}),
    ("4 місця на M2", {"m2_capacity": 4}, {}),
    ("4-й робот на маршруті 1 (A → M1)", {"n_robots": 4}, {"dedicated_routes": (1, 2, 3,
1)}),
    ("4-й робот на маршруті 2 (M1 → M2)", {"n_robots": 4}, {"dedicated_routes": (1, 2, 3,
2)}),
    ("4-й робот на маршруті 3 (M2 → W)", {"n_robots": 4}, {"dedicated_routes": (1, 2, 3,
3)}),
    # у варіанті 1 4-й робот у комбінації – на маршруті 2, бо окремо він дає найбільший
приріст
    ("4 місця на M2 і 4-й робот на маршруті 2", {"m2_capacity": 4, "n_robots": 4},
     {"dedicated_routes": (1, 2, 3, 2)}),
]

RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
"wip_end",
             "M1_occupied", "M2_occupied", "M2_blocked", "robots_util_mean")


def run(reps: int, t_end: float, t_warmup: float, jobs: int, base: Params | None = None)
-> list[dict]:
    base = base or Params()
    rows = []
    seen: set[tuple] = set()
    for name, change, dedicated_change in SCENARIOS:
```

```
        for strategy in (DEDICATED, NEAREST):
            params = base.with_(strategy=strategy, **change,
                                **(dedicated_change if strategy == DEDICATED else {}))
            key = (strategy, params.m2_capacity, params.n_robots,
params.dedicated_routes)
            if key in seen:
                continue
            seen.add(key)
            results, _ = replicate(params, reps, t_end, t_warmup, jobs=jobs)
            input_per_hour = 3600 / base.arrival.mean
            label = name if strategy == DEDICATED else name.split(" на маршруті")[0]
            row = {"scenario": label,
                   "strategy": strategy, "m2_capacity": params.m2_capacity, "n_robots":
params.n_robots,
                   "arrival_mean": base.arrival.mean, "input_per_hour": input_per_hour}
            for k in RESPONSES:
                st = summarize([r[k] for r in results])
                row[k] = st["mean"]
                row[k + "_ci"] = st["ci"]
            row["output_input_ratio"] = row["throughput_per_hour"] / input_per_hour
            row["stable"] = row["output_input_ratio"] >= 0.98 and row["wip_end"] < 30
            rows.append(row)
    return rows
```